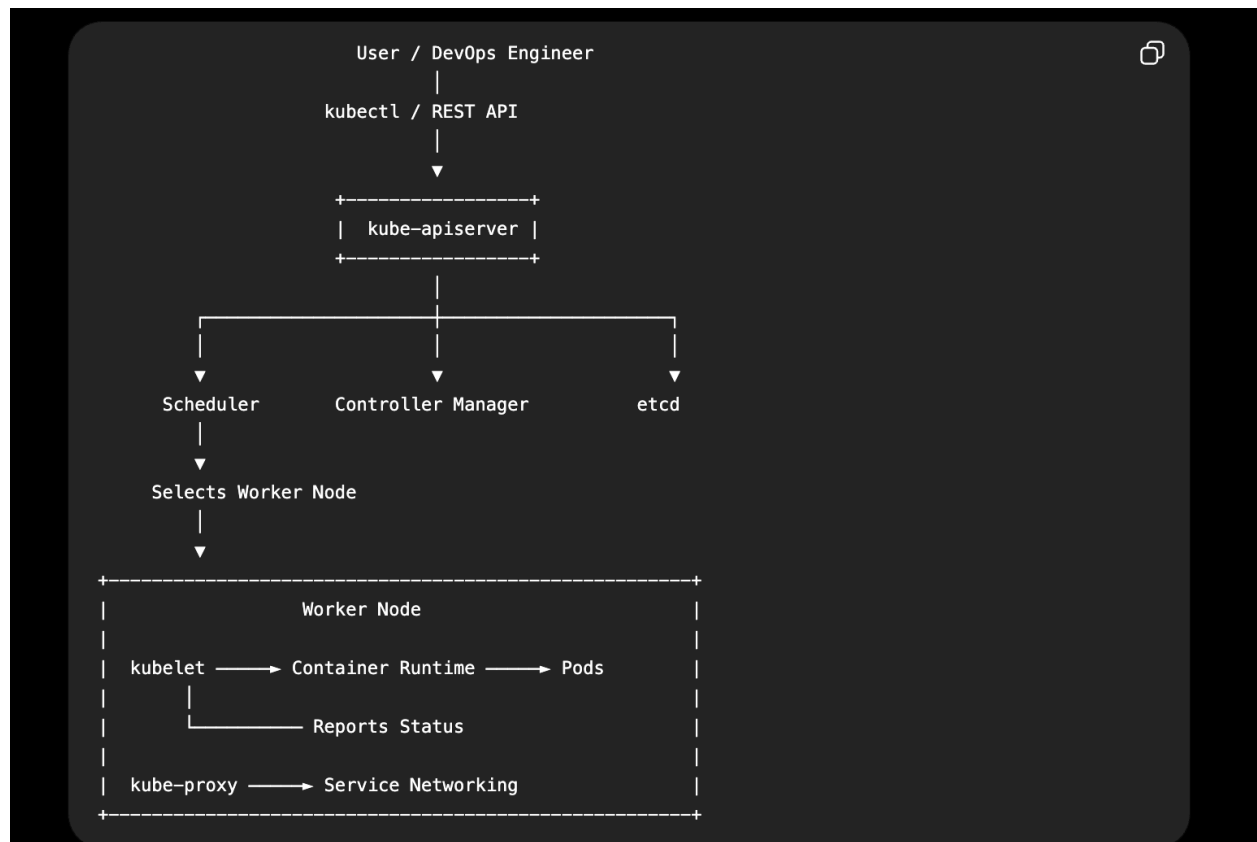


Kubernetes Architecture



What is Kubernetes?

Kubernetes is a container orchestration platform that manages containerized applications. It automates deployment, scaling, networking, load balancing, storage, and recovery of applications. Instead of manually managing containers, Kubernetes continuously ensures that applications are running as expected.

Why was Kubernetes created?

Docker can run containers, but it cannot manage applications at a large scale.

For example, suppose a company has only one application. Running a single Docker container is simple.

```
docker run nginx
```

Now imagine the company grows and has:

- Frontend
- Backend
- Payment Service
- Notification Service
- Redis
- Database

Now there are hundreds of containers running across multiple servers.

Some common problems start appearing:

- A container crashes.
- One server runs out of CPU.
- Traffic suddenly increases.
- A server goes down completely.
- Multiple application instances need load balancing.

Managing all of these manually becomes impossible.

Kubernetes was created to solve these problems automatically.

Kubernetes Architecture

Kubernetes follows a **Control Plane** and **Worker Node** architecture.

The **Control Plane** manages the entire Kubernetes cluster. It is responsible for receiving user requests, maintaining the desired state of the cluster, scheduling workloads, exposing Kubernetes APIs, and monitoring the overall health of the cluster. The Control Plane does not run application containers; instead, it manages where and how they should run.

The Control Plane consists of the following components:

kube-apiserver

The API Server is the main entry point for the Kubernetes cluster. Every request, whether from `kubectl`, REST API, or other Kubernetes components, first reaches the API Server. It authenticates and authorizes the request, validates it, and stores or retrieves cluster information from etcd. All communication within the cluster happens through the API Server.

etcd

etcd is a distributed key-value database that stores the complete state of the Kubernetes cluster. It stores information about Pods, Deployments, Services, ConfigMaps, Secrets, Nodes, and all other Kubernetes resources. Since etcd acts as the source of truth for the cluster, losing etcd data means losing the cluster state.

kube-scheduler

The Scheduler is responsible for assigning newly created Pods to the most suitable Worker Node. It evaluates available CPU, memory, node health, taints, tolerations, affinity rules, and other scheduling constraints before selecting a node. The Scheduler only decides **where** a Pod should run; it does not create or start the Pod.

kube-controller-manager

The Controller Manager runs multiple controllers that continuously compare the desired state of the cluster with the current state. If a difference is detected, it takes corrective action to restore the desired state. For example, if a Deployment requires three replicas and one Pod fails, the Controller Manager automatically creates a replacement Pod through the ReplicaSet.

The **Worker Nodes** are responsible for running the actual containerized applications. Every Worker Node receives instructions from the Control Plane and executes application workloads.

Each Worker Node contains the following components:

kubelet

kubelet is an agent that runs on every Worker Node. It communicates with the API Server, receives Pod specifications assigned to its node, pulls container images if required, starts the containers using the Container Runtime, continuously monitors their health, and reports the Pod status back to the Control Plane.

Container Runtime

The Container Runtime is responsible for downloading container images and running containers. Kubernetes supports runtimes such as **containerd** and **CRI-O**. Whenever kubelet receives instructions to start a Pod, it interacts with the Container Runtime to create and manage the containers.

kube-proxy

kube-proxy manages networking on each Worker Node. It maintains the networking rules required for Kubernetes Services and forwards incoming Service traffic to one of the healthy

backend Pods. It enables reliable communication between applications running inside the cluster.

Key Features Provided by This Architecture

Because of this architecture, Kubernetes provides several powerful capabilities:

- **Automatic Scheduling** – Pods are automatically placed on the most suitable Worker Node.
 - **Self-Healing** – Failed Pods are automatically recreated to maintain the desired state.
 - **Service Discovery** – Applications communicate using Service names instead of Pod IP addresses.
 - **Load Balancing** – Traffic is automatically distributed across multiple healthy Pods.
 - **Rolling Updates and Rollbacks** – Applications can be updated with minimal downtime and rolled back if necessary.
 - **Horizontal Auto Scaling** – The number of Pods can automatically increase or decrease based on workload.
 - **High Availability** – Applications remain available even if individual Pods or Worker Nodes fail.
-

Here's a crisp set of **notes on kube-apiserver** you can use for study, interviews, or quick recall:

Section 2: kube-apiserver (Deep Dive)

Objective: By the end of this topic, you should understand why the API Server exists, how every Kubernetes request flows through it, how it communicates with other components, and how to troubleshoot API Server issues in production.

What is kube-apiserver?

The **kube-apiserver** is the central management component of Kubernetes and acts as the **entry point** to the Kubernetes cluster.

Every operation performed in Kubernetes—whether it is creating a Pod, deleting a Deployment, scaling an application, or retrieving cluster information—is processed through the API Server.

The API Server exposes the Kubernetes REST API and is the only component allowed to directly communicate with the Kubernetes datastore (etcd).

Without the API Server, users and Kubernetes components cannot interact with the cluster.

Why do we need the API Server?

A Kubernetes cluster consists of many components such as the Scheduler, Controller Manager, kubelet, and etcd.

If every component communicated directly with every other component, the system would become difficult to manage, insecure, and tightly coupled.

Instead, Kubernetes uses the API Server as a central communication layer.

All requests are sent to the API Server, and the API Server coordinates communication between components.

This design provides:

- A single entry point for all requests.
- Authentication and authorization before any operation.
- Request validation.
- Consistent communication between Kubernetes components.
- Secure access to cluster data.

1.kube-apiserver Notes

Core Role

- Central entry point to the Kubernetes cluster.
- Exposes the Kubernetes REST API.
- Only component that talks directly to **etcd**.
- All kubectl and component requests flow through it.

Why It Exists

- Provides a single communication hub.
- Handles authentication, authorization, and validation.
- Ensures secure, consistent, decoupled communication between components.

Responsibilities

1. **Receives API requests**
(kubectl, controllers, external apps).
2. **Authentication**
(certs, tokens, OIDC, service accounts).
3. **Authorization**
(RBAC, ABAC, webhooks).
4. **Validation**
(rejects invalid configs).
5. **Stores cluster state**
in etcd.
6. **Watch mechanism**
(notifies components of changes).

Communication

- kubectl → API Server → etcd, Scheduler, Controller Manager, kubelet.
- Scheduler & controllers watch API Server, not etcd directly.
- kube-proxy updates Services based on API Server info.

Request Flow Example (`kubectl apply -f deployment.yaml`)

1. kubectl sends HTTPS request.
2. API Server authenticates & authorizes.
3. Validates YAML.
4. Stores Deployment in etcd.
5. Controller Manager creates ReplicaSet.
6. ReplicaSet creates Pods.
7. Scheduler assigns Pods to nodes.
8. kubelet starts containers.

Ports & Protocols

- Default secure port: **6443/TCP**.
- Communication: **HTTPS**.

Failure Scenario

- If API Server is down:

- kubectl commands fail.
 - No new Pods/updates.
 - Scheduler & controllers stop processing.
 - Existing Pods keep running via kubelet.
- Production: multiple API Server instances behind a load balancer for HA.

Troubleshooting

- `kubectl cluster-info` → control plane status.
- `kubectl get pods -n kube-system` → check API Server pod.
- `kubectl logs kube-apiserver-<node> -n kube-system` → inspect logs.
- `kubectl get componentstatuses` (deprecated in newer versions).

Interview Nuggets

- **Heart of Kubernetes**
→ all requests pass through it.
- **kubectl never talks to etcd directly**
.
- **Scheduler doesn't access etcd**
→ only API Server.
- **Watch mechanism**
→ efficient change notifications.

✓ **Key takeaway:** kube-apiserver is the **brainstem of Kubernetes** — everything flows through it, it enforces security and consistency, and it's the only gateway to etcd.

Excellent. Now we move to one of the **most important Kubernetes components**.

Many interviewers ask **etcd** immediately after kube-apiserver because if you understand etcd, it shows you understand how Kubernetes stores and retrieves information.

Section 3: etcd (Deep Dive)

Objective: By the end of this topic, you should understand what etcd is, why Kubernetes needs it, what data it stores, how it interacts with the API Server, and how to troubleshoot etcd issues.

What is etcd?

etcd is a distributed key-value database used by Kubernetes to store the complete state of the cluster.

Every object created inside Kubernetes, such as Pods, Deployments, Services, ConfigMaps, Secrets, Namespaces, Nodes, and ReplicaSets, is stored inside etcd.

Since etcd contains the complete cluster state, it is known as the **source of truth** for Kubernetes.

If Kubernetes needs to know the current or desired state of any resource, it retrieves that information from etcd through the kube-apiserver.

Why does Kubernetes need etcd?

A Kubernetes cluster is constantly changing.

Examples:

- A new Pod is created.
- A Deployment is scaled from 3 to 5 replicas.
- A Service is deleted.
- A Node joins the cluster.
- A Secret is updated.

Kubernetes needs a reliable place to store all of this information.

Without etcd:

- Kubernetes would not remember which Pods exist.
- The Scheduler would not know which Pods need scheduling.
- The Controller Manager would not know the desired number of replicas.
- The API Server would have nowhere to store or retrieve cluster information.

This is why etcd is one of the most critical components of the Control Plane.

Responsibilities of etcd

etcd performs several important functions within the Kubernetes cluster.

Stores Cluster State

etcd stores the complete configuration and current state of the cluster.

Examples include:

- Pods
- Deployments
- ReplicaSets
- Services
- Nodes
- Namespaces
- ConfigMaps
- Secrets
- Persistent Volumes
- Persistent Volume Claims

Stores Desired State

When a user creates a Deployment with three replicas, Kubernetes stores this desired state inside etcd.

Example:

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
```

This value is stored in etcd.

If one Pod fails later, the Controller Manager compares the current state with the desired state stored in etcd and creates a replacement Pod.

Stores Current State

etcd also stores the current state of the cluster.

For example:

Desired State:

Replicas = 3

Current State:

Running Pods = 2

This difference allows Kubernetes to detect failures and restore the desired state.

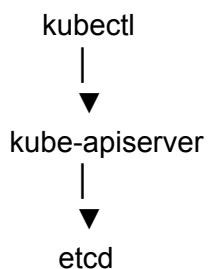
How etcd interacts with kube-apiserver

One of the most important Kubernetes concepts is:

Only the kube-apiserver communicates directly with etcd.

No other Kubernetes component accesses etcd directly.

The communication flow looks like this:



Other components such as:

- Scheduler
- Controller Manager
- kubelet

communicate only with the API Server.

This architecture provides:

- Better security
 - Centralized validation
 - Consistent communication
 - Easier maintenance
-

What happens when we create a Deployment?

Suppose we execute:

```
kubectl apply -f deployment.yaml
```

The sequence is:

Step 1

kubectl sends the request to the kube-apiserver.

Step 2

The API Server validates the Deployment.

Step 3

The API Server stores the Deployment object inside etcd.

At this point, etcd contains information such as:

- Deployment Name
 - Namespace
 - Replica Count
 - Labels
 - Container Image
 - Resource Requests
-

Step 4

The Controller Manager watches the API Server.

It notices a new Deployment has been created.

Step 5

The Controller Manager creates a ReplicaSet.

Step 6

The ReplicaSet creates Pods.

Step 7

The Scheduler assigns Pods to Worker Nodes.

Everything begins because the Deployment was successfully stored in etcd.

Does etcd store application data?

No.

This is one of the most common interview questions.

etcd stores **Kubernetes metadata**, not your application's business data.

Example:

Suppose your application is MySQL.

The customer table:

Customers

Orders

Products

is stored inside MySQL.

etcd does **not** store these records.

Instead, etcd stores Kubernetes information such as:

MySQL Pod

MySQL Service

PVC

Deployment

Secret

Distributed Database

etcd is a distributed database.

In production environments, multiple etcd nodes are usually deployed together.

Example:

etcd-1

etcd-2

etcd-3

Each node replicates the same data.

This provides:

- High Availability
- Fault Tolerance
- Data Consistency

If one etcd node fails, the remaining nodes continue serving requests as long as a majority (quorum) is available.

What happens if etcd goes down?

If etcd becomes unavailable:

- The API Server cannot store new objects.
- The API Server cannot retrieve cluster information.
- New Pods cannot be created.
- Deployments cannot be updated.
- Services cannot be modified.
- Cluster state cannot be changed.

However,

already running Pods usually continue running on Worker Nodes because kubelet keeps managing them locally.

The main impact is that **cluster management stops** until etcd becomes available again.

Troubleshooting etcd

If etcd is suspected to be unhealthy, an SRE may:

Check the etcd Pod:

kubectl get pods -n kube-system

Example:

etcd-control-plane

View logs:

kubectl logs etcd-control-plane -n kube-system

Check cluster health (using `etcdctl` on the control plane):

`etcdctl endpoint health`

Check endpoint status:

`etcdctl endpoint status`

These commands verify whether the etcd members are healthy and able to serve requests.

Common Interview Questions

1. Why is etcd called the source of truth?

Because it stores the complete desired and current state of the Kubernetes cluster. All cluster management decisions are based on the information stored in etcd.

2. Can kubectl communicate directly with etcd?

No.

kubectl always communicates with the kube-apiserver.

3. Can the Scheduler read data directly from etcd?

No.

The Scheduler communicates only with the kube-apiserver.

4. Does etcd store application data?

No.

It stores Kubernetes cluster metadata and configuration, not application business data.

5. Why is etcd deployed as multiple nodes in production?

To provide High Availability, fault tolerance, and protect against data loss if one member fails.

Important Points to Remember

- etcd is a distributed key-value database.
 - It stores the complete state of the Kubernetes cluster.
 - etcd is the **source of truth** for Kubernetes.
 - Only the kube-apiserver communicates directly with etcd.
 - etcd stores Kubernetes objects, not application business data.
 - Production clusters usually run multiple etcd members for High Availability.
 - If etcd becomes unavailable, existing workloads may continue running, but Kubernetes cannot reliably process new management operations.
-

Before moving to kube-scheduler, here's an interview tip.

A lot of candidates simply say:

"etcd is the Kubernetes database."

That's correct, but it's not enough for an advanced interview.

A stronger answer is:

"etcd is a distributed key-value database that stores the complete desired and current state of the Kubernetes cluster. It acts as the source of truth for Kubernetes. The kube-apiserver is the only component that communicates directly with etcd, while other control plane components access cluster information through the API Server."

That answer demonstrates both understanding of etcd's role and its interaction with the rest of the control plane.

That's great to hear. And I noticed something.

Section 4: Scheduler (Deep Dive)

Objective: By the end of this topic, you should understand how the Scheduler works, how it selects a Worker Node, what factors it considers, and how to troubleshoot scheduling problems.

What is kube-scheduler?

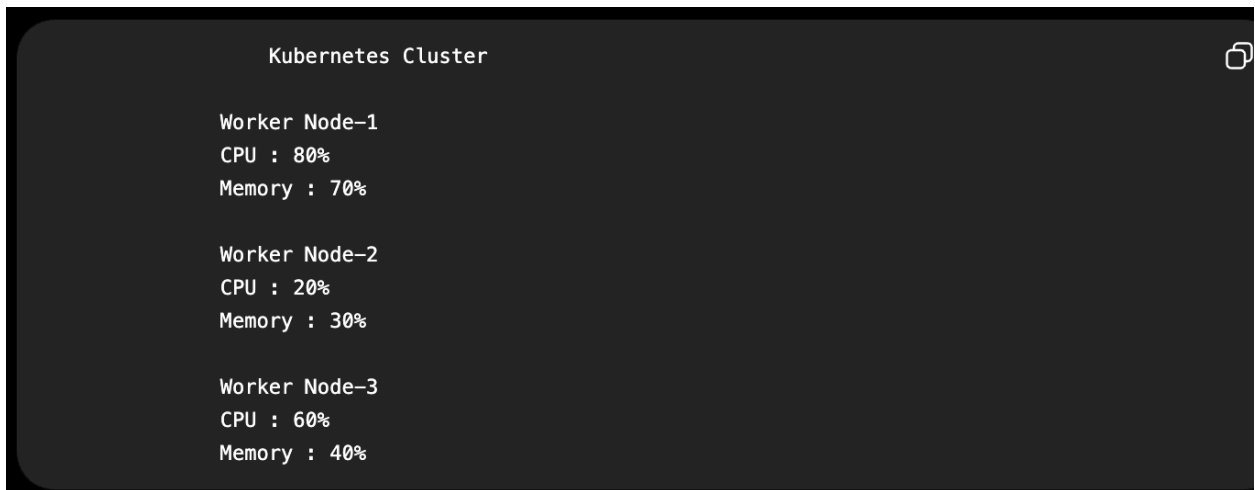
The **kube-scheduler** is a Control Plane component responsible for assigning newly created Pods to the most appropriate Worker Node.

When a Pod is created, Kubernetes knows **what** needs to run, but it does not know **where** it should run. The Scheduler solves this problem by selecting the best available Worker Node based on resource availability and scheduling rules.

The Scheduler only assigns Pods to Nodes. It does **not** create Pods or start containers.

Why does Kubernetes need a Scheduler?

Consider a Kubernetes cluster with three Worker Nodes.



Kubernetes Cluster	
Worker Node-1	
CPU : 80%	
Memory : 70%	
Worker Node-2	
CPU : 20%	
Memory : 30%	
Worker Node-3	
CPU : 60%	
Memory : 40%	

Now suppose you create a Deployment with three replicas.

replicas: 3

At this stage, Kubernetes has three Pods that need to run.

The question is:

- Which Node should run Pod-1?
- Which Node should run Pod-2?
- Which Node should run Pod-3?

Without a Scheduler, Pods would never know where to execute.

The Scheduler continuously solves this placement problem.

Responsibilities of kube-scheduler

The Scheduler performs the following responsibilities.

Watches for Unscheduled Pods

The Scheduler continuously watches the API Server for Pods that have not yet been assigned to any Worker Node.

These Pods are typically in the **Pending** state.

Evaluates Available Worker Nodes

For every Pending Pod, the Scheduler collects information about all Worker Nodes.

It evaluates:

- Available CPU
 - Available Memory
 - Node Health
 - Node Capacity
 - Scheduling Policies
-

Selects the Best Worker Node

After evaluating all available nodes, the Scheduler selects the most suitable Worker Node for the Pod.

The Scheduler then updates the Pod object through the API Server with the selected Node.

Does Not Start Containers

Many beginners think the Scheduler starts Pods.

It does **not**.

After selecting a Worker Node, the Scheduler's job is complete.

The kubelet running on that Worker Node starts the Pod.

How kube-scheduler Works

Suppose you execute:

```
kubectl apply -f deployment.yaml
```

The sequence is:

Step 1

The Deployment is stored in etcd through the API Server.

Step 2

The Controller Manager creates three Pods.

At this stage:

Pod-1 → Pending

Pod-2 → Pending

Pod-3 → Pending

No Worker Node has been selected.

Step 3

The Scheduler watches the API Server.

It notices three unscheduled Pods.

Step 4

The Scheduler evaluates every Worker Node.

Example:

Node-A → CPU Free : 500m → Memory Free : 1GB

Node-B → CPU Free : 2 CPU → Memory Free : 6GB

Node-C → CPU Free : 200m → Memory Free : 512MB

Step 5

Suppose each Pod requires:

resources:

requests:

cpu: 500m

memory: 512Mi

The Scheduler determines:

- Node-A can run the Pod.
- Node-B can also run the Pod.
- Node-C does not have enough CPU.

It then selects the most suitable Node.

Example:

Pod-1 → Node-B

Step 6

The Scheduler updates the Pod object through the API Server.

The Pod now contains:

Node Name = Worker Node-B

Step 7

The kubelet running on Worker Node-B notices:

"A new Pod has been assigned to me."

The kubelet then:

- Pulls the image
- Creates the container
- Starts the Pod

The Scheduler's work is now finished.

Communication Flow

kubectl

|



API Server → Controller Manager → Creates Pods → Pending Pods → Scheduler

|



Updates Pod with Selected Node → kubelet → Container Runtime → Running Pod

What Factors Does the Scheduler Consider?

The Scheduler does not randomly pick a Node.

It evaluates several factors before making a decision.

Available CPU

If a Node does not have sufficient CPU for the Pod's resource request, it is not selected.

Available Memory

Nodes with insufficient memory are excluded.

Node Status

Only healthy Worker Nodes are considered.

Nodes marked as **NotReady** are ignored.

Taints and Tolerations

If a Node has a taint and the Pod does not tolerate it, that Node is skipped.

Node Affinity

If the Pod specifies Node Affinity rules, only matching Nodes are considered.

Resource Requests

The Scheduler always checks the Pod's requested resources before assigning it to a Node.

What Happens If No Node Can Run the Pod?

Suppose every Worker Node has insufficient CPU.

The Scheduler cannot assign the Pod.

The Pod remains in:

Pending

If you run:

```
kubectl describe pod nginx
```

The Events section may show:

FailedScheduling

0/3 nodes are available

Insufficient CPU

The Scheduler keeps retrying until resources become available.

Troubleshooting Scheduling Problems

When a Pod remains in the **Pending** state, an SRE should investigate why the Scheduler could not assign it.

Useful commands:

```
kubectl get pods
```

Check whether the Pod is in the **Pending** state.

```
kubectl describe pod <pod-name>
```

Review the **Events** section.

Common scheduling errors include:

- Insufficient CPU
- Insufficient Memory
- Node NotReady
- Taints
- Affinity mismatch
- Unsatisfied Persistent Volume Claims

```
kubectl get nodes
```

Verify that Worker Nodes are in the **Ready** state.

```
kubectl top nodes
```

Check current CPU and memory usage to identify resource exhaustion.

Common Interview Questions

1. What is the responsibility of the Scheduler?

The Scheduler assigns newly created Pods to the most appropriate Worker Node. It evaluates available resources and scheduling rules before making a placement decision.

2. Does the Scheduler create Pods?

No.

Pods are created by the Controller Manager (through ReplicaSets or other controllers).

The Scheduler only assigns Pods to Worker Nodes.

3. Does the Scheduler start containers?

No.

After selecting a Worker Node, the kubelet starts the containers.

4. What happens if the Scheduler cannot find a suitable Node?

The Pod remains in the **Pending** state until a suitable Worker Node becomes available.

5. Which Kubernetes component starts the Pod after scheduling?

The kubelet on the selected Worker Node starts the Pod using the Container Runtime.

Important Points to Remember

- The Scheduler only assigns Pods to Worker Nodes.
 - It watches for Pods that have not yet been scheduled.
 - It evaluates CPU, memory, node health, taints, affinity, and resource requests.
 - It updates the Pod with the selected Node through the API Server.
 - It never starts containers or manages Pods after scheduling.
 - If no suitable Node is available, the Pod remains in the **Pending** state.
 - The kubelet is responsible for starting the Pod after scheduling.
-

★ SRE Interview Tip

Many candidates say:

"The Scheduler selects a Node."

A stronger answer is:

"The Scheduler watches for newly created Pods that don't yet have a node assignment. It evaluates all available Worker Nodes based on resource availability and scheduling constraints such as CPU, memory, taints, tolerations, affinity, and resource requests. After selecting the most suitable Worker Node, it updates the Pod through the API Server. The kubelet on that node then starts the Pod."

That's the kind of explanation that demonstrates **understanding**, not just memorization.

Next: kube-controller-manager. This is where Kubernetes' **self-healing** behavior comes from, and it's one of the most important concepts in production troubleshooting.

Awesome! You're building a solid foundation. So far, you've learned:

-  Kubernetes Architecture
-  kube-apiserver
-  etcd
-  kube-scheduler

Now we'll learn the component that makes Kubernetes **self-healing**.

Section 5: kube-controller-manager (Deep Dive)

Objective: By the end of this topic, you should understand why the Controller Manager exists, how it continuously maintains the desired state of the cluster, and why Kubernetes can automatically recover from failures.

What is kube-controller-manager?

The **kube-controller-manager** is a Control Plane component that continuously monitors the cluster and ensures the **current state matches the desired state**.

When the actual state of the cluster differs from what is stored in etcd, the Controller Manager takes corrective actions to bring the cluster back to the desired state.

It is responsible for Kubernetes features such as:

- Self-healing
- Replica management
- Node monitoring
- Endpoint updates
- Job execution

The Controller Manager does **not** run applications. Instead, it continuously checks the cluster and takes corrective actions whenever necessary.

Why do we need the Controller Manager?

Suppose you create a Deployment:

replicas: 3

The desired state is:

Running Pods = 3

After some time:

Pod-2 crashes.

Current state becomes:

Running Pods = 2

Desired state:

Running Pods = 3

There is now a mismatch.

Without the Controller Manager, Kubernetes would simply continue running with two Pods.

Instead, the Controller Manager detects the difference and creates a replacement Pod to restore the desired state.

This continuous reconciliation is what makes Kubernetes self-healing.

Responsibilities of kube-controller-manager

The Controller Manager runs multiple controllers, each responsible for a specific Kubernetes resource.

Some important controllers are:

- Deployment Controller
- ReplicaSet Controller
- Node Controller
- Job Controller
- Endpoint Controller
- Service Account Controller

Each controller watches a specific type of resource through the API Server.

How does the Controller Manager work?

The Controller Manager follows a continuous loop called the **Reconciliation Loop**.

It repeatedly performs the following steps:

1. Read the desired state from the API Server.
2. Read the current state from the API Server.
3. Compare the two states.
4. If they are different, take corrective action.
5. Repeat continuously.

This loop runs throughout the lifetime of the cluster.

Example: Self-Healing

Suppose you deploy an application:

replicas: 3

Initially:

Desired State : 3 Pods

Current State : 3 Pods

Everything is healthy.

Now suppose one Worker Node crashes.

The Pod running on that node also disappears.

Current state:

Desired State : 3 Pods

Current State : 2 Pods

The ReplicaSet Controller detects this mismatch.

It creates a new Pod object.

The Scheduler assigns the new Pod to another healthy Worker Node.

The kubelet starts the Pod.

Final state:

Desired State : 3 Pods

Current State : 3 Pods

Communication Flow (Horizontal)

kubectl → API Server → etcd (Desired State)

↓

Controller Manager → Compares Desired vs Current State



Mismatch Found?



Create/Delete/Update Resource



API Server → Scheduler → kubelet → Running Pod

What is a Controller?

A **Controller** is a process that watches a specific Kubernetes resource and continuously works to ensure its actual state matches the desired state.

Instead of one large program managing everything, Kubernetes uses many specialized controllers.

Each controller has a specific responsibility.

Important Controllers

1. ReplicaSet Controller

Responsible for maintaining the correct number of Pod replicas.

Example:

Desired:

Replicas = 5

Current:

Replicas = 4

Action:

Create 1 new Pod

2. Deployment Controller

Responsible for Deployments.

It manages:

- Rolling Updates
- Rollbacks
- ReplicaSets

When you update a Deployment image:

image: nginx:1.30

The Deployment Controller creates a new ReplicaSet and gradually replaces old Pods with new ones.

3. Node Controller

Continuously monitors Worker Nodes.

Example:

Worker-1 = Ready

Worker-2 = Ready

Worker-3 = NotReady

If a node remains unavailable for a configured period, the Node Controller marks it **NotReady** and begins recovery actions, such as evicting Pods so they can be recreated on healthy nodes (subject to workload configuration).

4. Job Controller

Ensures a Job completes successfully.

If a Job Pod fails before completion, the Job Controller creates another Pod until the Job succeeds or reaches its retry limit.

5. Endpoint Controller

Updates Service endpoints whenever Pods are added or removed.

This ensures Services send traffic only to healthy backend Pods.

What happens if the Controller Manager stops?

If the Controller Manager becomes unavailable:

- Failed Pods are not recreated.
- Deployments stop reconciling.
- Jobs may not be retried.
- Node failures are not processed.
- Self-healing stops.

Existing applications generally continue running, but Kubernetes stops automatically correcting cluster state.

Troubleshooting Controller Manager

Check the Control Plane Pods:

```
kubectll get pods -n kube-system
```

Look for:

kube-controller-manager-control-plane

View logs:

```
kubectll logs kube-controller-manager-control-plane -n kube-system
```

If Pods are not being recreated after failures, investigate:

- Controller Manager health
- ReplicaSet status
- Deployment status
- Events

Useful commands:

```
kubectll get deployments
```

```
kubectll get replicaset
```

```
kubectll describe deployment <deployment-name>
```

```
kubectll get events
```

Common Interview Questions

1. What is the responsibility of the Controller Manager?

It continuously compares the desired state with the current state of the cluster and takes corrective actions to keep them in sync.

2. Does the Controller Manager schedule Pods?

No.

The Scheduler selects Worker Nodes.

The Controller Manager creates or updates Kubernetes resources.

3. Which Kubernetes feature is mainly provided by the Controller Manager?

Self-healing through continuous reconciliation.

4. What is the Reconciliation Loop?

It is the continuous process of comparing the desired state with the current state and correcting any differences.

5. If a Pod crashes, which components are involved?

ReplicaSet Controller → Creates New Pod → API Server → Scheduler → kubelet → Running Pod

Important Points to Remember

- The Controller Manager is responsible for **maintaining the desired state** of the cluster.
 - It runs multiple specialized controllers, each managing a different Kubernetes resource.
 - It continuously performs a **Reconciliation Loop**.
 - It enables Kubernetes features such as **self-healing**, **replica management**, and **rolling updates**.
 - It does **not** schedule Pods or run containers.
-

★ SRE Interview Tip

A common mistake is saying:

"The Controller Manager manages the cluster."

A stronger answer is:

"The kube-controller-manager runs multiple controllers that continuously reconcile the desired state stored in the cluster with the current state. If there's a difference—for example, a Pod crashes while a Deployment requires three replicas—it creates or updates Kubernetes resources through the API Server so the Scheduler and kubelet can restore the desired state."

This explanation demonstrates that you understand not just **what** it does, but **how** it collaborates with the other Control Plane components.

Excellent. You've now completed the entire **Control Plane** except one important component: **Cloud Controller Manager** (used mainly in cloud environments like AWS, Azure, GCP). We'll cover that later because Akamai interviews usually focus first on the core components.

So far we've covered:

-  Kubernetes Architecture
-  kube-apiserver
-  etcd
-  kube-scheduler
-  kube-controller-manager

Now let's move to the **Worker Node**, where your applications actually run.

Section 6: Worker Node (Deep Dive)

Objective: By the end of this topic, you should understand what a Worker Node is, why Kubernetes separates the Control Plane from Worker Nodes, and how a Pod runs on a Worker Node.

What is a Worker Node?

A **Worker Node** is a machine (physical or virtual) where containerized applications actually run.

The Control Plane manages the cluster, while Worker Nodes execute the workloads.

Every Pod you create in Kubernetes eventually runs on a Worker Node.

Why do we need Worker Nodes?

The Control Plane is responsible for making decisions:

- Accepting requests
- Storing cluster state
- Scheduling Pods
- Maintaining desired state

However, it does **not** run your applications.

Running application workloads on the Control Plane would consume resources needed for cluster management and reduce reliability.

Therefore, Kubernetes separates responsibilities:

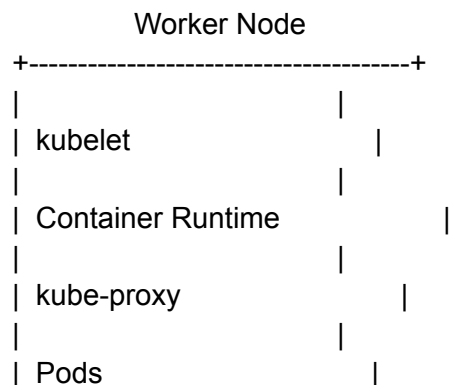
- **Control Plane** → Makes decisions
- **Worker Nodes** → Execute applications

This separation improves scalability, performance, and reliability.

Components of a Worker Node

Every Worker Node contains the following major components:

- kubelet
- Container Runtime
- kube-proxy





We'll study each component in detail in the coming sections.

Responsibilities of a Worker Node

A Worker Node is responsible for:

- Running Pods
- Running Containers
- Pulling container images
- Reporting Node and Pod status
- Forwarding network traffic
- Managing local storage attached to Pods

It does **not** decide where Pods should run. That decision is made by the Scheduler.

How does a Worker Node receive work?

Suppose you execute:

```
kubectl apply -f deployment.yaml
```

The flow is:

1. API Server accepts the request.
 2. etcd stores the Deployment.
 3. Controller Manager creates Pod objects.
 4. Scheduler selects a Worker Node.
 5. The Scheduler updates the Pod with the selected Node.
 6. The kubelet on that Worker Node detects the new assignment.
 7. The kubelet starts the Pod using the Container Runtime.
-

Communication Flow (Horizontal)

kubectl → API Server → etcd → Controller Manager → Scheduler → Worker Node → kubelet
→ Container Runtime → Running Pod

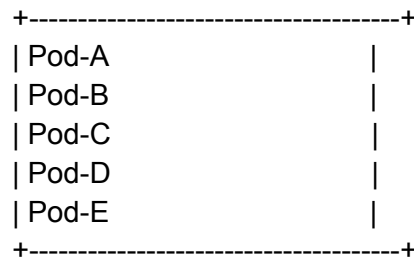
Can one Worker Node run multiple Pods?

Yes.

A Worker Node can run many Pods depending on its available CPU, memory, storage, and other resources.

Example:

Worker Node-1



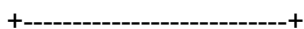
The Scheduler decides how Pods are distributed across Worker Nodes.

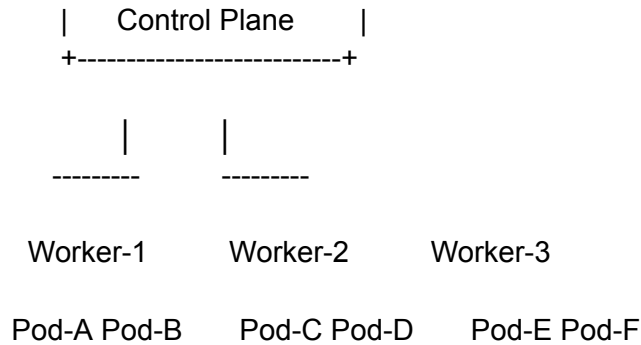
Can a cluster have multiple Worker Nodes?

Absolutely.

Example:

Kubernetes Cluster





Adding more Worker Nodes allows Kubernetes to run more applications and handle higher workloads.

What happens if a Worker Node fails?

Suppose Worker Node-2 crashes.

Worker-2

Pod-C

Pod-D

✖ Node Failure

The sequence is:

1. The Node Controller detects the node is **NotReady**.
2. The Controller Manager updates the cluster state.
3. Pods on the failed node become unavailable.
4. Replacement Pods are created (if managed by a Deployment, ReplicaSet, etc.).
5. The Scheduler assigns those new Pods to healthy Worker Nodes.
6. kubelet on the selected nodes starts the new Pods.

This is how Kubernetes provides **self-healing**.

Troubleshooting Worker Nodes

Check node status:

```
kubectl get nodes
```

Example:

NAME	STATUS
worker-1	Ready
worker-2	Ready
worker-3	NotReady

Get detailed information:

```
kubectl describe node worker-1
```

This shows:

- CPU capacity
 - Memory
 - Conditions
 - Allocated resources
 - Running Pods
 - Events
-

List Pods on a specific node:

```
kubectl get pods -o wide
```

This displays which Worker Node each Pod is running on.

Common Interview Questions

1. What is a Worker Node?

A Worker Node is a machine that runs containerized applications. It hosts Pods and includes components such as kubelet, the Container Runtime, and kube-proxy.

2. Does a Worker Node schedule Pods?

No.

The Scheduler selects the Worker Node.

The Worker Node only runs the assigned Pods.

3. What components are present on a Worker Node?

- kubelet
 - Container Runtime
 - kube-proxy
-

4. Can a Worker Node run multiple Pods?

Yes.

The number depends on the available resources and scheduling decisions.

5. What happens when a Worker Node fails?

The Node Controller marks it **NotReady**. Workloads managed by controllers such as Deployments are recreated on healthy Worker Nodes through the Controller Manager, Scheduler, and kubelet.

Important Points to Remember

- A Worker Node runs your applications.
- It contains kubelet, a Container Runtime, and kube-proxy.
- It does not make scheduling decisions.

- Multiple Pods can run on a single Worker Node.
 - Multiple Worker Nodes form a Kubernetes cluster.
 - If a Worker Node fails, Kubernetes can recreate managed workloads on healthy nodes.
-

★ SRE Interview Tip

A common answer is:

"A Worker Node runs Pods."

A stronger answer is:

"A Worker Node is the execution environment for Kubernetes workloads. It receives Pod assignments from the Scheduler through the API Server. The kubelet on the node communicates with the Container Runtime to create and manage containers, while kube-proxy handles network traffic for Services. A Worker Node focuses on executing workloads, whereas the Control Plane manages the cluster."

What's Next?

We'll now deep dive into the **kubelet**, which is arguably the **most important component on a Worker Node**.

You'll learn:

- Why kubelet is called the "node agent"
- How it watches for assigned Pods
- How it talks to the API Server
- How it uses the Container Runtime
- Health checks (liveness, readiness, startup)
- Static Pods
- Node registration
- Production troubleshooting
- Common interview questions

Once you understand **kubelet**, the entire Pod lifecycle on a Worker Node will become much clearer.

Perfect. Now we're entering one of the **most important Kubernetes topics**. If I had to rank Kubernetes components for an SRE interview, **kubelet would be in the top three**.

Everything the Scheduler decides is ultimately carried out by the kubelet.

Section 7: kubelet (Deep Dive)

Objective: By the end of this topic, you should understand how the kubelet works, how it communicates with the API Server and Container Runtime, how it manages Pods, and how to troubleshoot kubelet-related issues.

What is kubelet?

The **kubelet** is the primary agent running on every Worker Node.

Its responsibility is to ensure that the Pods assigned to its Worker Node are running exactly as defined in the Pod specification.

The kubelet continuously communicates with the Kubernetes API Server, monitors the Pods running on its node, and works with the Container Runtime to create, monitor, and terminate containers.

Unlike the Scheduler, which decides **where** a Pod should run, the kubelet ensures that the assigned Pod actually runs.

Why do we need kubelet?

Consider this situation.

The Scheduler has assigned a Pod to **Worker Node-2**.

Pod nginx

↓

Assigned to Worker Node-2

At this point, the Pod is only a Kubernetes object stored in the API Server.

Nothing is running yet.

Someone on Worker Node-2 must:

- Read the Pod specification
- Pull the container image
- Create the container
- Start the Pod
- Monitor the Pod
- Restart it if required
- Report status back to the API Server

That responsibility belongs to the **kubelet**.

Without the kubelet, Pods would never start.

Responsibilities of kubelet

The kubelet performs several critical functions.

1. Registers the Worker Node

When a Worker Node starts, the kubelet registers the node with the API Server.

Example:

Worker Node Starts

↓

kubelet Starts

↓

Registers Node

↓

Node appears in Kubernetes Cluster

This is why running:

```
kubectl get nodes
```

shows the Worker Node.

2. Watches for Assigned Pods

The kubelet continuously watches the API Server.

When the Scheduler assigns a Pod to its node:

Node Name = Worker-2

the kubelet detects the assignment.

3. Starts Containers

The kubelet does **not** create containers directly.

Instead, it sends instructions to the **Container Runtime**.

Example:

kubelet

↓

Container Runtime

↓

Create Container

↓

Run Container

4. Monitors Pod Health

The kubelet continuously checks:

- Are the containers running?
- Did a container exit unexpectedly?
- Did a health check fail?

If necessary, it restarts containers according to the Pod's restart policy.

5. Reports Status

The kubelet periodically reports information to the API Server, including:

- Node status
- Pod status
- Container status
- Resource usage (basic status; metrics are commonly collected by additional components such as Metrics Server)

This keeps the Control Plane informed about the state of the Worker Node.

How kubelet Works

Suppose you execute:

```
kubectl apply -f nginx.yaml
```

The sequence is:

Step 1

The API Server stores the Pod.

Step 2

The Scheduler assigns the Pod to Worker Node-1.

Step 3

The kubelet on Worker Node-1 detects that a Pod has been assigned to it.

Step 4

The kubelet reads the Pod specification.

Example:

image: nginx:latest

Step 5

The kubelet instructs the Container Runtime:

"Pull this image and create the container."

Step 6

The Container Runtime downloads the image if needed.

Step 7

The Container Runtime starts the container.

Step 8

The kubelet continuously monitors the Pod and reports its status back to the API Server.

Communication Flow (Horizontal)

kubectl → API Server → Scheduler → Worker Node → kubelet → Container Runtime → Pull Image → Create Container → Running Pod → kubelet Reports Status → API Server

kubelet and Container Runtime

One common interview question is:

Does kubelet run containers?

No.

The kubelet manages Pods, but the **Container Runtime** actually creates and runs the containers.

Think of their responsibilities like this:

Component	Responsibility
kubelet	Manages Pods and communicates with the API Server
Container Runtime	Pulls images, creates containers, starts containers, stops containers

kubelet and Health Checks

The kubelet performs health checks defined in the Pod specification.

The three probe types are:

Liveness Probe

Checks whether the application is still running correctly.

If it fails repeatedly, the kubelet restarts the container.

Readiness Probe

Checks whether the application is ready to receive traffic.

If it fails, the Pod stays running, but Kubernetes removes it from Service endpoints until it becomes ready again.

Startup Probe

Used for applications that take a long time to start.

It delays liveness and readiness checks until startup is complete.

We'll cover probes in detail later.

What happens if kubelet stops?

Suppose the kubelet process crashes.

Effects include:

- New Pods assigned to the node won't start.
- Pod status is no longer reported to the API Server.
- Health checks stop running.
- Restart actions stop.
- Eventually, the Node Controller detects the node is unhealthy and marks it **NotReady**.

Existing containers may continue running for a while because they are managed by the Container Runtime, but Kubernetes loses active management of that node.

Static Pods

Another important kubelet responsibility is managing **Static Pods**.

Normally:

API Server

↓

Pod

↓

kubelet

For Static Pods:

Manifest File on Worker/Control Plane Node

↓

kubelet

↓

Static Pod

The kubelet reads manifest files from a local directory (commonly `/etc/kubernetes/manifests` on control plane nodes) and creates Pods directly from them.

Static Pods are commonly used for Control Plane components in self-managed clusters.

Troubleshooting kubelet

Check the kubelet service:

```
systemctl status kubelet
```

View kubelet logs:

```
journalctl -u kubelet
```

Verify Node status:

```
kubectl get nodes
```

If the node is **NotReady**, investigate the kubelet first.

Describe the Node:

```
kubectl describe node worker-1
```

Review the **Conditions** and **Events** sections.

Common Interview Questions

1. What is kubelet?

The kubelet is the node agent that runs on every Worker Node. It ensures Pods assigned to the node are created, monitored, and kept running according to their specifications.

2. Does kubelet schedule Pods?

No.

Scheduling is performed by the Scheduler.

3. Does kubelet create containers?

No.

The Container Runtime creates and runs containers. The kubelet instructs it what to do.

4. Which component reports Pod status to the API Server?

The kubelet.

5. What happens if kubelet stops?

The node stops being actively managed by Kubernetes. New Pods won't start, health checks stop, and the Node Controller eventually marks the node as **NotReady**.

Important Points to Remember

- kubelet runs on every Worker Node.
 - It registers the Worker Node with the API Server.
 - It watches for Pods assigned to its node.
 - It communicates with the Container Runtime to run containers.
 - It performs health checks and reports status to the API Server.
 - It manages Static Pods.
 - It does **not** schedule Pods or directly create containers.
-

★ SRE Interview Tip

A common answer is:

"kubelet runs Pods."

A stronger answer is:

"The kubelet is the node agent responsible for managing Pods on a Worker Node. It watches the API Server for Pods assigned to its node, communicates with the Container Runtime to pull images and start containers, continuously monitors Pod health, and reports node and Pod status back to the API Server. It also manages Static Pods and executes health probes."

That answer demonstrates a clear understanding of the kubelet's role within the Kubernetes architecture.

Next Topic

We'll cover **Container Runtime** (containerd, CRI-O, Docker's historical role, CRI, image pulling, container lifecycle, and runtime troubleshooting). Once you understand that, you'll know exactly **how a Pod becomes a running process on a Linux machine**.

Excellent. You're now at the point where you'll understand **how Kubernetes actually runs containers**. Many people know Pods and Deployments but don't understand what happens underneath. This topic connects Kubernetes with Linux.

Section 8: Container Runtime (Deep Dive)

Objective: By the end of this topic, you should understand what a Container Runtime is, why Kubernetes needs it, how it creates containers, what CRI is, and how to troubleshoot runtime issues.

What is a Container Runtime?

A **Container Runtime** is the software responsible for running containers on a Worker Node.

The kubelet does **not** create containers itself. Instead, it sends instructions to the Container Runtime, which performs the actual work of pulling images, creating containers, starting them, stopping them, and deleting them.

Examples of Container Runtimes include:

- **containerd** (Most commonly used today)
 - **CRI-O**
 - **Docker Engine** (historically used with Kubernetes, but not as the runtime interface in modern Kubernetes)
-

Why do we need a Container Runtime?

Suppose the Scheduler assigns a Pod to Worker Node-1.

The kubelet receives the Pod specification:

containers:
- name: nginx
 image: nginx:1.27

The kubelet understands **what** needs to run, but it doesn't know **how** to:

- Download the image
- Create Linux namespaces
- Configure cgroups
- Start the container process
- Stop the container
- Delete the container

These low-level container operations are performed by the Container Runtime.

Without a Container Runtime, Kubernetes could schedule Pods, but no containers would actually run.

Responsibilities of the Container Runtime

The Container Runtime performs several important tasks.

1. Pull Container Images

If the required image is not available locally, it downloads it from the configured container registry.

Example:

Registry → Worker Node → Image Downloaded

2. Create Containers

After downloading the image, it creates the container by setting up:

- Linux namespaces
- cgroups
- Filesystem
- Networking

- Process isolation
-

3. Start Containers

The runtime starts the application's main process inside the container.

Example:

nginx container

↓

nginx process starts

↓

Container Running

4. Stop Containers

When a Pod is deleted or restarted, the runtime stops the container gracefully and cleans up resources.

5. Remove Containers

After the container stops, the runtime removes:

- Container metadata
 - Temporary filesystem
 - Networking configuration
-

How the Container Runtime Works

Suppose you run:

```
kubectl apply -f nginx.yaml
```

The sequence is:

Step 1

API Server stores the Pod.

Step 2

Scheduler assigns the Pod to Worker Node-1.

Step 3

The kubelet detects the Pod assignment.

Step 4

The kubelet sends a request to the Container Runtime.

Example:

Create a container using image nginx:1.27

Step 5

The Container Runtime checks:

Is the image already available locally?

If **Yes**:

Start container immediately.

If **No**:

Download image from registry.

Step 6

The Container Runtime creates the container.

Step 7

The application process starts.

Step 8

The kubelet monitors the running Pod.

Communication Flow (Horizontal)

kubectl → API Server → Scheduler → kubelet → Container Runtime → Check Local Image → Pull Image (if needed) → Create Container → Start Application → Running Pod

What is CRI (Container Runtime Interface)?

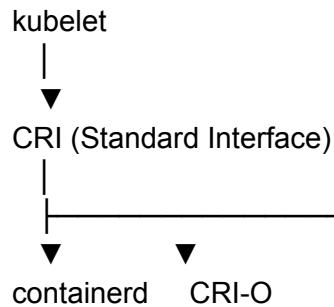
Earlier versions of Kubernetes were tightly integrated with Docker.

This meant Kubernetes could directly communicate with Docker.

As container technologies evolved, Kubernetes introduced the **Container Runtime Interface (CRI)**.

CRI is a standard interface that allows the kubelet to communicate with different container runtimes without needing to know their internal implementation.

Instead of talking directly to Docker or containerd, the kubelet talks to the CRI.



This makes Kubernetes flexible and runtime-independent.

Why did Kubernetes stop using Docker directly?

This is a common interview question.

Docker is a complete platform for building, packaging, and running containers.

Kubernetes only needs the runtime portion.

Since Docker itself uses **containerd** underneath, Kubernetes removed its direct dependency on Docker (a change commonly referred to as the removal of "Dockershim") and now communicates with CRI-compatible runtimes such as containerd or CRI-O.

This simplified Kubernetes and reduced maintenance complexity.

What happens if the Container Runtime stops?

Suppose `containerd` crashes.

Effects include:

- New containers cannot be created.
- Images cannot be pulled.
- Failed containers cannot be restarted.
- kubelet reports runtime errors.
- Pods may remain in states such as `ContainerCreating` or fail to start.

The kubelet continues running, but it cannot manage containers until the runtime becomes healthy again.

Troubleshooting Container Runtime

Check whether the runtime service is running.

For **containerd**:

```
systemctl status containerd
```

Restart the service if required:

```
sudo systemctl restart containerd
```

View logs:

```
journalctl -u containerd
```

Check Kubernetes events:

```
kubectl describe pod <pod-name>
```

Common errors include:

Failed to pull image

ImagePullBackOff

ErrImagePull

FailedCreatePodSandBox

These often indicate runtime, image, registry, or networking issues.

Common Interview Questions

1. What is a Container Runtime?

A Container Runtime is the software responsible for pulling images, creating containers, starting them, stopping them, and removing them on a Worker Node.

2. Does kubelet create containers?

No.

The kubelet instructs the Container Runtime, which performs the actual container operations.

3. What is CRI?

CRI (Container Runtime Interface) is a standard interface that allows the kubelet to communicate with different container runtimes such as containerd and CRI-O.

4. Why did Kubernetes remove Docker support?

Kubernetes removed its direct dependency on Docker (Dockershim). It now communicates with CRI-compatible runtimes like containerd, which Docker itself uses internally.

5. What happens if the Container Runtime fails?

New containers cannot be created or started, image pulls fail, and kubelet cannot manage container lifecycle operations until the runtime is restored.

Important Points to Remember

- A Container Runtime executes containers on a Worker Node.
 - The kubelet communicates with the runtime through the **Container Runtime Interface (CRI)**.
 - Common runtimes are **containerd** and **CRI-O**.
 - The runtime pulls images, creates containers, starts processes, and cleans up resources.
 - The runtime does **not** schedule Pods or communicate directly with the API Server.
 - If the runtime fails, kubelet cannot start or manage containers.
-

SRE Interview Tip

A common answer is:

"containerd runs containers."

A stronger answer is:

"The Container Runtime is responsible for the complete lifecycle of containers on a Worker Node. The kubelet communicates with it through the Container Runtime Interface (CRI) to pull images, create containers, start application processes, stop containers, and clean up resources. Modern Kubernetes clusters commonly use containerd or CRI-O as the runtime."

Current Progress

You've now mastered the core architecture:

-  Kubernetes Architecture
-  kube-apiserver
-  etcd
-  kube-scheduler
-  kube-controller-manager
-  Worker Node
-  kubelet

-  Container Runtime

Next: kube-proxy, where you'll learn how Pods communicate with each other and how Kubernetes Services route traffic. This topic is essential before moving on to Services, Ingress, and CoreDNS.

Excellent. You're now entering **Kubernetes Networking**, which is one of the most important areas for an SRE and is frequently covered in Akamai interviews.

So far, you've learned **how a Pod gets created and starts running**. Now the next question is:

"How does traffic reach that Pod?"

The answer starts with **kube-proxy**.

Section 9: kube-proxy (Deep Dive)

Objective: By the end of this topic, you should understand what kube-proxy is, why Kubernetes needs it, how Services work with kube-proxy, and how traffic reaches Pods.

What is kube-proxy?

kube-proxy is a network component that runs on every Worker Node.

Its primary responsibility is to implement Kubernetes **Service networking** by managing network rules that route traffic from a Service to the correct backend Pod(s).

Unlike kubelet, kube-proxy does not manage Pods or containers. Its only responsibility is networking.

Why do we need kube-proxy?

Suppose you have an application with three Pods.

Pod-A → IP: 10.244.1.10

Pod-B → IP: 10.244.2.15

Pod-C → IP: 10.244.3.20

Each Pod gets its own IP.

Now imagine **Pod-A crashes**.

Kubernetes creates a replacement Pod.

Old Pod IP

10.244.1.10

↓

New Pod IP

10.244.1.45

The Pod IP has changed.

If users directly accessed Pod IPs, every Pod restart would require clients to update the destination IP.

This is not practical.

Kubernetes solves this problem using **Services**.

A Service provides a stable virtual IP.

kube-proxy ensures that requests sent to the Service are forwarded to one of the healthy Pods behind it.

Responsibilities of kube-proxy

kube-proxy performs the following tasks.

1. Watches Services

kube-proxy continuously watches the API Server for:

- Services
- Endpoints (or EndpointSlices in newer Kubernetes versions)

Whenever these resources change, kube-proxy updates the networking rules on the node.

2. Creates Network Rules

When a Service is created, kube-proxy configures networking rules (commonly using **iptables** or **IPVS**, depending on cluster configuration).

These rules determine how incoming traffic should be forwarded to backend Pods.

3. Load Balances Traffic

Suppose a Service has three Pods.

Service

↓

Pod-A

Pod-B

Pod-C

kube-proxy distributes incoming requests across these Pods.

The exact algorithm depends on the proxy mode and configuration, but the goal is to spread traffic across available healthy backends.

4. Updates Rules Automatically

Suppose:

Pod-B crashes.

The Endpoint list changes.

kube-proxy detects the change through the API Server and updates the networking rules.

Traffic is now routed only to:

Pod-A

Pod-C

No manual intervention is required.

How kube-proxy Works

Suppose you create:

apiVersion: v1

kind: Service

The sequence is:

Step 1

The Service object is stored in the API Server.

Step 2

kube-proxy watches the API Server.

It detects the new Service.

Step 3

kube-proxy configures networking rules on every Worker Node.

Step 4

A client sends a request to the Service IP.

Step 5

kube-proxy forwards the request to one of the backend Pods.

Step 6

The selected Pod processes the request.

Communication Flow (Horizontal)

Client → Service (ClusterIP) → kube-proxy → Backend Pod (Selected) → Response → Client

Complete flow:

Client → Service → kube-proxy → Pod-1 / Pod-2 / Pod-3 → Response

Example

Suppose:

Service IP

10.96.0.50

Backend Pods:

Pod-A → 10.244.1.10

Pod-B → 10.244.2.15

Pod-C → 10.244.3.20

A client sends:

GET http://10.96.0.50

kube-proxy may forward it like this:

Client

↓

Service IP

↓

kube-proxy

↓

Pod-B

The client doesn't know which Pod handled the request. It only communicates with the Service.

kube-proxy Proxy Modes

kube-proxy can operate in different modes.

1. iptables Mode (Most Common)

Uses Linux **iptables** rules.

Advantages:

- Fast
- Simple

- Widely used
-

2. IPVS Mode

Uses the Linux **IP Virtual Server (IPVS)** framework.

Advantages:

- Better performance
 - Efficient with large numbers of Services
 - Better scalability
-

What kube-proxy Does NOT Do

A common interview mistake is to overstate kube-proxy's responsibilities.

kube-proxy does **not**:

- Create Pods
- Schedule Pods
- Start Containers
- Pull Images
- Register Nodes
- Store Cluster Data

It only manages network routing for Services.

What happens if kube-proxy stops?

Suppose kube-proxy crashes on Worker Node-2.

Effects include:

- Existing Pods continue running.
- New or updated Service rules are not applied on that node.
- Traffic routing through Services on that node may fail or become outdated.

- Direct Pod-to-Pod communication (handled by the cluster's CNI plugin) is generally unaffected.
-

Troubleshooting kube-proxy

Check the kube-proxy Pods:

```
kubectl get pods -n kube-system
```

Example:

```
kube-proxy-abc123
```

View logs:

```
kubectl logs kube-proxy-abc123 -n kube-system
```

Check Service information:

```
kubectl get svc
```

Check Endpoints:

```
kubectl get endpoints
```

or (newer clusters):

```
kubectl get endpointslices
```

If a Service isn't working:

1. Check if the Service exists.
2. Verify that backend Pods are **Running** and **Ready**.

3. Check the Service's Endpoints/EndpointSlices.
 4. Verify the kube-proxy Pod is healthy.
 5. Inspect networking rules if needed (`iptables` or `ipvsadm`, depending on proxy mode).
-

Common Interview Questions

1. What is kube-proxy?

kube-proxy is a networking component that runs on every Worker Node and manages network rules to route Service traffic to backend Pods.

2. Does kube-proxy create Pods?

No.

It only manages network routing.

3. Why do we need kube-proxy?

Because Pod IPs are temporary. kube-proxy enables stable Service networking and routes traffic to the appropriate backend Pods.

4. What resources does kube-proxy watch?

It watches Services and Endpoints (or EndpointSlices).

5. What happens if kube-proxy stops?

Existing Pods continue running, but Service traffic on that node may stop working correctly or use outdated routing information until kube-proxy is restored.

Important Points to Remember

- kube-proxy runs on every Worker Node.
 - It implements Kubernetes Service networking.
 - It watches Services and Endpoints/EndpointSlices.
 - It creates networking rules using **iptables** or **IPVS**.
 - It routes traffic from a Service to healthy backend Pods.
 - It does not manage Pods or containers.
-

★ SRE Interview Tip

Many candidates answer:

"kube-proxy forwards traffic."

A stronger answer is:

"kube-proxy is a network component that runs on every Worker Node. It watches Services and Endpoints through the API Server and programs networking rules, typically using iptables or IPVS. When traffic reaches a Kubernetes Service, kube-proxy forwards it to one of the healthy backend Pods, providing stable Service networking even though Pod IPs may change."

Current Progress

You've now completed the **core architecture** of a Kubernetes cluster:

-  Kubernetes Architecture
-  kube-apiserver
-  etcd
-  kube-scheduler
-  kube-controller-manager
-  Worker Node
-  kubelet
-  Container Runtime
-  kube-proxy

Next: CNI (Container Network Interface).

This is where you'll understand **how Pods on different Worker Nodes communicate with each other**, which is one of the most common follow-up questions after kube-proxy in advanced Kubernetes interviews.

Section 10: Container Network Interface (CNI)

Objective: By the end of this topic, you should understand what CNI is, why Kubernetes needs it, how Pod networking works, and how Pods on different Worker Nodes communicate.

What is CNI?

Container Network Interface (CNI) is a standard interface that allows Kubernetes to configure networking for Pods.

Kubernetes itself does **not** create Pod networks. Instead, it delegates this responsibility to a CNI plugin.

Popular CNI plugins include:

- Calico
 - Flannel
 - Cilium
 - Weave Net
 - AWS VPC CNI (Amazon EKS)
-

Why does Kubernetes need CNI?

Suppose Kubernetes creates a Pod.

Pod Created

The Pod cannot communicate yet.

It needs:

- An IP address
- A network interface
- Routes to other Pods

- Connectivity to Services
- Connectivity across Worker Nodes

Kubernetes doesn't perform these networking tasks itself.

Instead, it asks the CNI plugin to configure networking for the Pod.

Without a CNI plugin:

- Pods won't receive IP addresses.
 - Pods can't communicate.
 - Services won't function correctly.
 - Multi-node clusters won't work.
-

Responsibilities of a CNI Plugin

A CNI plugin is responsible for:

1. Assigning IP Addresses

Every Pod receives a unique IP.

Example:

Pod-A → 10.244.1.10

Pod-B → 10.244.2.15

2. Creating Network Interfaces

When a Pod is created, the CNI plugin creates the necessary virtual network interfaces to connect the Pod to the cluster network.

3. Configuring Routing

If Pod-A wants to communicate with Pod-B on another Worker Node, the CNI plugin ensures that routing is configured correctly.

4. Connecting Multiple Worker Nodes

Suppose:

Worker-1

Pod-A

Worker-2

Pod-B

Although the Pods are on different machines, they can communicate directly because the CNI plugin connects the node networks.

Kubernetes Networking Rules

1. Every Pod has a unique IP.
2. Pods communicate without NAT.
3. Pods on different Worker Nodes can talk directly.
4. Nodes can talk to Pods.
5. Pods can talk to Services.

Workflow

- Scheduler assigns Pod → kubelet manages Pod → Container Runtime creates container → CNI plugin configures networking (IP, routes, interfaces).

Pod-to-Pod Communication

- Pod-A (Worker-1) → Pod-B (Worker-2) via CNI network.
- Application doesn't need to know Pod location — IP is enough.

Popular Plugins

- Calico → production-grade, supports Network Policies.
- Flannel → simple, good for learning.
- Cilium → eBPF-based, advanced security & observability.
- AWS VPC CNI → Pods get native VPC IPs in EKS.

Failure Scenario

- If CNI fails:
 - Pods stuck in ContainerCreating.
 - No IP assignment.
 - Pod-to-Pod communication fails.

- Services break.

Troubleshooting

- Check CNI Pods: `kubectl get pods -n kube-system`
- Logs: `kubectl logs <cni-pod> -n kube-system`
- Pod IPs: `kubectl get pods -o wide`
- Connectivity: `kubectl exec -it <pod> -- ping <pod-ip>`

Interview Nuggets

- CNI vs kube-proxy:
 - CNI → Pod networking (IP assignment, routing).
 - kube-proxy → Service networking (load balancing, forwarding).
- One-liner:
- “CNI provides Pod-to-Pod connectivity, kube-proxy provides Service-to-Pod connectivity.”

Excellent. 🎉

You've now completed the Kubernetes infrastructure (Control Plane + Worker Node + Networking). From here, we move to the **Kubernetes Objects**. The first and most fundamental object is the **Pod**.

Almost every Kubernetes object (Deployment, StatefulSet, DaemonSet, Job) ultimately creates **Pods**.

Section 11: Pod (Deep Dive)

Objective: By the end of this topic, you should understand what a Pod is, why Kubernetes uses Pods instead of containers, how Pods are structured, their lifecycle, and how to troubleshoot Pod issues.

What is a Pod?

A **Pod** is the smallest deployable unit in Kubernetes.

A Pod is a wrapper around one or more containers that share the same:

- Network Namespace
- IP Address
- Storage Volumes
- Hostname
- Lifecycle

When you deploy an application in Kubernetes, Kubernetes does **not** create containers directly. It creates a Pod, and the Pod contains the container(s).

Why does Kubernetes use Pods instead of Containers?

This is one of the most common interview questions.

A container runtime (like containerd) can run containers independently.

However, Kubernetes needs a higher-level object that can:

- Manage one or more related containers together.
- Restart failed containers.
- Share networking.
- Share storage.
- Schedule workloads.
- Apply policies.

That higher-level object is the **Pod**.

Think of it this way:

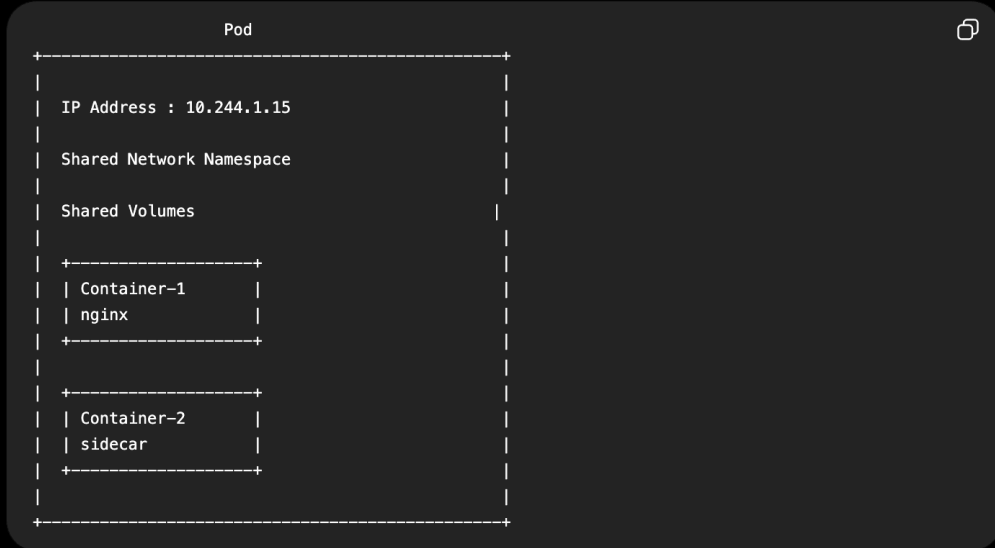
Kubernetes manages Pods.

Pods contain Containers.

Container Runtime runs Containers.

Pod Architecture

Pod Architecture



Most Pods contain **one container**, but Kubernetes supports multiple containers within the same Pod.

Most Pods contain **one container**, but Kubernetes supports multiple containers within the same Pod.

Responsibilities of a Pod

A Pod provides:

1. Shared Network

All containers inside a Pod share the same IP address.

Example:

Pod

Container-A

Container-B

IP : 10.244.1.20

Container-A communicates with Container-B using:

localhost

No network configuration is required between containers in the same Pod.

2. Shared Storage

Containers inside a Pod can share Volumes.

Example:

Container-A

↓

Shared Volume

↑

Container-B

If one container writes a file:

/data/report.txt

another container can read the same file from the shared volume.

3. Shared Lifecycle

Containers inside the Pod are created and deleted together.

If the Pod is deleted:

Pod Deleted

↓

Container-A Deleted

↓

Container-B Deleted

The Pod is treated as a single unit.

Single-Container Pod

Most production workloads use one application container per Pod.

Example:

Pod

↓

Nginx Container

Advantages:

- Simpler architecture
 - Easier scaling
 - Better isolation
 - Easier troubleshooting
-

Multi-Container Pod

Sometimes multiple containers need to work closely together.

Example:

Pod

↓

Application Container



Log Collector Container

The application writes logs to a shared volume.

The sidecar container reads those logs and sends them to Elasticsearch or another logging system.

Because they share the same Pod, communication is simple and efficient.

How a Pod is Created

Suppose you run:

```
kubectl apply -f pod.yaml
```

The sequence is:

Step 1

API Server stores the Pod definition.

Step 2

etcd stores the desired state.

Step 3

Scheduler selects a Worker Node.

Step 4

kubelet receives the assignment.

Step 5

Container Runtime creates the container(s).

Step 6

CNI assigns a Pod IP.

Step 7

The Pod reaches the **Running** state.

Communication Flow (Horizontal)

kubectl → API Server → etcd → Scheduler → Worker Node → kubelet → Container Runtime → CNI → Running Pod

Pod Lifecycle

A Pod moves through several phases during its lifetime.

Pending

The Pod has been accepted by Kubernetes but one or more resources are still being prepared.

Examples:

- Waiting for scheduling
 - Pulling image
 - Creating network
-

Running

The Pod has been scheduled, containers have started, and at least one container is running.

Succeeded

All containers completed successfully and exited normally.

Common for Jobs.

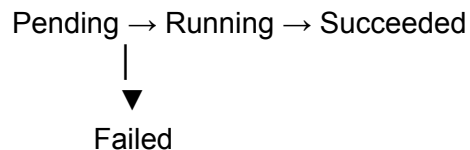
Failed

One or more containers exited with failure and will not be restarted according to the Pod's restart policy.

Unknown

The API Server cannot determine the Pod's current state, often due to communication issues with the node.

Pod Lifecycle Flow



Pod Restart Policy

Pods support three restart policies.

Always

Containers are restarted whenever they exit.

Default for Deployments.

OnFailure

Restart only if the container exits with a non-zero exit code.

Common for Jobs.

Never

Never restart the container.

Useful for one-time tasks.

Pod YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod

spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

Important fields:

- `apiVersion`
- `kind`
- `metadata`
- `spec`
- `containers`

We'll study every field later.

Troubleshooting Pods

List Pods:

```
kubectl get pods
```

Detailed information:

```
kubectl describe pod nginx-pod
```

Look at:

- Events
 - Conditions
 - Node assignment
-

View logs:

```
kubectl logs nginx-pod
```

Execute commands inside a Pod:

```
kubectl exec -it nginx-pod -- /bin/sh
```

Common Pod Problems

Problem	Possible Cause
Pending	Scheduler couldn't find a suitable node
ImagePullBackOff	Image not found or registry authentication issue
ErrImagePull	Failed to download image
CrashLoopBackOff	Application repeatedly crashes after starting
ContainerCreating	Image pull, volume, or networking still in progress

OOMKilled

Container exceeded its memory limit

Common Interview Questions

1. What is a Pod?

A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers that share networking, storage, and lifecycle.

2. Can a Pod contain multiple containers?

Yes.

Containers within the same Pod share the same network namespace, IP address, and storage volumes.

3. Why does Kubernetes use Pods instead of Containers?

Pods provide a higher-level abstraction for managing containers together, enabling shared networking, storage, lifecycle management, and scheduling.

4. What is the smallest deployable unit in Kubernetes?

A Pod.

5. Do all containers inside a Pod have different IP addresses?

No.

All containers inside the same Pod share a single IP address.

Important Points to Remember

- A Pod is the smallest deployable unit in Kubernetes.
 - A Pod contains one or more containers.
 - Containers in the same Pod share:
 - Network namespace
 - IP address
 - Storage volumes
 - Lifecycle
 - Most production Pods contain a single application container.
 - Multi-container Pods are commonly used for sidecars and tightly coupled helper processes.
 - Pods are **ephemeral**—if a Pod is deleted, Kubernetes creates a new Pod rather than repairing the old one (when managed by a Deployment or similar controller).
-

★ SRE Interview Tip

One of the most common interview questions is:

"What happens internally when you run `kubectl apply -f pod.yaml`?"

A concise, technically accurate answer is:

```
kubectl
↓
API Server validates the request
↓
etcd stores the Pod definition
↓
Scheduler selects a Worker Node
↓
kubelet detects the Pod assignment
↓
Container Runtime creates the container
↓
CNI configures networking and assigns a Pod IP
↓
Pod enters the Running state
```

This answer ties together almost everything you've learned so far and demonstrates a solid understanding of the Kubernetes architecture.

Next Topic

Next, we'll cover **ReplicaSet**, where you'll learn:

- Why Pods are not created directly in production
- How Kubernetes maintains the desired number of Pods
- Self-healing in action
- ReplicaSet architecture
- Scaling
- Pod replacement
- Production troubleshooting

Once you understand ReplicaSets, Deployments become very easy because **a Deployment manages ReplicaSets, and ReplicaSets manage Pods**.

What is Node in K8?

A Node in Kubernetes is just a worker machine — it can be a physical server or a virtual machine — where your application Pods actually run.

Each Node provides the compute resources (CPU, memory, storage, and networking) that Pods need, and it has three key components:

- kubelet → the agent that talks to the control plane and makes sure containers are running.
- container runtime → the software that runs the containers (like containerd or Docker).
- kube-proxy → handles networking so Pods and Services can talk to each other.

- Section 12: ReplicaSet (Deep Dive)

Objective

By the end of this topic, you should be able to:

- Explain ReplicaSet in plain English.
- Answer interview questions confidently.
- Understand how it works internally.
- Troubleshoot ReplicaSet issues.
- Practice it in your Kind cluster.

What is a ReplicaSet?

A ReplicaSet is a Kubernetes object that makes sure the required number of Pods are always running.

For example, if you want **3 Pods**, ReplicaSet continuously checks the cluster.

- If all 3 Pods are running → It does nothing.
- If one Pod crashes → It creates a new Pod.
- If one Pod is deleted → It creates another Pod.

Its only job is to maintain the desired number of Pods.

ReplicaSet YAML

```
</> YAML
apiVersion: apps/v1
kind: ReplicaSet

metadata:
  name: nginx-rs

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      labels:
        app: nginx

    spec:
      containers:
        - name: nginx
          image: nginx
```

Don't memorize the YAML.

Just understand the important parts:

- `replicas` → Number of Pods.
- `selector` → How ReplicaSet finds its Pods.
- `template` → Blueprint for creating new Pods.

Commands You Should Know

Create ReplicaSet

```
kubectl apply -f rs.yaml
```

View ReplicaSets

```
kubectl get rs
```

View Pods

kubectl get pods

Describe ReplicaSet

kubectl describe rs nginx-rs

Delete one Pod

kubectl delete pod <pod-name>

Watch self-healing

kubectl get pods -w

Important Interview Question

Q: Can we create Pods directly?

Answer:

"Yes, we can create standalone Pods for testing or learning. But in production we usually use a Deployment, because Deployments create ReplicaSets, and ReplicaSets make sure Pods are automatically recreated if they fail."

Q: What happens if you delete a Pod managed by a ReplicaSet?

Answer:

"ReplicaSet notices that the current number of Pods is less than the desired number, so it automatically creates a replacement Pod."

Q: What happens if a node crashes?

Answer:

"The Pods on that node become unavailable. ReplicaSet detects that the required number of running Pods is not met and creates replacement Pods on another healthy node if resources are available."

Absolutely. I like that suggestion.

From the next topic onwards I'll use a **compact notes format** that's easy to paste into Google Docs/PDF.

Changes I'll make:

- Smaller headings (no huge markdown headings)
- Short paragraphs
- Plain English only
- Candidate-style interview answers
- Real examples
- Architecture diagrams
- Production scenarios
- Troubleshooting
- Important commands
- Interview questions
- Key points at the end

It will look something like this:

Phase 2 - Section 13: Deployment

Objective

Learn what a Deployment is, why we use it, how it works, and how to answer Deployment interview questions confidently.

What is a Deployment?

...

instead of using large # or ## headings.

This will save a lot of space in your PDF while keeping it easy to read.

Let's continue.

Phase 2 - Section 13: Deployment (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a Deployment is.
 - Know why we use Deployment instead of ReplicaSet.
 - Understand rolling updates and rollbacks.
 - Answer Deployment interview questions confidently.
 - Troubleshoot Deployment issues.
-

What is a Deployment?

A Deployment is a Kubernetes object that manages ReplicaSets.

It makes it easy to:

- Deploy applications
- Update applications
- Roll back to an older version
- Scale applications

Think of it as the **manager** of ReplicaSets.

Interview Answer

Q: What is a Deployment?

Answer:

"A Deployment is a Kubernetes object that manages ReplicaSets. It helps us deploy applications, perform rolling updates, roll back to previous versions, and scale Pods without downtime."

Why do we need a Deployment?

Suppose you have 3 Nginx Pods running.

Now you want to update from:

nginx:1.25

to

nginx:1.26

Without a Deployment:

- You delete old Pods.
- Create new Pods.
- Users may face downtime.

With a Deployment:

- Kubernetes updates one Pod at a time.
 - Users continue using the application.
 - No downtime (or very minimal).
-

Real Life Example

Imagine changing all the cashiers in a supermarket.

Bad approach:

- Send all 5 cashiers home.
- Bring 5 new cashiers.
- Customers wait.

Better approach:

- Replace one cashier at a time.
- Billing continues.

A Deployment works the same way.

Architecture

Deployment

|



ReplicaSet

|



Pods

Remember:

- Deployment manages ReplicaSets.

- ReplicaSet manages Pods.
 - Pods run Containers.
-

How Deployment Works

Suppose you apply:

replicas: 3

image: nginx:1.25

Deployment creates:

Deployment

|



ReplicaSet

|



3 Pods

Now you change:

image: nginx:1.26

Deployment creates a **new ReplicaSet**.

Then:

New Pod (1.26) ↑

Old Pod (1.25) ↓

↓

Repeat

↓

All Pods become 1.26

This is called a **Rolling Update**.

Rolling Update

Rolling Update means:

Instead of updating all Pods together,

Kubernetes updates them one by one.

Benefits:

- No downtime
 - Easy monitoring
 - Safer deployments
-

Rollback

Suppose version 1.26 has a bug.

Deployment can return to 1.25.

This is called **Rollback**.

Very useful in production.

Deployment YAML

```
Deployment YAML

</> YAML

apiVersion: apps/v1
kind: Deployment

metadata:
  name: nginx-deploy

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      labels:
        app: nginx

    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

Important fields:

- replicas → Number of Pods
- selector → Finds Pods
- template → Creates Pods
- image → Application version

Production Scenario

Suppose your team releases a new application version.

Instead of deleting all Pods,

you update the Deployment image.

Deployment performs a rolling update.

If customers report issues,

you immediately roll back to the previous version.

This is one of the biggest reasons Deployments are used in production.

Common Problems

Pods not updating

Possible reasons:

- Wrong image
 - Image pull failure
 - Deployment paused
-

Pods stuck in Pending

Possible reasons:

- No CPU or memory
 - Node unavailable
-

New version keeps crashing

Possible reasons:

- Application bug
 - Wrong environment variables
 - Wrong ConfigMap or Secret
-

Important Commands

Create Deployment

```
kubectl apply -f deployment.yaml
```

View Deployments

```
kubectl get deployments
```


View ReplicaSets

```
kubectl get rs
```

View Pods

```
kubectl get pods
```

Scale Deployment

```
kubectl scale deployment nginx-deploy --replicas=5
```

Update Image

```
kubectl set image deployment/nginx-deploy nginx=nginx:1.26
```

View Rollout Status

```
kubectl rollout status deployment/nginx-deploy
```

Rollback

```
kubectl rollout undo deployment/nginx-deploy
```

Interview Questions

Q1. What is a Deployment?

Answer:

"A Deployment manages ReplicaSets and provides features like rolling updates, rollbacks, and scaling."

Q2. Deployment vs ReplicaSet?

Answer:

"ReplicaSet only maintains the required number of Pods. Deployment manages ReplicaSets and adds rolling updates, rollbacks, and version management."

Q3. What is a Rolling Update?

Answer:

"A rolling update replaces old Pods with new Pods one at a time, which helps avoid downtime."

Q4. What is Rollback?

Answer:

"Rollback restores the previous working version if the new deployment has issues."

Q5. What happens when you update the image in a Deployment?

Answer:

"Deployment creates a new ReplicaSet with the new image and gradually replaces the old Pods. Once all new Pods are healthy, the old ReplicaSet is scaled down."

Key Points

- Deployment is the most common way to deploy applications.
 - Deployment manages ReplicaSets.
 - ReplicaSets manage Pods.
 - Deployments support rolling updates.
 - Deployments support rollbacks.
 - Most production applications are deployed using Deployments.
-

SRE Interview Tip

Interviewer: "A new application version was deployed and users started reporting issues. What would you do?"

Candidate Answer:

"First, I'd check the rollout status and verify whether the new Pods are healthy using `kubectl get pods`, `kubectl describe pod`, and `kubectl logs`. If the issue is caused by the new version and impacts users, I'd roll back the Deployment to the previous stable version using `kubectl rollout undo` while investigating the root cause."

This is exactly the kind of practical answer interviewers look for.

Phase 2 - Section 14: Namespace (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a Namespace is.
- Know why Kubernetes uses Namespaces.
- Create and manage Namespaces.
- Answer Namespace interview questions confidently.
- Troubleshoot Namespace-related issues.

What is a Namespace?

A Namespace is a logical partition inside a Kubernetes cluster.

It helps us organize and isolate resources.

Think of it as creating different folders inside the same computer.

Example:

Kubernetes Cluster

|

|— default

|— kube-system

|— dev

|— test

└─ production

Each Namespace contains its own Pods, Services, Deployments, etc.

Interview Answer

Q: What is a Namespace?

Answer:

"A Namespace is a logical way to divide a Kubernetes cluster into multiple environments. It helps organize resources and isolate applications within the same cluster."

Why do we need Namespaces?

Imagine your company has three teams.

- Development Team
- Testing Team
- Production Team

Without Namespaces:

Cluster

Pod1

Pod2

Pod3

Pod4

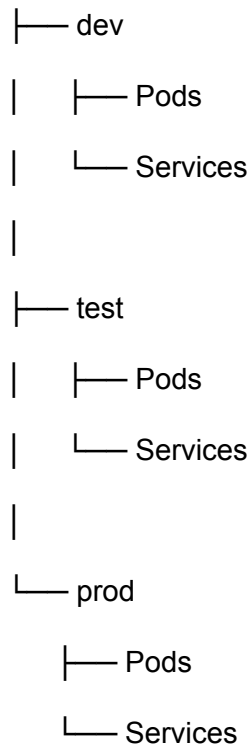
Pod5

Pod6

Everything is mixed together.

With Namespaces:

Cluster



Now each team works independently.

Default Namespaces

Every cluster has some built-in Namespaces.

default

Used if you don't specify a Namespace.

kube-system

Contains Kubernetes system components like:

- API Server
- etcd
- Scheduler
- CoreDNS

- kube-proxy

kube-public

Stores resources that are publicly accessible.

Rarely used in day-to-day work.

kube-node-lease

Stores heartbeat information for nodes.

Used internally by Kubernetes.

Production Example

Suppose your company hosts three applications.

Instead of creating three clusters:

Payment

HR

Inventory

You can create one cluster with three Namespaces.

Cluster

|— payment

|— hr

└─ inventory

This saves infrastructure cost while keeping applications organized.

Important Commands

Create Namespace

```
kubectl create namespace dev
```

View Namespaces

```
kubectl get ns
```

Delete Namespace

```
kubectl delete namespace dev
```

Create Pod in Namespace

```
kubectl apply -f pod.yaml -n dev
```

View Pods in Namespace

```
kubectl get pods -n dev
```

Describe Namespace

```
kubectl describe namespace dev
```

Interview Questions

Q1. What is the default Namespace?

Answer:

"If we don't specify a Namespace, Kubernetes creates resources in the default Namespace."

Q2. Why do we use Namespaces?

Answer:

"Namespaces help organize resources, isolate applications, and allow multiple teams to share the same Kubernetes cluster."

Q3. Can two Pods have the same name?

Answer:

"Yes, if they are in different Namespaces. Pod names must be unique only within the same Namespace."

Example:

dev

└─ nginx

prod

└─ nginx

This is valid.

Q4. Can a Pod in one Namespace access a Service in another Namespace?

Answer:

"Yes. It can use the Service's full DNS name."

Example:

mysql.database.svc.cluster.local

Q5. Is a Namespace a security boundary?

Answer:

"No. A Namespace provides logical isolation. For stronger security, we use RBAC and Network Policies."

This is a good interview point.

Common Problems

Problem: No resources found

Possible reason:

You're checking the wrong Namespace.

Example:

```
kubectl get pods
```

No output.

But:

```
kubectl get pods -n dev
```

Shows the Pods.

Problem: Pod cannot find a Service.

Possible reasons:

- Wrong Namespace
 - Wrong Service name
 - Wrong DNS name
-

SRE Production Scenario

Suppose a developer says:

"My application is not running."

First thing I check:

```
kubectl get pods -A
```

Then verify whether the application was deployed to the correct Namespace.

Many production issues happen simply because resources are created in the wrong Namespace.

Key Points

- Namespace logically divides a Kubernetes cluster.
 - Default Namespace is `default`.
 - System components run in `kube-system`.
 - Namespaces help multiple teams share one cluster.
 - Names are unique within a Namespace, not across the entire cluster.
 - Namespaces improve organization but are not a complete security mechanism.
-

SRE Interview Tip

Interviewer: "A developer says, 'My Pod doesn't exist,' but you know it was created successfully. What would you check first?"

Candidate Answer:

"First, I'd check all Namespaces using `kubectl get pods -A` or verify the expected Namespace with `kubectl get pods -n <namespace>`. A common reason is that the Pod was created in a different Namespace than the one being checked."

Phase 2 - Section 14: Namespace (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a Namespace is.
 - Know why Kubernetes uses Namespaces.
 - Create and manage Namespaces.
 - Answer Namespace interview questions confidently.
 - Troubleshoot Namespace-related issues.
-

What is a Namespace?

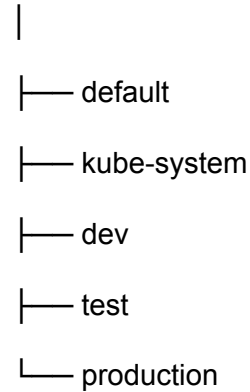
A Namespace is a logical partition inside a Kubernetes cluster.

It helps us organize and isolate resources.

Think of it as creating different folders inside the same computer.

Example:

Kubernetes Cluster



Each Namespace contains its own Pods, Services, Deployments, etc.

Interview Answer

Q: What is a Namespace?

Answer:

"A Namespace is a logical way to divide a Kubernetes cluster into multiple environments. It helps organize resources and isolate applications within the same cluster."

Why do we need Namespaces?

Imagine your company has three teams.

- Development Team
- Testing Team
- Production Team

Without Namespaces:

Cluster

Pod1

Pod2

Pod3

Pod4

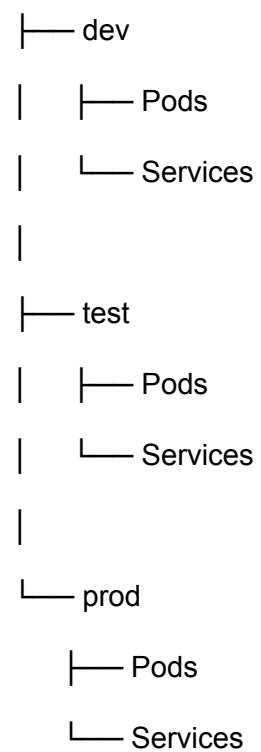
Pod5

Pod6

Everything is mixed together.

With Namespaces:

Cluster



Now each team works independently.

Default Namespaces

Every cluster has some built-in Namespaces.

default

Used if you don't specify a Namespace.

kube-system

Contains Kubernetes system components like:

- API Server
 - etcd
 - Scheduler
 - CoreDNS
 - kube-proxy
-

kube-public

Stores resources that are publicly accessible.

Rarely used in day-to-day work.

kube-node-lease

Stores heartbeat information for nodes.

Used internally by Kubernetes.

Production Example

Suppose your company hosts three applications.

Instead of creating three clusters:

Payment

HR

Inventory

You can create one cluster with three Namespaces.

Cluster

└─ payment

└─ hr

└─ inventory

This saves infrastructure cost while keeping applications organized.

Important Commands

Create Namespace

```
kubectl create namespace dev
```

View Namespaces

```
kubectl get ns
```

Delete Namespace

```
kubectl delete namespace dev
```

Create Pod in Namespace

```
kubectl apply -f pod.yaml -n dev
```

View Pods in Namespace

```
kubectl get pods -n dev
```

Describe Namespace

```
kubectl describe namespace dev
```

Interview Questions

Q1. What is the default Namespace?

Answer:

"If we don't specify a Namespace, Kubernetes creates resources in the default Namespace."

Q2. Why do we use Namespaces?

Answer:

"Namespaces help organize resources, isolate applications, and allow multiple teams to share the same Kubernetes cluster."

Q3. Can two Pods have the same name?

Answer:

"Yes, if they are in different Namespaces. Pod names must be unique only within the same Namespace."

Example:

dev

└─ nginx

prod

└─ nginx

This is valid.

Q4. Can a Pod in one Namespace access a Service in another Namespace?

Answer:

"Yes. It can use the Service's full DNS name."

Example:

mysql.database.svc.cluster.local

Q5. Is a Namespace a security boundary?

Answer:

"No. A Namespace provides logical isolation. For stronger security, we use RBAC and Network Policies."

This is a good interview point.

Common Problems

Problem: No resources found

Possible reason:

You're checking the wrong Namespace.

Example:

kubectl get pods

No output.

But:

kubectl get pods -n dev

Shows the Pods.

Problem: Pod cannot find a Service.

Possible reasons:

- Wrong Namespace
 - Wrong Service name
 - Wrong DNS name
-

SRE Production Scenario

Suppose a developer says:

"My application is not running."

First thing I check:

`kubectl get pods -A`

Then verify whether the application was deployed to the correct Namespace.

Many production issues happen simply because resources are created in the wrong Namespace.

Key Points

- Namespace logically divides a Kubernetes cluster.
 - Default Namespace is `default`.
 - System components run in `kube-system`.
 - Namespaces help multiple teams share one cluster.
 - Names are unique within a Namespace, not across the entire cluster.
 - Namespaces improve organization but are not a complete security mechanism.
-

SRE Interview Tip

Interviewer: "A developer says, 'My Pod doesn't exist,' but you know it was created successfully. What would you check first?"

Candidate Answer:

"First, I'd check all Namespaces using `kubectl get pods -A` or verify the expected Namespace with `kubectl get pods -n <namespace>`. A common reason is that the Pod was created in a different Namespace than the one being checked."

Great. This is one of the **most important Kubernetes topics**. If you understand **Labels & Selectors**, then ReplicaSets, Deployments, Services, and Network Policies become much easier.

Phase 2 - Section 15: Labels & Selectors (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what Labels are.
 - Understand what Selectors are.
 - Know how Kubernetes uses them.
 - Answer interview questions confidently.
 - Troubleshoot label-related issues.
-

What are Labels?

Labels are **key-value pairs** attached to Kubernetes objects.

They help identify and organize resources.

Example:

labels:

app: nginx

env: production

team: backend

Think of labels as **tags**.

Just like you tag photos on your phone, Kubernetes tags Pods with labels.

Interview Answer

Q: What are Labels?

Answer:

"Labels are key-value pairs attached to Kubernetes objects. They help identify, organize, and group resources. Kubernetes uses labels to find and manage Pods."

Why do we need Labels?

Imagine you have 100 Pods.

Without labels:

Pod1

Pod2

Pod3

Pod4

...

Pod100

It's difficult to know which Pod belongs to which application.

With labels:

Pod1

app=nginx

Pod2

app=nginx

Pod3

app=mysql

Pod4

app=redis

Now Kubernetes can easily identify Pods.

Real-Life Example

Think of an office.

Employees wear ID cards.

Department = HR

Department = Finance

Department = IT

The company identifies employees using these tags.

Labels work the same way.

Common Labels

labels:

app: nginx

env: prod

version: v1

team: backend

You can create any label you want.

What is a Selector?

A Selector is used to **find objects based on labels**.

Example:

selector:

matchLabels:

app: nginx

This means:

Find all Pods where `app=nginx`.

Interview Answer

Q: What is a Selector?

Answer:

"A Selector is used to identify Kubernetes objects based on their labels. Controllers like ReplicaSets and Services use selectors to find the Pods they should manage."

How Labels & Selectors Work

Suppose you have these Pods:

Pod1

app=nginx

Pod2

app=nginx

Pod3

app=mysql

ReplicaSet Selector:

matchLabels:

app: nginx

Result:

ReplicaSet

↓

Pod1 

Pod2 

Pod3 

Only Pods with `app=nginx` are selected.

Where are Labels Used?

Labels are used by:

- ReplicaSet
- Deployment
- Service
- Network Policy
- DaemonSet
- StatefulSet

Almost every Kubernetes object uses labels.

What Happens if Labels Don't Match?

Suppose ReplicaSet expects:

app: nginx

But Pod has:

app: apache

Result:

ReplicaSet cannot find the Pod.

It thinks no Pods exist.

So it creates another Pod.

This can lead to extra Pods.

Production Example

Suppose you have:

Frontend Pods

app=frontend

Backend Pods

app=backend

Database Pods

app=mysql

A Service for the frontend uses:

selector:

app: frontend

Only frontend Pods receive traffic.

Important Commands

View Labels

```
kubectl get pods --show-labels
```

View Pod Details

```
kubectl describe pod <pod-name>
```

Filter Pods by Label

```
kubectl get pods -l app=nginx
```

Filter by Environment

```
kubectl get pods -l env=prod
```


Edit Labels

```
kubectl label pod <pod-name> env=dev
```

Remove Label

```
kubectl label pod <pod-name> env-
```

Interview Questions

Q1. What are Labels?

Answer:

"Labels are key-value pairs used to identify and organize Kubernetes resources."

Q2. What are Selectors?

Answer:

"Selectors use labels to identify which resources Kubernetes should manage."

Q3. Why are Labels important?

Answer:

"Without labels, Kubernetes cannot identify which Pods belong to a Deployment, ReplicaSet, or Service."

Q4. Can two Pods have the same label?

Answer:

"Yes. In fact, that's common. A ReplicaSet or Service usually manages multiple Pods with the same label."

Q5. What happens if the Selector doesn't match any Pods?

Answer:

"Controllers like ReplicaSet create new Pods because they think the required Pods don't exist."

Q6. Can we change labels after creating a Pod?

Answer:

"Yes. We can update or add labels using the `kubectl label` command."

Common Problems

Problem: Service is not routing traffic.

Possible reason:

The Service selector doesn't match the Pod labels.

Problem: ReplicaSet keeps creating Pods.

Possible reason:

Pod labels don't match the ReplicaSet selector.

Problem: Deployment shows zero available Pods.

Possible reason:

Incorrect labels or selectors.

SRE Production Scenario

Suppose users report that the application is down.

Pods are running, but the Service isn't sending traffic.

First, I would check:

```
kubectl get pods --show-labels
```

```
kubectl describe svc <service-name>
```

Then compare:

- Pod labels
- Service selector

In many cases, the labels don't match, so the Service has no endpoints.

Key Points

- Labels are key-value pairs.
 - Selectors find resources using labels.
 - ReplicaSets, Deployments, and Services depend on labels.
 - Wrong labels can cause Pods not to be managed or Services not to route traffic.
 - Labels are one of the core concepts in Kubernetes.
-

★ SRE Interview Tip

Interviewer: *"Your Pods are running, but users can't access the application. What would you check first?"*

Candidate Answer:

"First, I'd check whether the Service selector matches the Pod labels using `kubectl get pods --show-labels` and `kubectl describe svc`. If the labels don't match, the Service won't have any endpoints to send traffic to."



Commands to add to your notebook

```
kubectl get pods --show-labels
```

```
kubectl get pods -l app=nginx
```

```
kubectl get pods -l env=prod
```

```
kubectl label pod <pod-name> env=dev
```

```
kubectl label pod <pod-name> env-
```

```
kubectl describe pod <pod-name>
```



Next Section: Service (Deep Dive)

This is one of the **top 5 Kubernetes interview topics**. We'll cover:

- Why Pods cannot be accessed directly
- ClusterIP
- NodePort
- LoadBalancer
- ExternalName
- Endpoints
- kube-proxy's role
- Service troubleshooting
- Real production interview scenarios

Phase 2 – Section 17: DaemonSet (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a DaemonSet is.
- Know why we use it.
- Understand how it differs from Deployment.
- Answer DaemonSet interview questions confidently.
- Know common production use cases.

What is a DaemonSet?

A DaemonSet is a Kubernetes object that ensures **one Pod runs on every worker node**.

Whenever a new node joins the cluster, Kubernetes automatically creates a Pod on that node.

If a node is removed, the Pod on that node is also removed.

Interview Answer

Q: What is a DaemonSet?

Answer:

"A DaemonSet ensures that one copy of a Pod runs on every node in the Kubernetes cluster. It is commonly used for monitoring, logging, and networking agents."

Why do we need a DaemonSet?

Suppose you have 5 worker nodes.

You want every node to send logs to Elasticsearch.

If you use a Deployment:

Deployment

↓

3 Pods

Only 3 Pods are created.

Two nodes won't have a logging agent.

If you use a DaemonSet:

Worker Node 1 → Log Agent

Worker Node 2 → Log Agent

Worker Node 3 → Log Agent

Worker Node 4 → Log Agent

Worker Node 5 → Log Agent

Every node gets exactly one Pod.

Real-Life Example

Imagine a security guard for every building.

Company has:

Building A

Building B

Building C

You need one security guard at each building.

If a new building is opened,

a new security guard is assigned automatically.

DaemonSet works the same way.

How DaemonSet Works

Suppose your cluster has:

Node1

Node2

Node3

DaemonSet creates:

Node1 → Monitoring Pod

Node2 → Monitoring Pod

Node3 → Monitoring Pod

Now a new node joins.

Node4

DaemonSet immediately creates:

Node4 → Monitoring Pod

No manual action is needed.

Architecture

DaemonSet

|

|— Node1 → Pod

|— Node2 → Pod

|— Node3 → Pod

└— Node4 → Pod

One node = One Pod.

When do we use DaemonSet?

Common production examples:

- Fluentd (Log Collection)
- Filebeat
- Prometheus Node Exporter
- Datadog Agent
- New Relic Agent
- Security Agents
- kube-proxy
- CNI plugins like Calico

These need to run on every node.

DaemonSet YAML

apiVersion: apps/v1

kind: DaemonSet

metadata:

name: nginx-daemon

spec:

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx

Notice:

There is **no replicas field**.

Because Kubernetes decides the number of Pods based on the number of nodes.

Deployment vs DaemonSet

Deployment

DaemonSet

Runs the required number of Pods

Runs one Pod on every node

Used for applications

Used for node-level services

Can scale replicas

Automatically scales with nodes

Example: Nginx, Spring Boot

Example: Fluentd, Datadog, kube-proxy

Interview Question

Q: Why doesn't DaemonSet have a replicas field?

Answer:

"Because Kubernetes automatically creates one Pod per node. The number of Pods depends on the number of nodes in the cluster."

What happens when a new node joins?

Example:

Current Cluster

Node1

Node2

Node3

DaemonSet

3 Pods

New Node joins.

Node4

Automatically:

Node4 → DaemonSet Pod

No one needs to create it manually.

What happens if a node is removed?

Suppose:

Node3

is deleted.

The Pod running on Node3 also disappears.

DaemonSet now runs Pods only on the remaining nodes.

Production Scenario

Suppose your company uses **Datadog** for monitoring.

Every node must send CPU, memory, disk, and network metrics.

You deploy the Datadog Agent as a DaemonSet.

When your Auto Scaling Group adds 20 new worker nodes,
Kubernetes automatically starts the Datadog Agent on all 20 nodes.
No manual deployment is required.

Common Problems

Problem: DaemonSet Pod not running on a node.

Possible reasons:

- Node is NotReady
 - Taints prevent scheduling
 - Insufficient resources
 - Node selector doesn't match
-

Problem: Only some nodes have DaemonSet Pods.

Possible reasons:

- Node Selector configured
 - Tolerations missing
 - Node labels don't match
-

Important Commands

Create DaemonSet

```
kubectl apply -f daemonset.yaml
```

View DaemonSets

```
kubectl get daemonsets
```

Short form

```
kubectl get ds
```

Describe DaemonSet

```
kubectl describe ds <daemonset-name>
```

View Pods

```
kubectl get pods -o wide
```

Delete DaemonSet

```
kubectl delete ds <daemonset-name>
```

Interview Questions

Q1. What is a DaemonSet?

Answer:

"A DaemonSet ensures one Pod runs on every node in the cluster."

Q2. Why do we use DaemonSet?

Answer:

"We use DaemonSets for applications that must run on every node, such as monitoring, logging, and networking agents."

Q3. Deployment vs DaemonSet?

Answer:

"Deployment runs a specified number of application Pods. DaemonSet runs one Pod on every node for node-level services."

Q4. Does DaemonSet support scaling?

Answer:

"Not by changing replicas. It automatically scales when nodes are added or removed."

Q5. Can DaemonSet run multiple Pods on the same node?

Answer:

"By default, no. It runs one Pod per eligible node."

Q6. Name some real-world examples of DaemonSets.

Answer:

"Fluentd, Filebeat, Datadog Agent, Prometheus Node Exporter, kube-proxy, and CNI plugins like Calico."

Key Points

- DaemonSet runs one Pod per node.
 - No **replicas** field is required.
 - Automatically adjusts when nodes are added or removed.
 - Mainly used for monitoring, logging, networking, and security agents.
 - Common examples include kube-proxy, Fluentd, Datadog Agent, and Node Exporter.
-

★ SRE Interview Tip

Interviewer: *"Your company adds 10 new worker nodes to the Kubernetes cluster. How will the monitoring agent be deployed to those nodes?"*

Candidate Answer:

"If the monitoring agent is deployed as a DaemonSet, Kubernetes automatically creates one Pod on each new node. No manual deployment is needed, so monitoring starts as soon as the nodes join the cluster."

Commands to add to your notebook

```
kubectl apply -f daemonset.yaml
```

```
kubectl get ds
```

```
kubectl describe ds <daemonset-name>
```

```
kubectl get pods -o wide
```

```
kubectl delete ds <daemonset-name>
```

Next Section: StatefulSet (Deep Dive)

This is one of the most frequently asked Kubernetes interview topics because interviewers often ask:

"If Deployment already manages Pods, why does Kubernetes need StatefulSet?"

Understanding that difference is essential for SRE interviews.

Phase 2 – Section 18: StatefulSet (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a StatefulSet is.
 - Know why Deployments cannot manage stateful applications.
 - Understand stable Pod names and persistent storage.
 - Answer StatefulSet interview questions confidently.
 - Know common production use cases.
-

What is a StatefulSet?

A StatefulSet is a Kubernetes object used to manage **stateful applications** like databases.

Unlike a Deployment, a StatefulSet gives each Pod:

- A **fixed Pod name**
 - A **fixed network identity**
 - Its own **persistent storage**
-

Interview Answer

Q: What is a StatefulSet?

Answer:

"A StatefulSet is used for stateful applications like databases. It provides stable Pod names, stable network identities, and dedicated persistent storage for each Pod."

What is a Stateful Application?

A stateful application stores important data.

Examples:

- MySQL
- PostgreSQL
- MongoDB
- Cassandra
- Kafka
- Elasticsearch

If these applications lose their data, they cannot work properly.

What is a Stateless Application?

A stateless application doesn't store important data locally.

Examples:

- Nginx
- React
- Spring Boot APIs
- Node.js APIs

If one Pod dies, another Pod can replace it without any problem.

Why can't Deployment manage databases?

Suppose you deploy MySQL using a Deployment.

Deployment

↓

mysql-pod

The Pod crashes.

Deployment creates a new Pod.

Old Pod ❌

↓

New Pod

The new Pod gets:

- New name
- New IP
- Empty storage (unless configured with persistent storage)

This is not suitable for databases.

StatefulSet Solution

StatefulSet creates Pods like this:

mysql-0

mysql-1

mysql-2

Notice:

The names never change.

If `mysql-1` crashes,

Kubernetes recreates:

mysql-1

Not:

mysql-xyz123

This is very important for database clusters.

Real-Life Example

Imagine a hotel.

Each guest has a room number.

Room 101

Room 102

Room 103

If a guest returns tomorrow,
they expect the same room.

StatefulSet works the same way.

Each Pod has a permanent identity.

Stable Storage

Each Pod gets its own Persistent Volume.

Example:

mysql-0

↓

PV-1

mysql-1

↓

PV-2

mysql-2

↓

PV-3

Even if the Pod restarts,
it reconnects to the same storage.

Architecture

StatefulSet

|

|— mysql-0 → PV-1

|— mysql-1 → PV-2

└— mysql-2 → PV-3

Pod Creation Order

Deployment:

Creates Pods in any order.

pod-3

pod-1

pod-2

No guarantee.

StatefulSet:

Creates Pods one by one.

mysql-0

↓

mysql-1

↓

mysql-2

Deletion also happens in reverse order.

mysql-2



mysql-1



mysql-0

Why is ordering important?

Imagine a database cluster.

Primary Node



Replica 1



Replica 2

The primary should start first.

Then replicas.

StatefulSet guarantees this order.

Deployment vs StatefulSet

Deployment	StatefulSet
Stateless apps	Stateful apps
Random Pod names	Fixed Pod names
Shared or temporary storage	Dedicated storage per Pod
Pods created in any order	Pods created in order
Example: Nginx	Example: MySQL

StatefulSet YAML

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: mysql

spec:

serviceName: mysql

replicas: 3

```
selector:

  matchLabels:

    app: mysql
```

```
template:

  metadata:

    labels:

      app: mysql
```

```
spec:

  containers:

    - name: mysql

      image: mysql:8
```

Don't memorize it.

Understand:

- **serviceName** → Gives stable DNS.
- **replicas** → Number of Pods.
- Each Pod gets its own storage.

Production Scenario

Suppose your company runs MongoDB in Kubernetes.

Three Pods:

mongo-0

mongo-1

mongo-2

If **mongo-1** crashes,

StatefulSet recreates:

mongo-1

with the same storage.

The database continues working without data loss.

Common Problems

Problem: Database lost data.

Possible reasons:

- No Persistent Volume
- Using Deployment instead of StatefulSet
- Storage deleted

Problem: Pod stuck in Pending.

Possible reasons:

- PVC not bound
- StorageClass issue
- No available storage

Problem: StatefulSet Pod not starting.

Possible reasons:

- Previous Pod not ready
- Storage issue
- Database initialization failed

Important Commands

Create StatefulSet

```
kubectl apply -f statefulset.yaml
```

View StatefulSets

```
kubectl get statefulsets
```

Short form

```
kubectl get sts
```

View Pods

```
kubectl get pods
```

Describe StatefulSet

```
kubectl describe sts <statefulset-name>
```

Delete StatefulSet

```
kubectl delete sts <statefulset-name>
```

Interview Questions

Q1. What is a StatefulSet?

Answer:

"A StatefulSet is used for stateful applications. It provides stable Pod names, stable network identities, and persistent storage."

Q2. Deployment vs StatefulSet?

Answer:

"Deployment is used for stateless applications. StatefulSet is used for stateful applications that require stable identities and persistent storage."

Q3. Why can't we use Deployment for MySQL?

Answer:

"A Deployment recreates Pods with new identities. Databases need stable identities and persistent storage, so StatefulSet is the better choice."

Q4. Give some StatefulSet examples.

Answer:

"MySQL, PostgreSQL, MongoDB, Cassandra, Kafka, Elasticsearch, and ZooKeeper."

Q5. Why are Pod names fixed in a StatefulSet?

Answer:

"Database nodes and clustered applications depend on stable hostnames to identify each other. StatefulSet provides fixed Pod names for this purpose."

Q6. Does every StatefulSet Pod get its own storage?

Answer:

"Yes. Each Pod gets its own Persistent Volume, which remains associated with that Pod even after a restart."

Key Points

- StatefulSet is used for stateful applications.
 - Pod names remain fixed.
 - Every Pod gets its own Persistent Volume.
 - Pods are created and deleted in order.
 - Commonly used for databases and clustered applications.
-

★ SRE Interview Tip

Interviewer: *"Your team wants to deploy MySQL on Kubernetes. Would you choose a Deployment or a StatefulSet? Why?"*

Candidate Answer:

"I'd use a StatefulSet because MySQL needs stable Pod names and persistent storage. If a Pod restarts, it should come back with the same identity and reconnect to its existing data. A Deployment doesn't provide those guarantees."

Commands to add to your notebook

```
kubectl apply -f statefulset.yaml
```

```
kubectl get sts
```

```
kubectl describe sts <statefulset-name>
```

```
kubectl get pods
```

```
kubectl delete sts <statefulset-name>
```

★ Before moving to Job

Here's a common interview question that many candidates struggle with:

Workload	Best Use Case
Pod	Single container for testing or learning
ReplicaSet	Maintain the desired number of Pods
Deployment	Stateless applications with rolling updates
DaemonSet	One Pod per node (monitoring, logging, networking)
StatefulSet	Databases and stateful applications with persistent storage

If you can explain this table in an interview, you'll answer a large number of workload-related questions confidently.

Next Section: Job (Deep Dive) – where we'll learn how Kubernetes runs one-time tasks like backups, report generation, and database migrations.

Phase 2 – Section 19: Job (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a Job is.
- Know why we use Jobs.
- Understand the difference between Job and Deployment.
- Answer Job interview questions confidently.
- Know common production use cases.

What is a Job?

A Job is a Kubernetes object used to run a task **only once**.

Unlike a Deployment, a Job is **not meant to run forever**.

Once the task finishes successfully, the Job is complete.

Interview Answer

Q: What is a Job?

Answer:

"A Job is a Kubernetes workload that runs a task to completion. Once the task finishes successfully, Kubernetes marks the Job as completed instead of keeping the Pod running."

Why do we need a Job?

Suppose you want to:

- Take a database backup
- Migrate database tables
- Generate a report
- Import data

These tasks should run once and then stop.

Using a Deployment would restart the Pod continuously.

A Job runs the task once and exits.

Real-Life Example

Imagine downloading your bank statement.

You download it once.

After it's downloaded, the task is finished.

You don't keep downloading it every second.

A Job works the same way.

How Job Works

You create a Job.

Job



Pod Starts



Task Executes



Task Completes



Pod Exits



Job Status = Completed

Unlike Deployments, the Pod is **not restarted after successful completion**.

Architecture

Job



Pod



Run Task



Completed

Job vs Deployment

Deployment	Job
Runs continuously	Runs once

Web applications

Batch tasks

Restarts to stay
available

Stops after success

Example: Nginx

Example: Database
backup

Production Examples

Jobs are commonly used for:

- Database backup
- Database migration
- Sending reports
- Importing CSV files
- Running scripts
- Cleaning temporary files

Job YAML

apiVersion: batch/v1

kind: Job

metadata:

name: backup-job

spec:

template:

spec:

containers:

- name: backup

image: busybox

command: ["echo", "Backup Completed"]

restartPolicy: Never

Important fields:

- `kind: Job`
- `restartPolicy: Never`
- `command` → Task to execute

What happens if the Job fails?

Suppose the task fails.

Example:

Backup Script

↓

Error

↓

Exit Code = 1

Kubernetes starts another Pod to retry the Job.

The number of retries depends on **backoffLimit**.

Example:

backoffLimit: 3

Kubernetes retries three times before marking the Job as failed.

Production Scenario

Suppose every night your company takes a database backup.

A Job starts.

Backup Starts

↓

Database Dump

↓

Upload to S3

↓

Job Completed

Once the backup finishes,

the Job ends.

No Pod keeps running.

Common Problems

Problem: Job never completes.

Possible reasons:

- Application stuck
 - Script waiting for input
 - Database unreachable
-

Problem: Job keeps failing.

Possible reasons:

- Wrong image
 - Script error
 - Database credentials incorrect
-

Problem: Job Pod in CrashLoopBackOff.

Possible reasons:

- Application crash
 - Wrong command
 - Missing ConfigMap or Secret
-

Important Commands

Create Job

```
kubectl apply -f job.yaml
```

View Jobs

kubecttl get jobs

Short form

kubecttl get job

View Pods

kubecttl get pods

Describe Job

kubecttl describe job backup-job

View Logs

kubecttl logs <pod-name>

Delete Job

kubecttl delete job backup-job

Interview Questions

Q1. What is a Job?

Answer:

"A Job is used to run a task once until it completes successfully."

Q2. When do we use a Job?**Answer:**

"We use Jobs for one-time tasks like database backups, migrations, report generation, or data imports."

Q3. Job vs Deployment?**Answer:**

"A Deployment keeps applications running continuously. A Job runs a task once and finishes."

Q4. What happens if a Job Pod fails?**Answer:**

"Kubernetes retries the Job based on the configured backoffLimit."

Q5. Does a Job run forever?**Answer:**

"No. A Job stops after the task completes successfully."

Q6. What is backoffLimit?**Answer:**

"backoffLimit defines how many times Kubernetes retries a failed Job before marking it as failed."

Key Points

- Job is used for one-time tasks.
- It stops after successful completion.
- Kubernetes retries failed Jobs.

- Common examples are backups, migrations, and batch processing.
 - Jobs are not suitable for web applications.
-

★ SRE Interview Tip

Interviewer: *"Your nightly database backup didn't complete. How would you troubleshoot?"*

Candidate Answer:

"First, I'd check the Job status using `kubectl get jobs`. Then I'd describe the Job with `kubectl describe job` to look for events. Next, I'd identify the Job Pod, check its logs using `kubectl logs`, and verify whether the backup script, database connectivity, or credentials caused the failure."

Commands to add to your notebook

`kubectl apply -f job.yaml`

`kubectl get jobs`

`kubectl describe job <job-name>`

`kubectl get pods`

`kubectl logs <pod-name>`

`kubectl delete job <job-name>`

★ Before moving to CronJob

A quick comparison:

Workload

Best Use Case

Deployment Long-running applications

DaemonSet One Pod on every node

StatefulSet Databases and stateful apps

Job One-time tasks like backups or migrations

Next Section: CronJob (Deep Dive) — where we'll learn how Kubernetes schedules Jobs automatically, just like Linux `cron`.

Phase 2 – Section 20: CronJob (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a CronJob is.
- Know why we use CronJobs.
- Understand the difference between Job and CronJob.
- Answer CronJob interview questions confidently.
- Know common production use cases.

What is a CronJob?

A CronJob is a Kubernetes object that runs a **Job automatically on a schedule**.

Think of it as the Kubernetes version of the Linux `cron` scheduler.

Instead of running a task manually, Kubernetes runs it automatically at the scheduled time.

Interview Answer

Q: What is a CronJob?

Answer:

"A CronJob is a Kubernetes object that creates Jobs automatically based on a schedule. It's commonly used for tasks like backups, cleanup, and report generation."

Why do we need a CronJob?

Suppose your company needs to:

- Take a database backup every night.
- Delete old log files every Sunday.
- Generate a daily sales report.
- Send weekly email reports.

Doing these manually is not practical.

A CronJob automates them.

Real-Life Example

Imagine setting an alarm for **6 AM every day**.

You don't manually create the alarm every morning.

The alarm rings automatically.

A CronJob works the same way.

Perfect. Now we're entering **Phase 3**, which is one of the most important phases for **Akamai**, because their business is heavily focused on networking.

We'll start with the most important networking concept.

Phase3 - Section 16: Services (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a Service is.
- Know why we need Services.
- Understand ClusterIP, NodePort, LoadBalancer, and ExternalName.
- Learn how Services use Labels and Selectors.

- Answer interview questions confidently.
 - Troubleshoot Service issues.
-

What is a Service?

A Service is a Kubernetes object that provides a **stable way to access Pods**.

Pods are temporary. They can be deleted, recreated, or get a new IP address.

A Service gives them one permanent access point.

Interview Answer

Q: What is a Service?

Answer:

"A Service provides a stable network endpoint for a group of Pods. Even if Pods are recreated and their IP addresses change, the Service continues to route traffic to the correct Pods."

Why do we need a Service?

Suppose you have one Pod.

Nginx Pod

IP = 10.244.0.5

Tomorrow the Pod crashes.

ReplicaSet creates a new Pod.

New Pod

IP = 10.244.0.12

The IP changed.

If users were using **10.244.0.5**, the application becomes unreachable.

A Service solves this problem.

User



Service (Stable IP)



Pod1

Pod2

Pod3

The Service IP never changes.

Real-Life Example

Think of a customer care number.

The company may have 100 employees answering calls.

Employees change every day.

But the customer always dials the same number.

That phone number is like a **Service**.

The employees are like **Pods**.

How Service Works

User

↓

Service

↓

Selector (app=nginx)

↓

Pod1

Pod2

Pod3

The Service finds Pods using **Labels and Selectors**.

Important Interview Question

Q: Does a Service communicate with Pods using Pod names?

Answer:

"No. A Service uses labels and selectors to identify which Pods should receive traffic."

Types of Services

1. ClusterIP (Default)

Used for communication **inside the Kubernetes cluster**.

Example:

Frontend Pod → Backend Service

Users outside the cluster cannot access it.

Architecture:

Frontend Pod

↓

Backend Service

↓

Backend Pods

Interview Answer

Q: What is ClusterIP?

Answer:

"ClusterIP is the default Service type. It allows communication between applications inside the Kubernetes cluster."

2. NodePort

NodePort exposes the application on every Kubernetes node.

Example:

Browser

↓

NodeIP:30080

↓

Service

↓

Pods

NodePort range:

30000 - 32767

Interview Answer

Q: What is NodePort?

Answer:

"NodePort exposes an application on a port of every worker node, allowing external users to access it using the node's IP address and port."

3. LoadBalancer

Mainly used in cloud environments like AWS, Azure, or GCP.

Cloud provider automatically creates an external Load Balancer.

Architecture:

Internet

↓

AWS Load Balancer

↓

Service

↓

Pods

Interview Answer

Q: What is a LoadBalancer Service?

Answer:

"A LoadBalancer Service creates an external load balancer in cloud environments and routes internet traffic to the application's Pods."

4. ExternalName

Used when Kubernetes needs to connect to an external service.

Example:

Application

↓

ExternalName Service

↓

database.company.com

Interview Answer

Q: What is ExternalName?

Answer:

"ExternalName maps a Kubernetes Service to an external DNS name instead of routing traffic to Pods."

Service YAML

apiVersion: v1

kind: Service

metadata:

name: nginx-service

spec:

selector:

app: nginx

ports:

- port: 80

targetPort: 80

type: ClusterIP

Important fields:

- selector → Finds Pods
- port → Service port
- targetPort → Container port
- type → ClusterIP, NodePort, LoadBalancer, ExternalName

port vs targetPort

Suppose your container listens on port **8080**.

Users should connect using port **80**.

User

↓

Service Port : 80

↓

Target Port : 8080

↓

Container

Interview Question

Q: What is the difference between port and targetPort?

Answer:

"The Service port is the port clients connect to. The targetPort is the port on which the application is running inside the Pod."

Endpoints

When a Service finds matching Pods,
it creates **Endpoints**.

Example:

Service

↓

Endpoints

↓

10.244.0.5

10.244.0.6

10.244.0.7

Traffic is sent to these endpoint IPs.

Interview Question

Q: What are Endpoints?

Answer:

"Endpoints are the list of Pod IP addresses that a Service sends traffic to."

Production Scenario

Suppose your Deployment has 5 Pods.

One Pod crashes.

ReplicaSet creates another Pod.

The Service automatically updates its Endpoints.

Users continue accessing the application without changing anything.

Common Problems

Problem: Service not accessible.

Possible reasons:

- Wrong selector
 - Pod not running
 - Wrong targetPort
 - Firewall or Security Group issue
-

Problem: No Endpoints.

Check:

`kubectl get endpoints`

Usually the selector doesn't match the Pod labels.

Problem: Users get Connection Refused.

Possible reasons:

- Application not listening on the targetPort.
- Wrong container port.
- Application crashed.

Important Commands

Create Service

```
kubectl apply -f service.yaml
```

View Services

```
kubectl get svc
```

Describe Service

```
kubectl describe svc nginx-service
```

View Endpoints

```
kubectl get endpoints
```

Delete Service

```
kubectl delete svc nginx-service
```

Interview Questions

Q1. Why do we need a Service?

Answer:

"Pods are temporary and their IP addresses can change. A Service provides a stable IP and DNS name so applications can always reach the Pods."

Q2. What are the different Service types?

Answer:

"ClusterIP, NodePort, LoadBalancer, and ExternalName."

Q3. Which Service type is the default?

Answer:

"ClusterIP."

Q4. How does a Service find Pods?

Answer:

"It uses labels and selectors."

Q5. What is the difference between ClusterIP and NodePort?

Answer:

"ClusterIP is used only inside the cluster. NodePort exposes the application externally through a port on every worker node."

Q6. Why are there no Endpoints for my Service?

Answer:

"Usually because the Service selector doesn't match the Pod labels, or the Pods are not running."

Q7. What is the role of kube-proxy in a Service?

Answer:

"kube-proxy runs on every node and manages the networking rules that route Service traffic to the correct Pod endpoints."

Key Points

- A Service provides a stable IP and DNS name.
 - Services use labels and selectors to find Pods.
 - ClusterIP is the default Service type.
 - NodePort exposes applications using a node's IP.
 - LoadBalancer is commonly used in cloud environments.
 - Endpoints contain the IP addresses of the Pods behind the Service.
 - kube-proxy handles traffic routing to the Pods.
-

★ SRE Interview Tip

Interviewer: *"Your Pods are running, but the application is not accessible through the Service. How would you troubleshoot?"*

Candidate Answer:

"First, I'd verify the Pods are running using `kubectl get pods`. Then I'd check the Service with `kubectl describe svc` and verify the selector matches the Pod labels. Next, I'd check the Endpoints using `kubectl get endpoints`. If there are no endpoints, it's usually a selector mismatch or the Pods are unhealthy. Finally, I'd confirm the application is listening on the correct targetPort."

📌 Commands to add to your notebook

```
kubectl get svc
```

```
kubectl describe svc <service-name>
```

```
kubectl get endpoints
```

```
kubectl apply -f service.yaml
```

```
kubectl delete svc <service-name>
```

```
kubectl expose deployment nginx --port=80 --target-port=80
```



Next Section: Ingress7 (Deep Dive)

We'll learn:

- Why Ingress is needed if we already have Services.
- What an Ingress Controller is.
- Host-based routing.
- Path-based routing.
- SSL/TLS termination.
- How traffic flows from the internet to your Pods.

Common Ingress interview questions and troubleshooting.

Phase 3 – Section 21: ClusterIP (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what ClusterIP is.
- Know why Kubernetes created ClusterIP.
- Understand how Pods communicate.
- Answer ClusterIP interview questions confidently.
- Troubleshoot ClusterIP issues.

What is ClusterIP?

ClusterIP is the **default Service type** in Kubernetes.

It provides a **stable internal IP address** so Pods inside the cluster can communicate with each other.

Only applications **inside the cluster** can access it.

Interview Answer

Q: What is ClusterIP?

Answer:

"ClusterIP is the default Kubernetes Service type. It provides a stable internal IP address that allows Pods inside the cluster to communicate with each other."

Why do we need ClusterIP?

Suppose your frontend application needs to call the backend.

Without ClusterIP

Frontend

↓

Backend Pod

IP = 10.244.1.5

Now the backend Pod crashes.

A new Pod starts.

Backend Pod

IP = 10.244.3.20

The frontend still tries to connect to:

10.244.1.5

Connection fails.

With ClusterIP

Frontend



Backend Service

10.96.10.5



Backend Pods

Even if Pods change,

the Service IP remains the same.

Frontend doesn't need to know Pod IPs.

Real-Life Example

Imagine ordering food.

You don't call the chef directly.

You call the restaurant.

The restaurant sends your order to whichever chef is available.

Customer



Restaurant Number



Chef 1

Chef 2

Chef 3

Restaurant = ClusterIP Service

Chefs = Pods

Traffic Flow

When a frontend Pod sends a request:

Frontend Pod



ClusterIP Service



kube-proxy



Backend Pod

Notice something important.

Traffic **does not go directly** from the Service to the Pod.

kube-proxy handles the routing.

This is a common interview question.

How does ClusterIP find Pods?

Suppose your Service has:

selector:

app: backend

Pods:

Pod A

app=backend

Pod B

app=backend

Pod C

app=mysql

ClusterIP selects:

Pod A 

Pod B 

Pod C 

using labels.

Where is ClusterIP used?

Very common examples:

Frontend → Backend

React

↓

Spring Boot

Backend → Database

Spring Boot

↓

MySQL

Microservice → Microservice

Order Service

↓

Payment Service

All of these usually use ClusterIP.

Can users on the internet access ClusterIP?

No.

ClusterIP is only accessible **inside the Kubernetes cluster**.

This is one of the most common interview questions.

Production Example

Suppose your company has:

Frontend

Backend

Database

Communication:

Frontend

↓

ClusterIP

↓

Backend

↓

ClusterIP

↓

MySQL

Customers only access the frontend.

Backend and database remain private.

This improves security.

Common Problems

Problem: Frontend cannot call backend.

Possible reasons:

- Service selector incorrect
 - Backend Pods not running
 - Wrong Service name
 - DNS issue
-

Problem: No Endpoints.

Check:

`kubectl get endpoints`

Usually labels don't match.

Problem: Connection refused.

Possible reasons:

- Application not listening on targetPort
 - Wrong container port
 - Backend application crashed
-

Important Commands

View Services

```
kubectl get svc
```

Describe Service

```
kubectl describe svc backend-service
```

View Endpoints

```
kubectl get endpoints
```

View Pod Labels

```
kubectl get pods --show-labels
```

Test Connectivity

```
kubectl exec -it <pod-name> -- curl http://backend-service
```

Interview Questions

Q1. What is ClusterIP?

Answer:

"ClusterIP is the default Service type that provides a stable internal IP for communication between Pods inside the Kubernetes cluster."

Q2. Can I access ClusterIP from my browser?

Answer:

"No. ClusterIP is only accessible from inside the Kubernetes cluster."

Q3. Why do we need ClusterIP?

Answer:

"Pods are temporary and their IP addresses change. ClusterIP gives applications a stable IP and DNS name."

Q4. How does ClusterIP know which Pods to send traffic to?

Answer:

"It uses labels and selectors to identify the correct Pods."

Q5. Does ClusterIP perform load balancing?

Answer:

"Yes. It distributes requests among the healthy Pods behind the Service."

Q6. What component actually routes the traffic?

Answer:

"kube-proxy configures the networking rules on each node to route traffic from the ClusterIP Service to the backend Pods."

Key Points

- ClusterIP is the default Service type.
 - Used only for internal communication.
 - Provides a stable IP and DNS name.
 - Uses labels and selectors.
 - kube-proxy routes the traffic.
 - Commonly used in microservices.
-

★ SRE Interview Tip

Interviewer: *"The frontend cannot reach the backend Service. What will you check?"*

Candidate Answer:

"First, I'd verify that the backend Pods are running using `kubectl get pods`. Then I'd check the Service with `kubectl describe svc` and confirm the selector matches the Pod labels. Next, I'd check the Endpoints using `kubectl get endpoints`. Finally, I'd test connectivity from the frontend Pod using `curl` or `wget` to the Service name."

Commands to add to your notebook

```
kubectl get svc
```

```
kubectl describe svc <service-name>
```

```
kubectl get endpoints
```

```
kubectl get pods --show-labels
```

```
kubectl exec -it <pod-name> -- curl http://<service-name>
```



Next Section: NodePort (Deep Dive)

We'll answer:

- Why isn't ClusterIP enough?
- How can users outside the cluster access an application?
- How does NodePort work internally?
- NodePort vs ClusterIP vs LoadBalancer (very common interview question)
- Real production scenarios and troubleshooting.

Phase 3 – Section 22: NodePort (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what NodePort is.
 - Know why we need NodePort.
 - Understand how external users access an application.
 - Answer NodePort interview questions confidently.
 - Troubleshoot NodePort issues.
-

What is NodePort?

NodePort is a Kubernetes Service type that exposes an application **outside the Kubernetes cluster**.

It opens a port on every worker node, allowing users to access the application using the node's IP address and port.

Interview Answer

Q: What is NodePort?

Answer:

"NodePort is a Service type that exposes an application on a fixed port of every Kubernetes node, allowing external users to access it using the node's IP address and the NodePort."

Why do we need NodePort?

Suppose you have a web application.

Using ClusterIP:

Browser

 Cannot Access

↓

ClusterIP

Because ClusterIP works only inside the cluster.

Now use NodePort.

Browser



192.168.1.20:30080



NodePort Service



Pods

Now users outside the cluster can access the application.

Real-Life Example

Imagine an apartment.

ClusterIP is like an **internal extension number**.

Only people inside the apartment can call it.

NodePort is like the **main gate**.

Visitors enter through the gate before reaching any apartment.

How NodePort Works

Internet



Worker Node IP:30080



NodePort Service



kube-proxy



Backend Pods

Traffic Flow:

1. User sends request.
2. Request reaches Worker Node.

3. kube-proxy receives it.
4. kube-proxy forwards it to a healthy Pod.

Default NodePort Range

Kubernetes reserves these ports:

30000 - 32767

Example:

Node IP = 192.168.1.20

NodePort = 30080

Access URL:

`http://192.168.1.20:30080`

Production Example

Suppose you have an internal application.

Employees access:

`http://10.0.0.15:30080`

Request Flow:

Employee

↓

Worker Node

↓

NodePort

↓

Application Pods

NodePort YAML

apiVersion: v1

kind: Service

metadata:

name: nginx-service

spec:

type: NodePort

selector:

app: nginx

ports:

- port: 80

targetPort: 80

nodePort: 30080

Important fields:

- `type: NodePort`
 - `port` → Service port
 - `targetPort` → Container port
 - `nodePort` → External port on the node
-

ClusterIP vs NodePort

ClusterIP	NodePort
Internal only	External access
Default Service	Opens a port on every node
Used between Pods	Used by external users
No Node IP required	Requires Node IP + Port

Interview Question

Q: Can NodePort be accessed from outside the cluster?

Answer:

"Yes. Users access it using the worker node's IP address and the configured NodePort."

Interview Question

Q: Does NodePort create a Load Balancer?

Answer:

"No. It only opens a port on every worker node. If we need a cloud load balancer, we use the LoadBalancer Service type."

Common Problems

Problem: NodePort not accessible.

Possible reasons:

- Application Pod not running
 - Service selector incorrect
 - Firewall blocking the NodePort
 - Security Group blocking the port
 - Wrong Node IP
-

Problem: Connection timed out.

Possible reasons:

- Node unreachable
 - kube-proxy issue
 - Network ACL or firewall issue
-

Problem: Service has no endpoints.

Check:

```
kubectl get endpoints
```

Usually:

- Labels don't match.
- Pods are not Ready.

Important Commands

View Services

```
kubectl get svc
```

Describe Service

```
kubectl describe svc nginx-service
```

View Endpoints

```
kubectl get endpoints
```

View Pods

```
kubectl get pods -o wide
```

Test from Inside Cluster

```
kubectl exec -it <pod-name> -- curl http://nginx-service
```

Interview Questions

Q1. What is NodePort?**Answer:**

"NodePort exposes an application outside the Kubernetes cluster by opening a port on every worker node."

Q2. What is the default NodePort range?**Answer:**

"30000 to 32767."

Q3. How do users access a NodePort Service?**Answer:**

"Using the worker node's IP address and the NodePort, for example
<http://192.168.1.20:30080>."

Q4. ClusterIP vs NodePort?**Answer:**

"ClusterIP is for internal communication between Pods. NodePort allows external users to access the application."

Q5. Why isn't NodePort commonly used in production?**Answer:**

"Because users need to know the node's IP and port. In cloud environments, we usually use a LoadBalancer or Ingress, which provide a better and more scalable way to expose applications."

Key Points

- NodePort exposes applications outside the cluster.

- Uses the worker node's IP address.
 - Default port range is 30000–32767.
 - kube-proxy routes traffic to the Pods.
 - Commonly used for testing or small environments.
-

★ SRE Interview Tip

Interviewer: *"Users can't access the application through NodePort. How would you troubleshoot?"*

Candidate Answer:

"First, I'd check if the Pods are running using `kubectl get pods`. Then I'd verify the Service using `kubectl get svc` and `kubectl describe svc`. I'd check the Endpoints to make sure the Service has healthy backend Pods. Finally, I'd verify the node's firewall or cloud Security Group allows the NodePort, and confirm I'm using the correct node IP and port."

Commands to add to your notebook

```
kubectl get svc
```

```
kubectl describe svc <service-name>
```

```
kubectl get endpoints
```

```
kubectl get pods -o wide
```

```
kubectl exec -it <pod-name> -- curl http://<service-name>
```

★ Quick Interview Tip

Many interviewers ask this sequence:

How do users access an application in Kubernetes?

A good answer is:

- **ClusterIP** → Internal communication only.
- **NodePort** → External access using `NodeIP:NodePort`.
- **LoadBalancer** → External access through a cloud load balancer.
- **Ingress** → One entry point for multiple applications with routing and TLS.

This answer naturally leads us to the next topic.



Next Section: LoadBalancer (Deep Dive)

We'll cover:

- Why NodePort is not enough in production.
- How AWS, Azure, and GCP create external load balancers.
- Traffic flow from the Internet to Pods.
- LoadBalancer vs NodePort vs Ingress.
- Real production scenarios and interview questions.

Phase 3 – Section 23: LoadBalancer (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what a LoadBalancer Service is.
- Know why we need it.
- Understand how cloud providers create Load Balancers.
- Answer LoadBalancer interview questions confidently.
- Troubleshoot LoadBalancer issues.

What is a LoadBalancer Service?

A **LoadBalancer** is a Kubernetes Service type that exposes an application to the **internet** using a **cloud provider's load balancer**.

When you create a Service of type `LoadBalancer`, Kubernetes asks the cloud provider (AWS, Azure, GCP) to create an external Load Balancer automatically.

Interview Answer

Q: What is a LoadBalancer Service?

Answer:

"A LoadBalancer Service exposes an application to the internet by creating a cloud load balancer. It distributes incoming traffic across backend Pods."

Why do we need LoadBalancer?

Suppose you deployed an application using **NodePort**.

Users access it like this:

`http://54.10.20.30:30080`

Problems:

- Users must know the node IP.
- If the node changes, the IP changes.
- Not user-friendly.
- Not suitable for production.

Instead:

`https://mycompany.com`

The Load Balancer receives the request and forwards it to healthy Pods.

Real-Life Example

Imagine a hospital.

Patients don't go directly to doctors.

They first go to the **Reception Desk**.

The receptionist sends them to an available doctor.

Patient

↓

Reception

↓

Doctor 1

Doctor 2

Doctor 3

Here,

- Reception = Load Balancer
- Doctors = Pods

How LoadBalancer Works

Internet

↓

Cloud Load Balancer

↓

NodePort Service

↓

kube-proxy

↓

Pods

Notice something important.

The Load Balancer doesn't directly communicate with Pods.

It usually forwards traffic to the **NodePort Service**, which then routes traffic to the Pods.

Traffic Flow

Example:

Customer opens:

<https://shop.example.com>

Flow:

Browser



AWS Load Balancer



Worker Node



NodePort



kube-proxy



Backend Pod

Production Example

Suppose you're running an e-commerce website.

Customers



AWS Application Load Balancer



Kubernetes Service



Frontend Pods

If one Pod crashes:

Pod-1 ❌

Pod-2 ✅

Pod-3 

Traffic is automatically sent only to healthy Pods.

Customers don't notice the failure.

LoadBalancer YAML

apiVersion: v1

kind: Service

metadata:

name: web-service

spec:

type: LoadBalancer

selector:

app: nginx

ports:

- port: 80

targetPort: 80

Notice:

We don't specify an external IP.

The cloud provider automatically creates it.

How Kubernetes Creates a Load Balancer

You apply:

```
kubectl apply -f service.yaml
```

Flow:

```
kubectl
```

↓

API Server

↓

Cloud Controller Manager

↓

AWS / Azure / GCP API

↓

Cloud Load Balancer Created

↓

External IP Assigned

The **Cloud Controller Manager** talks to the cloud provider and creates the load balancer.

LoadBalancer vs NodePort

NodePort	LoadBalancer
Uses Node IP + Port	Uses External IP or DNS
Manual access	Automatic cloud load balancer
Mainly for testing	Production environments
No cloud integration	Cloud-managed

LoadBalancer vs ClusterIP

ClusterIP	LoadBalancer
Internal communication	Internet-facing
No external access	External access
Default Service	Creates cloud load balancer

Common Problems

Problem: External IP is Pending

Example:

```
kubectl get svc
```

Output:

NAME	TYPE	EXTERNAL-IP
web-service	LoadBalancer	<pending>

Possible reasons:

- No cloud provider configured
- Running on Minikube or Kind
- Cloud Controller Manager not working

- Cloud permissions missing
-

Problem: Users can't access the application

Possible reasons:

- Security Group blocking port 80 or 443
 - Firewall issue
 - Pod not running
 - Wrong selector
 - No Endpoints
-

Problem: Health checks failing

Possible reasons:

- Application not listening on the correct port
 - Readiness probe failing
 - Service targetPort incorrect
-

Important Commands

View Services

```
kubectl get svc
```

Describe Service

```
kubectl describe svc web-service
```

View Endpoints

```
kubectl get endpoints
```


View Pods

```
kubecttl get pods -o wide
```

Check Events

```
kubecttl get events
```

Interview Questions

Q1. What is a LoadBalancer Service?

Answer:

"A LoadBalancer Service exposes an application to the internet by creating a cloud load balancer that forwards traffic to backend Pods."

Q2. Who creates the Load Balancer?

Answer:

"The Cloud Controller Manager communicates with the cloud provider's API to create the load balancer."

Q3. Does LoadBalancer work on Kind or Minikube?

Answer:

"Not by default. Since Kind and Minikube don't have a cloud provider, the external IP usually stays in the Pending state unless you install additional tools like MetalLB."

Q4. Why is LoadBalancer preferred over NodePort?

Answer:

"LoadBalancer provides a stable external IP or DNS name, automatically distributes traffic, and integrates with cloud infrastructure. It's much better suited for production."

Q5. Does LoadBalancer send traffic directly to Pods?

Answer:

"Usually no. It sends traffic to the Service (typically through a NodePort), and kube-proxy forwards the traffic to healthy Pods."

Q6. What happens if one Pod crashes?

Answer:

"The Service removes the unhealthy Pod from its endpoints, and traffic is sent only to the remaining healthy Pods."

Key Points

- LoadBalancer exposes applications to the internet.
 - It is mainly used in cloud environments like AWS, Azure, and GCP.
 - Cloud Controller Manager creates the cloud load balancer.
 - Users access the application through an external IP or DNS.
 - It is the preferred choice for production when exposing a single application.
-

SRE Interview Tip

Interviewer: "You created a LoadBalancer Service on EKS, but the EXTERNAL-IP is still Pending. How would you troubleshoot?"

Candidate Answer:

"First, I'd check the Service events using `kubectl describe svc` to see if there are provisioning errors. Then I'd verify that the AWS Load Balancer Controller or Cloud Controller Manager is working correctly. I'd also check IAM permissions, subnet tags, and whether the cluster is running in a supported cloud environment. If this happened on Kind or Minikube, I'd expect the external IP to remain Pending because there's no cloud provider."

Commands to add to your notebook

```
kubectl get svc
```

```
kubectl describe svc <service-name>
```

```
kubectl get endpoints
```

```
kubectl get pods -o wide
```

```
kubectl get events
```

Quick Comparison

Service Type	Used For	Access
ClusterIP	Pod-to-Pod communication	Internal only
NodePort	Basic external access	<code>NodeIP:Port</code>
LoadBalancer	Production external access	External IP or DNS



Next Topic: Ingress (Deep Dive)

This is one of the **most frequently asked Akamai interview topics** because it covers:

- Why LoadBalancer is expensive
- How one LoadBalancer can expose multiple applications
- Path-based routing (`/api`, `/login`)
- Host-based routing (`app.example.com`, `admin.example.com`)
- TLS/SSL termination
- Ingress Controller (NGINX, AWS ALB)

We'll also cover the complete request flow from **Browser** → **Ingress** → **Service** → **Pod**, which is a favorite interview question.

Phase 3 – Section 24: Ingress (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what Ingress is.
- Know why we need Ingress when we already have a LoadBalancer.
- Understand Ingress Controller.
- Learn path-based and host-based routing.
- Answer Ingress interview questions confidently.
- Troubleshoot Ingress issues.

What is Ingress?

Ingress is a Kubernetes resource that manages **HTTP and HTTPS traffic** entering the cluster.

Instead of creating a separate LoadBalancer for every application, Ingress allows **one LoadBalancer to expose multiple applications**.

Interview Answer

Q: What is Ingress?

Answer:

"Ingress is a Kubernetes resource that routes external HTTP and HTTPS traffic to different Services based on the request's host or URL path. It requires an Ingress Controller to work."

Why do we need Ingress?

Suppose your company has three applications.

- Shopping App
- Payment App
- Admin Portal

Without Ingress:

Shopping App → LoadBalancer

Payment App → LoadBalancer

Admin App → LoadBalancer

Three applications = Three LoadBalancers

Problems:

- Expensive
 - Difficult to manage
 - Multiple public IPs
-

With Ingress:

Internet



One LoadBalancer



Ingress

|— shop.example.com → Shopping Service

|— pay.example.com → Payment Service

|— admin.example.com → Admin Service

One LoadBalancer serves multiple applications.

Real-Life Example

Think of an airport.

Passengers enter through **one main entrance**.

Security checks their boarding pass and sends them to the correct gate.

Passenger



Airport Entrance



Gate A

Gate B

Gate C

Here,

- Airport Entrance = Ingress
- Gates = Services

Ingress simply routes traffic.

Important Point

Ingress does NOT forward traffic by itself.

It needs an **Ingress Controller**.

Without an Ingress Controller,

Ingress is just a configuration.

This is one of the most common interview questions.

What is an Ingress Controller?

The Ingress Controller is the component that reads the Ingress rules and routes traffic.

Common controllers:

- NGINX Ingress Controller
 - AWS Load Balancer Controller
 - HAProxy Ingress
 - Traefik
-

How Ingress Works

Browser

↓

Load Balancer

↓

Ingress Controller

↓

Ingress Rules

↓

Service

↓

Pods

Remember this flow.

Path-Based Routing

Example:

example.com/api

↓

API Service

example.com/login

↓

Login Service

example.com/images

↓

Image Service

Different URLs go to different Services.

Host-Based Routing

Example:

shop.example.com

↓

Shopping Service

admin.example.com



Admin Service

api.example.com



API Service

Different domains go to different Services.

Production Example

Your company owns:

www.company.com

admin.company.com

api.company.com

Traffic:

Internet

↓

Ingress

└─ www → Frontend

└─ admin → Admin Service

└─ api → Backend Service

Customers use different URLs, but only one LoadBalancer is required.

Ingress YAML

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: web-ingress

spec:

rules:

- host: shop.example.com

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: shop-service

port:

number: 80

Don't memorize every field.

Understand:

- `host` → Domain name
 - `path` → URL path
 - `service.name` → Destination Service
-

Can Ingress work without a LoadBalancer?

Usually in cloud environments:

Internet



Cloud Load Balancer



Ingress Controller



Ingress Rules



Services



Pods

The LoadBalancer brings traffic into the cluster.

Ingress decides where it should go.

TLS/SSL Termination

Ingress can also handle HTTPS.

Example:

Browser



HTTPS



Ingress Controller



HTTP



Application

The SSL certificate is configured at the Ingress Controller.

The application doesn't need to manage certificates itself.

Ingress vs LoadBalancer

LoadBalancer

Exposes one Service

Creates cloud
LoadBalancer

Doesn't route by URL

Higher cost for many apps

Ingress

Routes to multiple Services

Uses an Ingress Controller

Supports host and path routing

More cost-effective for many apps

Common Problems

Problem: Ingress not working

Possible reasons:

- No Ingress Controller installed
 - Wrong host name
 - Wrong Service name
 - Incorrect Ingress rules
-

Problem: 404 Not Found

Possible reasons:

- Wrong path
 - Wrong host
 - Rule doesn't match the request
-

Problem: 503 Service Unavailable

Possible reasons:

- Backend Pods are down
 - Service has no Endpoints
 - Selector doesn't match Pod labels
-

Problem: HTTPS not working

Possible reasons:

- Missing TLS certificate
 - Incorrect Secret
 - Wrong domain configuration
-

Important Commands

View Ingress

```
kubectl get ingress
```

Short form

```
kubectl get ing
```

Describe Ingress

```
kubectl describe ingress web-ingress
```

View Services

```
kubectl get svc
```

View Endpoints

```
kubectl get endpoints
```

View Ingress Controller Pods

```
kubectl get pods -n ingress-nginx
```

Interview Questions

Q1. What is Ingress?

Answer:

"Ingress is a Kubernetes resource that routes incoming HTTP and HTTPS requests to different Services based on host names or URL paths."

Q2. Does Ingress work by itself?

Answer:

"No. It requires an Ingress Controller to read the rules and route the traffic."

Q3. Why do we need Ingress if LoadBalancer already exists?

Answer:

"A LoadBalancer exposes one Service, while Ingress can route traffic to multiple Services using a single LoadBalancer. This reduces cost and simplifies management."

Q4. What is path-based routing?

Answer:

"It routes traffic based on the URL path. For example, `/api` goes to the API Service, and `/login` goes to the Login Service."

Q5. What is host-based routing?

Answer:

"It routes traffic based on the domain name. For example, `shop.example.com` goes to the shopping application, while `admin.example.com` goes to the admin application."

Q6. Name some Ingress Controllers.

Answer:

"NGINX Ingress Controller, AWS Load Balancer Controller, HAProxy, and Traefik."

Key Points

- Ingress manages HTTP/HTTPS traffic.
 - It requires an Ingress Controller.
 - Supports host-based and path-based routing.
 - Reduces the number of LoadBalancers needed.
 - Can terminate SSL/TLS.
 - Commonly used in production to expose multiple applications.
-

SRE Interview Tip

Interviewer: "Users are getting a 503 error when accessing *api.company.com*. How would you troubleshoot?"

Candidate Answer:

"First, I'd verify that the backend Pods are running using `kubectl get pods`. Then I'd check the Service and its Endpoints using `kubectl get svc` and `kubectl get endpoints`. Next, I'd review the Ingress rules with `kubectl describe ingress` to ensure the host and path are correct. Finally, I'd check the Ingress Controller logs for routing errors."

Commands to add to your notebook

```
kubectl get ingress
```

```
kubectl get ing
```

```
kubectl describe ingress <ingress-name>
```

```
kubectl get svc
```

kubectl get endpoints

kubectl get pods -n ingress-nginx



Quick Comparison

Type	Purpose
ClusterIP	Internal Pod-to-Pod communication
NodePort	External access using NodeIP:Port
LoadBalancer	Exposes one Service to the internet
Ingress	Routes HTTP/HTTPS traffic to multiple Services using one entry point



Akamai Interview Tip

One question that comes up frequently is:

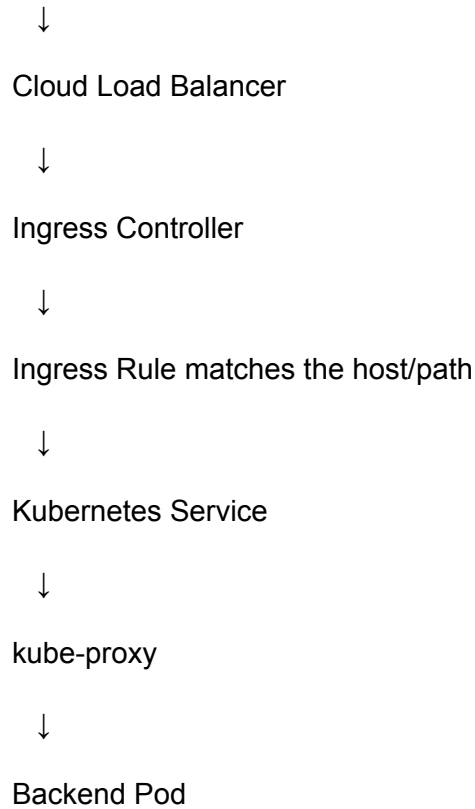
**"Explain the complete flow when a user opens
<https://shop.example.com>."**

A good answer is:

Browser



DNS resolves shop.example.com



If you can explain this flow clearly, you'll demonstrate that you understand how Kubernetes networking works end to end—not just the individual components.

Next Topic: Endpoints & EndpointSlice (Deep Dive)

This is often overlooked, but it's important for troubleshooting. It answers questions like:

- How does a Service know which Pods to send traffic to?
- What happens when a Pod becomes unhealthy?
- Why does `kubectl get endpoints` sometimes show `<none>`?

Phase 3 – Section 25: Endpoints & EndpointSlice (Deep Dive)

Objective

By the end of this topic, you should:

- Understand what Endpoints are.
 - Know why Kubernetes needs Endpoints.
 - Understand what EndpointSlice is.
 - Learn how Services find Pods.
 - Troubleshoot Service connectivity issues.
-

What are Endpoints?

An **Endpoint** is the list of **Pod IP addresses** behind a Kubernetes Service.

A Service **doesn't know Pod IPs by itself**.

Kubernetes automatically creates an Endpoint object that contains all the healthy Pods matching the Service selector.

Interview Answer

Q: What are Endpoints?

Answer:

"Endpoints are the list of healthy Pod IP addresses associated with a Service. Kubernetes automatically updates them whenever Pods are added, removed, or become unhealthy."

Why do we need Endpoints?

Suppose your Service looks like this:

Backend Service

But there are three backend Pods.

Backend Pod-1

Backend Pod-2

Backend Pod-3

How does the Service know where these Pods are?

Answer:

Through **Endpoints**.

Real-Life Example

Imagine ordering a taxi.

You call the taxi company.

The company checks which drivers are available.

Customer

↓

Taxi Company

↓

Driver 1

Driver 2

Driver 3

The taxi company maintains the list of available drivers.

Endpoints maintain the list of available Pods.

How Endpoints Work

Suppose:

Pods:

backend-1

IP = 10.244.1.10

Label = app=backend

backend-2

IP = 10.244.1.11

Label = app=backend

backend-3

IP = 10.244.1.12

Label = app=backend

Service:

selector:

app: backend

Kubernetes creates:

Endpoints

↓

10.244.1.10

10.244.1.11

10.244.1.12

Now kube-proxy sends traffic to one of these IPs.

Traffic Flow

Browser



Service



Endpoints



Pod IPs



Pod

Notice:

A Service forwards traffic to **Endpoints**, not directly to Pods.

What happens if a Pod crashes?

Initially:

backend-1

backend-2

backend-3

Endpoints:

10.244.1.10

10.244.1.11

10.244.1.12

Now backend-2 crashes.

Kubernetes automatically updates:

Endpoints

↓

10.244.1.10

10.244.1.12

Traffic is no longer sent to the failed Pod.

What happens when a new Pod starts?

Suppose a new Pod is created.

backend-4

IP = 10.244.1.20

Endpoints become:

10.244.1.10

10.244.1.12

10.244.1.20

Everything happens automatically.

What is EndpointSlice?

EndpointSlice is a newer and more scalable way to store endpoint information.

Instead of putting all Pod IPs into one large Endpoint object, Kubernetes splits them into multiple smaller EndpointSlices.

This improves performance in large clusters.

Why was EndpointSlice introduced?

Imagine you have:

10 Pods

One Endpoint object is fine.

Now imagine:

10,000 Pods

One huge Endpoint object becomes difficult to manage.

Instead:

EndpointSlice 1

100 Pod IPs

EndpointSlice 2

100 Pod IPs

EndpointSlice 3

100 Pod IPs

Kubernetes manages them more efficiently.

Endpoints vs EndpointSlice

Endpoints	EndpointSlice
Older resource	Newer resource
One large list	Multiple smaller lists
Less scalable	More scalable
Still supported	Recommended for large clusters

Production Example

Suppose your e-commerce application has **500 backend Pods**.

Instead of creating one huge Endpoint list,

Kubernetes creates several EndpointSlices.

This reduces API load and improves scalability.

Common Problems

Problem: Service has no Endpoints

Check:

```
kubectl get endpoints
```

Output:

```
backend-service  <none>
```

Possible reasons:

- Wrong selector
 - Pod labels don't match
 - Pods not Ready
-

Problem: Only one Pod receiving traffic

Possible reasons:

- Other Pods failing readiness checks
 - Endpoints contain only one healthy Pod
-

Problem: Service not routing traffic

Possible reasons:

- Empty Endpoints
 - Wrong labels
 - Application not Ready
-

Important Commands

View Endpoints

```
kubectl get endpoints
```

Describe Endpoints

```
kubectl describe endpoints backend-service
```

View EndpointSlices

```
kubectl get endpointslices
```

Describe EndpointSlice

```
kubectl describe endpointslice
```

View Services

```
kubectl get svc
```

View Pod Labels

```
kubectl get pods --show-labels
```

Interview Questions

Q1. What are Endpoints?

Answer:

"Endpoints are the list of healthy Pod IP addresses behind a Service."

Q2. Who creates Endpoints?

Answer:

"Kubernetes automatically creates and updates Endpoints based on the Service selector and Pod readiness."

Q3. Why do we need Endpoints?

Answer:

"A Service needs to know which Pods should receive traffic. Endpoints provide that list."

Q4. What happens when a Pod crashes?

Answer:

"Kubernetes removes that Pod from the Endpoints, so new traffic is sent only to healthy Pods."

Q5. What is EndpointSlice?

Answer:

"EndpointSlice is a newer resource that stores endpoint information in smaller chunks, making Kubernetes more scalable."

Q6. Why was EndpointSlice introduced?

Answer:

"To improve scalability and performance in clusters with a large number of Pods."

Troubleshooting Scenario

Interviewer: *"The frontend can't reach the backend Service. What will you check?"*

Candidate Answer:

"First, I'd check whether the backend Pods are running with `kubectl get pods`. Then I'd verify the Service using `kubectl get svc` and `kubectl describe svc`. Next, I'd check the Endpoints using `kubectl get endpoints`. If the Endpoints are empty, I'd compare the Service selector with the Pod labels using `kubectl get pods --show-labels`. Finally, I'd ensure the Pods are in the Ready state."

Key Points

- Endpoints contain the IP addresses of healthy Pods.
 - Services use Endpoints to forward traffic.
 - Kubernetes updates Endpoints automatically.
 - EndpointSlice is the scalable replacement for large clusters.
 - Empty Endpoints usually indicate a selector mismatch or Pods that are not Ready.
-

 **SRE Interview Tip**

A very common troubleshooting command is:

```
kubectl get endpoints
```

If it returns:

```
backend-service  <none>
```

don't immediately assume the Service is broken. First check:

1. Are the Pods running?
2. Do the Pod labels match the Service selector?
3. Are the Pods passing their readiness checks?

In many production incidents, the issue is simply that **the Service has no healthy endpoints to send traffic to**.

Commands to add to your notebook

```
kubectl get endpoints
```

```
kubectl describe endpoints <service-name>
```

```
kubectl get endpointslices
```

```
kubectl describe endpointslice <name>
```

```
kubectl get pods --show-labels
```

```
kubectl get svc
```

Phase 3 Progress

Completed:

-  ClusterIP
-  NodePort
-  LoadBalancer
-  Ingress
-  Endpoints & EndpointSlice

Next Topic: Network Policies (Deep Dive)

Although not every Kubernetes course emphasizes them, **NetworkPolicies are frequently discussed in SRE and production interviews** because they control which Pods are allowed to communicate with each other. Understanding them demonstrates knowledge of Kubernetes security and troubleshooting.

Phase 3 – Section 26: CoreDNS (Deep Dive)

This is one of the most frequently asked Kubernetes networking topics in Akamai interviews.

Objective

By the end of this topic, you should:

- Understand what CoreDNS is.
 - Know why Kubernetes needs DNS.
 - Understand how Pods communicate using names instead of IPs.
 - Troubleshoot DNS issues.
 - Answer CoreDNS interview questions confidently.
-

What is CoreDNS?

CoreDNS is the **DNS server inside a Kubernetes cluster**.

Its job is to **translate Service names into IP addresses**.

Instead of remembering IP addresses, applications simply use Service names.

Interview Answer

Q: What is CoreDNS?

Answer:

"CoreDNS is the DNS server in Kubernetes. It resolves Service names to ClusterIP addresses so Pods can communicate using names instead of IP addresses."

Why do we need CoreDNS?

Suppose your frontend wants to call the backend.

Without CoreDNS:

Frontend

↓

10.96.15.20

Tomorrow the Service is recreated.

New IP:

10.96.80.15

Now the frontend is still trying:

10.96.15.20

Application breaks.

Instead:

Frontend

↓

backend-service

CoreDNS always resolves it to the correct ClusterIP.

Real-Life Example

Imagine calling your friend.

You don't remember their phone number.

You save:

Rahul

Your phone contacts find the number.

CoreDNS is like your phone contacts.

Instead of remembering IPs,

applications remember Service names.

How CoreDNS Works

Suppose:

Backend Service

backend-service

ClusterIP

10.96.15.20

Frontend sends:

http://backend-service

CoreDNS replies:

backend-service

↓

10.96.15.20

Then traffic goes to the Service.

Complete Flow

Frontend Pod

↓

backend-service

↓

CoreDNS

↓

10.96.15.20

↓

ClusterIP Service

↓

Endpoints

↓

Backend Pod

This entire flow is commonly asked in interviews.

Where is CoreDNS Running?

CoreDNS itself runs as Pods.

Check:

```
kubectl get pods -n kube-system
```

Example:

```
coredns-5d78b6f8b-abc12
```

```
coredns-5d78b6f8b-def34
```

Usually there are two replicas for high availability.

Fully Qualified Domain Name (FQDN)

Every Service gets a DNS name.

Example:

```
backend-service.default.svc.cluster.local
```

Break it down:

```
backend-service
```

↓

default (namespace)

↓

svc

↓

cluster.local

Normally, Pods can simply use:

backend-service

if both are in the same namespace.

Different Namespace Example

Suppose:

Frontend

Namespace:

frontend

Backend

Namespace:

backend

The frontend cannot simply use:

backend-service

Instead it should use:

backend-service.backend.svc.cluster.local

Production Example

Your company has:

Payment Service

Order Service

Inventory Service

Notification Service

Instead of storing IP addresses,
all services communicate like this:

Order Service



payment-service



inventory-service



notification-service

CoreDNS resolves each name automatically.

Common Problems

Problem: DNS lookup failed

Example:

curl backend-service



Could not resolve host

Possible reasons:

- CoreDNS Pods are down
 - Wrong Service name
 - Wrong namespace
 - Network issue
-

Problem: One Service cannot reach another

Possible reasons:

- Service doesn't exist
 - Wrong namespace
 - CoreDNS failure
-

Problem: CoreDNS Pod CrashLoopBackOff

Possible reasons:

- Configuration error
 - Resource shortage
 - Corrupted CoreDNS ConfigMap
-

Important Commands

View CoreDNS Pods

```
kubectl get pods -n kube-system
```

View CoreDNS Service

```
kubectl get svc -n kube-system
```

View CoreDNS Logs

```
kubectl logs -n kube-system <coredns-pod>
```

Test DNS from a Pod

```
kubectl exec -it <pod-name> -- nslookup backend-service
```

or

```
kubectl exec -it <pod-name> -- dig backend-service
```

View CoreDNS ConfigMap

```
kubectl get configmap coredns -n kube-system
```

Interview Questions

Q1. What is CoreDNS?

Answer:

"CoreDNS is the DNS server inside Kubernetes. It resolves Service names into ClusterIP addresses."

Q2. Why do we need CoreDNS?

Answer:

"Service IPs can change, but Service names remain the same. CoreDNS allows applications to communicate using names instead of IP addresses."

Q3. Where does CoreDNS run?

Answer:

"CoreDNS runs as Pods in the `kube-system` namespace."

Q4. How do you troubleshoot DNS issues?

Answer:

"I'd verify the Service exists, check the CoreDNS Pods, test DNS resolution using `nslookup` or `dig` from another Pod, and review the CoreDNS logs for errors."

Q5. What is the default Kubernetes DNS suffix?

Answer:

`"cluster.local."`

Q6. Can Pods communicate using Service names?

Answer:

"Yes. CoreDNS resolves the Service name to the Service's ClusterIP."

Troubleshooting Scenario

Interviewer: *"The frontend Pod cannot resolve `backend-service`. What will you do?"*

Candidate Answer:

"First, I'd verify the Service exists using `kubectl get svc`. Then I'd check whether the CoreDNS Pods are running in the `kube-system` namespace. Next, I'd

test DNS resolution from the frontend Pod using `nslookup backend-service`. If it still fails, I'd check the CoreDNS logs and confirm both Services are in the correct namespace."

Key Points

- CoreDNS is Kubernetes' DNS server.
 - It resolves Service names to ClusterIP addresses.
 - Runs in the `kube-system` namespace.
 - Applications should use Service names instead of IP addresses.
 - If DNS fails, check CoreDNS Pods, Service names, namespaces, and logs.
-

★ SRE Interview Tip

One of the most common production issues is:

Application Error:

Could not resolve host

Don't immediately blame the network. Follow this order:

1. Is the Service created? (`kubectl get svc`)
2. Are CoreDNS Pods running? (`kubectl get pods -n kube-system`)
3. Can the Pod resolve the Service? (`nslookup` or `dig`)
4. Are the frontend and backend in the same namespace?
5. Check CoreDNS logs.

This systematic approach is what interviewers expect from an SRE.

Commands to add to your notebook

```
kubectl get pods -n kube-system
```

```
kubectl get svc -n kube-system
```

```
kubectl logs -n kube-system <coredns-pod>
```

```
kubectl exec -it <pod-name> -- nslookup <service-name>
```

```
kubectl exec -it <pod-name> -- dig <service-name>
```

```
kubectl get configmap coredns -n kube-system
```

✓ Phase 3 (Networking) Status

Completed:

- ✓ ClusterIP
- ✓ NodePort
- ✓ LoadBalancer
- ✓ Ingress
- ✓ Endpoints & EndpointSlice
- ✓ CoreDNS

Next Topic: kube-proxy (Deep Dive)

This is another **high-frequency interview topic**, where we'll cover:

- What kube-proxy actually does.
- How it works with Services and Endpoints.
- iptables vs IPVS modes.
- The complete packet flow from **Service** → **kube-proxy** → **Endpoint** → **Pod**.
- Common troubleshooting scenarios when Service routing fails.

Phase 3 – Section 27: kube-proxy (Deep Dive)

This is one of the most important Kubernetes networking topics for SRE interviews.

Objective

By the end of this topic, you should:

- Understand what kube-proxy is.
 - Know why Kubernetes needs kube-proxy.
 - Understand how Services send traffic to Pods.
 - Learn iptables and IPVS modes.
 - Troubleshoot Service routing issues.
-

What is kube-proxy?

kube-proxy is a **network component** that runs on **every worker node**.

Its job is to **route traffic from a Kubernetes Service to the correct Pod**.

A Service itself is just an object in Kubernetes. It cannot forward packets.

kube-proxy performs the actual traffic routing.

Interview Answer

Q: What is kube-proxy?

Answer:

"kube-proxy runs on every worker node and manages the networking rules that route traffic from a Service to the backend Pods."

Why do we need kube-proxy?

Suppose your Service has this IP:

backend-service

↓

10.96.0.20

A frontend Pod sends a request to:

10.96.0.20

But there is **no actual server** listening on that ClusterIP.

So who forwards the request?

Answer:

kube-proxy

Real-Life Example

Imagine a receptionist in an office.

A customer asks:

"I want to meet the HR department."

The receptionist doesn't do HR work.

Instead, they send the customer to the correct employee.

Customer

↓

Receptionist

↓

HR Employee

Similarly,
Application

↓

Service

↓

kube-proxy

↓

Backend Pod

How kube-proxy Works

Suppose you have:

Frontend

↓

backend-service

↓

ClusterIP

10.96.0.20

Backend Pods:

Pod-1

10.244.1.10

Pod-2

10.244.1.11

Pod-3

10.244.1.12

Flow:

Frontend

↓

ClusterIP

↓

kube-proxy

↓

Pod-2

Next request:

Frontend



ClusterIP



kube-proxy



Pod-1

Traffic is distributed across healthy Pods.

Complete Traffic Flow

Browser



Ingress

↓

Service

↓

kube-proxy

↓

Endpoints

↓

Pod

Remember:

A Service defines **what** should receive traffic.

kube-proxy decides **where** to send it.

Where does kube-proxy run?

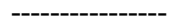
It runs as a **DaemonSet**.

That means:

Worker Node 1



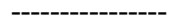
kube-proxy



Worker Node 2



kube-proxy



Worker Node 3



kube-proxy

Every node has its own kube-proxy Pod.

Check it using:

```
kubectl get pods -n kube-system
```

You'll see:

```
kube-proxy-abc123
```

```
kube-proxy-def456
```

```
kube-proxy-ghi789
```

How does kube-proxy know where Pods are?

The API Server continuously informs kube-proxy about:

- Services
- Endpoints
- EndpointSlices

Whenever a Pod is added or removed,

kube-proxy updates its routing rules automatically.

iptables Mode

This is the most common mode.

kube-proxy creates **iptables rules** inside Linux.

Flow:

Packet



iptables Rule



Backend Pod

Advantages:

- Fast
 - Simple
 - Default mode in many clusters
-

IPVS Mode

IPVS is another networking mode.

Instead of iptables,

it uses the Linux IPVS subsystem.

Advantages:

- Better performance
 - Better for very large clusters
 - Handles thousands of Services efficiently
-

iptables vs IPVS

iptables	IPVS
Default in many clusters	Used for high-scale clusters
Simpler	Faster
Good for small/medium clusters	Better for large clusters

Production Example

Suppose your backend Deployment has:

5 Pods

Frontend sends:

`http://backend-service`

kube-proxy might distribute requests like this:

Request 1 → Pod-1

Request 2 → Pod-4

Request 3 → Pod-2

Request 4 → Pod-5

This helps spread the workload across all healthy Pods.

Common Problems

Problem: Service not reachable

Possible reasons:

- kube-proxy not running
 - No Endpoints
 - Service selector mismatch
 - Network issue
-

Problem: Traffic goes to only one Pod

Possible reasons:

- Only one healthy Endpoint
 - Other Pods not Ready
-

Problem: Requests timing out

Possible reasons:

- kube-proxy issue
 - Node networking problem
 - Firewall rules
 - Backend Pods unavailable
-

Important Commands

Check kube-proxy Pods

```
kubectl get pods -n kube-system
```

View kube-proxy Logs

```
kubectl logs -n kube-system <kube-proxy-pod>
```

Describe kube-proxy Pod

```
kubectl describe pod <kube-proxy-pod> -n kube-system
```

View Services

```
kubectl get svc
```

View Endpoints

```
kubectl get endpoints
```

Check iptables (Linux)

```
sudo iptables -L
```

Interview Questions

Q1. What is kube-proxy?

Answer:

"kube-proxy is a networking component that runs on every worker node and routes traffic from Kubernetes Services to backend Pods."

Q2. Does kube-proxy run on the control plane?

Answer:

"Its primary role is to run on worker nodes because that's where Service traffic is forwarded. In a single-node or control-plane node that is also schedulable, kube-proxy can run there as well."

Q3. How does kube-proxy know where to send traffic?

Answer:

"It watches the API Server for changes to Services and Endpoints, then updates the node's networking rules."

Q4. What are the modes of kube-proxy?

Answer:

"The two common modes are iptables and IPVS."

Q5. What happens if a Pod crashes?

Answer:

"The Endpoint list is updated, and kube-proxy stops sending new traffic to that Pod."

Q6. Is kube-proxy a Load Balancer?

Answer:

"No. It doesn't create cloud load balancers. It performs Service-level traffic forwarding and simple load distribution between backend Pods."

Troubleshooting Scenario

Interviewer: *"Your Service exists, the Pods are running, but users still can't access the application. What will you check?"*

Candidate Answer:

"First, I'd verify the Service and Endpoints using `kubectl get svc` and `kubectl get endpoints`. Then I'd check that the Pods are Ready. After that, I'd verify the kube-proxy Pods are running in the `kube-system` namespace and review their logs for networking errors. If needed, I'd also check the node's networking rules and firewall."

Key Points

- kube-proxy runs on every worker node.
 - It routes Service traffic to backend Pods.
 - It watches the API Server for Service and Endpoint changes.
 - Common modes are **iptables** and **IPVS**.
 - Without kube-proxy, ClusterIP and NodePort Services cannot route traffic correctly.
-

SRE Interview Tip

A common interview question is:

"Who actually forwards traffic from a Service to a Pod?"

A strong answer is:

"The Service defines the virtual IP and selects the backend Pods, but kube-proxy programs the node's networking rules using iptables or IPVS to forward the traffic to one of the healthy Pods."

This answer shows you understand the difference between **Kubernetes objects (Service)** and the **component that implements the networking (kube-proxy)**.

Commands to add to your notebook

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system <kube-proxy-pod>
```

```
kubectl describe pod <kube-proxy-pod> -n kube-system
```

```
kubectl get svc
```

```
kubectl get endpoints
```

```
sudo iptables -L
```

Phase 3 (Networking) Completed

You now have a solid understanding of Kubernetes networking:

-  ClusterIP
-  NodePort
-  LoadBalancer
-  Ingress
-  Endpoints & EndpointSlice
-  CoreDNS
-  kube-proxy

This covers the networking topics that are most commonly discussed in Kubernetes and SRE interviews.

You're right. We covered the **concepts**, but we **didn't complete the troubleshooting phase** that I originally promised. For an SRE interview (especially Akamai), this is the most important part.

So **Phase 3 is NOT complete yet**.

The remaining sections are:

Phase 3 – Networking

Concepts

-  ClusterIP
-  NodePort
-  LoadBalancer
-  Ingress
-  CoreDNS
-  kube-proxy
-  Endpoints
-  EndpointSlice

Production Troubleshooting (Remaining)

Now we'll do real production scenarios.

1.  Service not reachable
2. DNS not resolving
3. Connection timeout
4. 503 Service Unavailable
5. Wrong selectors
6. Missing endpoints

This is where you'll think like an SRE instead of just explaining Kubernetes concepts.

For every scenario we'll follow the same format:

- Problem
- Production scenario
- How to think
- Commands
- Root cause
- Fix
- Interview answer

Scenario 1 – Service Not Reachable (Deep Dive)

Production Scenario

Suppose your company has:

Frontend Pod

↓

backend-service

↓

Backend Pods

The frontend team reports:

"Our application cannot reach the backend."

Step 1 – Don't Panic

A good SRE never guesses.

Always troubleshoot layer by layer.

Think like this:

Application



Service



Endpoints



Pods



Container



Node



Network

Step 2 – Are the Pods Running?

First command:

```
kubectl get pods
```

Example:

```
backend-7c8d9 Running
```

```
backend-5d4e8 Running
```

If Pods are not Running,
there is no point checking the Service.

Step 3 – Check the Service

```
kubectl get svc
```

Example:

```
backend-service
```

ClusterIP

10.96.20.15

Verify:

- Service exists
 - Correct port
 - Correct ClusterIP
-

Step 4 – Describe the Service

```
kubectl describe svc backend-service
```

Check:

Selector

app=backend

Now compare with Pod labels.

Step 5 – Check Pod Labels

```
kubectl get pods --show-labels
```

Example:

backend-1

app=mybackend

Service expects:

app=backend

Pods have:

app=mybackend

Selector mismatch.

This is one of the most common production issues.

Step 6 – Check Endpoints

kubectl get endpoints

Output:

backend-service

<none>

Immediately tells us:

The Service has no healthy backend Pods.

Step 7 – Test DNS

From frontend Pod:


```
kubectl exec -it frontend-pod -- nslookup backend-service
```

If DNS works,

move on.

If not,

check CoreDNS.

Step 8 – Test Connectivity

```
kubectl exec -it frontend-pod -- curl http://backend-service
```

Possible outputs:

200 OK

Good.

or

Connection refused

Application not listening.

or

Connection timed out

Network issue.

Troubleshooting Flow

Application



Pods Running?



Service Exists?



Selector Correct?



Endpoints Available?



DNS Resolving?



Application Listening?



Network OK?

Never skip steps.

Common Root Causes

Problem	Root Cause
Service not found	Wrong Service name
<none> endpoints	Wrong selector
Connection refused	Wrong targetPort or app not listening
Timeout	Firewall, NetworkPolicy, or node networking issue
DNS failed	CoreDNS issue
Pod Pending	Scheduler or resource issue

Important Commands

`kubectl get pods`

`kubectl get svc`

`kubectl describe svc backend-service`

`kubectl get endpoints`

`kubectl get pods --show-labels`

`kubectl exec -it frontend-pod -- curl http://backend-service`

`kubectl exec -it frontend-pod -- nslookup backend-service`

`kubectl logs <pod-name>`

Interview Question

Interviewer:

"Frontend cannot access the backend Service. How will you troubleshoot?"

Candidate Answer:

"First, I'd verify the backend Pods are running using `kubectl get pods`. Then I'd check the Service with `kubectl get svc` and `kubectl describe svc` to verify the selector. Next, I'd check the Endpoints using `kubectl get endpoints` to confirm the Service has healthy backend Pods. After that, I'd test DNS resolution using `nslookup` and application connectivity using `curl` from the frontend Pod. If needed, I'd review the application logs and node networking."

★ **SRE Interview Tip**

Interviewers are not looking for someone who immediately says, *"It's a DNS issue"* or *"It's a Service issue."*

They're looking for someone who follows a **structured troubleshooting process**. If you consistently explain your steps as:

Pods → Service → Endpoints → DNS → Application → Network

you'll demonstrate the systematic thinking expected of an SRE.



Next Scenario: DNS Not Resolving

We'll troubleshoot issues like:

- Could not resolve host
- CoreDNS Pod failures
- Wrong namespace
- Incorrect Service name
- DNS debugging using `nslookup` and `dig`

This is another very common Akamai interview scenario.

Phase 3 – Troubleshooting Scenario 2: DNS Not Resolving (Deep Dive)

This is one of the most common Kubernetes production issues and a favorite SRE interview question.

Production Scenario

Your frontend application is trying to call the backend.

Frontend Pod

|



`http://backend-service`

Instead of getting a response, the application logs show:

`curl: (6) Could not resolve host: backend-service`

Or

`java.net.UnknownHostException: backend-service`

This means **DNS resolution failed**.

How an SRE Thinks

Don't immediately blame CoreDNS.

Follow this order:

Application



Service



Namespace

↓

CoreDNS

↓

Network

Step 1 – Verify the Service Exists

`kubectl get svc`

Example:

NAME	TYPE	CLUSTER-IP
backend-service	ClusterIP	10.96.0.20

If the Service doesn't exist:

Error:

Service not found

Root Cause:

- Wrong Service name
- Service deleted

Fix:

Create the Service or use the correct name.

Step 2 – Verify the Namespace

Suppose:

Frontend:

Namespace:

frontend

Backend:

Namespace:

backend

Using

backend-service

will fail.

Instead use:

backend-service.backend.svc.cluster.local

or

backend-service.backend

This is a very common interview question.

Step 3 – Test DNS from the Pod

```
kubectl exec -it frontend-pod -- nslookup backend-service
```

Successful output:

Name:

backend-service

Address:

10.96.0.20

Failed output:

server can't find backend-service

DNS is failing.

Step 4 – Test Using dig

```
kubectl exec -it frontend-pod -- dig backend-service
```

This provides detailed DNS information.

Step 5 – Check CoreDNS Pods

```
kubectl get pods -n kube-system
```

Healthy:

coredns-xxxxx Running

coredns-yyyyy Running

Problem:

CrashLoopBackOff

Pending

Error

CoreDNS itself has failed.

Step 6 – Check CoreDNS Logs

```
kubectl logs -n kube-system <coredns-pod>
```

Look for:

- Startup errors
 - Configuration errors
 - Plugin failures
 - Connection issues with the API Server
-

Step 7 – Check the CoreDNS Service

```
kubectl get svc -n kube-system
```

Example:

```
kube-dns
```

Verify it exists and has a ClusterIP.

Step 8 – Verify Pod DNS Configuration

Inside the Pod:

```
cat /etc/resolv.conf
```

Typical output:

```
nameserver 10.96.0.10
```

search default.svc.cluster.local svc.cluster.local cluster.local

If the nameserver or search domains are incorrect, DNS lookups may fail.

DNS Resolution Flow

Application

↓

backend-service

↓

CoreDNS

↓

ClusterIP

↓

Service

↓

Endpoints

↓

Backend Pod

If DNS fails, the request never reaches the Service.

Common Root Causes

Error	Root Cause
Could not resolve host	Wrong Service name
UnknownHostException	DNS resolution failed
nslookup fails	CoreDNS issue
Service not found	Wrong namespace
CoreDNS CrashLoopBackOff	CoreDNS configuration problem

Empty DNS response

Service missing

Production Example

Suppose:

Frontend:

shopping

Backend:

payment-service

Developer accidentally writes:

payments-service

Notice the extra **s**.

CoreDNS searches:

payments-service

No Service exists.

Result:

Could not resolve host

The issue isn't CoreDNS—it's a typo in the Service name.

Important Commands

```
kubectl get svc
```

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system <coredns-pod>
```

```
kubectl exec -it <pod> -- nslookup backend-service
```

```
kubectl exec -it <pod> -- dig backend-service
```

```
kubectl exec -it <pod> -- cat /etc/resolv.conf
```

```
kubectl get svc -n kube-system
```

Interview Questions

Q1. DNS is not resolving inside a Pod. What will you check first?

Answer:

"First, I'd verify that the Service exists using `kubectl get svc`. Then I'd test DNS resolution from the Pod using `nslookup`. If it fails, I'd check the CoreDNS Pods, review their logs, and confirm the frontend and backend are using the correct namespace."

Q2. Where does Kubernetes store DNS information?

Answer:

"CoreDNS watches the Kubernetes API Server for Services and automatically creates DNS records for them."

Q3. How do you test DNS inside a Pod?

Answer:

"Using `nslookup` or `dig` from inside the Pod."

Q4. How do Pods know which DNS server to use?

Answer:

"Each Pod gets a `/etc/resolv.conf` file that points to the CoreDNS Service (`kube-dns`) in the cluster."

Q5. Why can a Service resolve in one namespace but not another?

Answer:

"Because short Service names only work within the same namespace. Across namespaces, you must use the full service name like `backend-service.backend.svc.cluster.local` or at least `backend-service.backend`."

Troubleshooting Scenario

Interviewer:

"The application shows `Could not resolve host: backend-service`. What will you do?"

Candidate Answer:

"I'd first check if the Service exists using `kubectl get svc`. Then I'd verify the frontend and backend namespaces. Next, I'd run `nslookup backend-service` from the frontend Pod. If DNS still fails, I'd check the CoreDNS Pods in the `kube-system` namespace, review their logs, and verify the Pod's `/etc/resolv.conf` is pointing to the cluster DNS service."

Key Points

- DNS resolution is handled by **CoreDNS**.
 - Always verify the Service exists before blaming DNS.
 - Check namespaces when using short Service names.
 - Use **nslookup** or **dig** to test DNS from inside a Pod.
 - Review CoreDNS Pod status and logs if DNS resolution fails.
 - Verify **/etc/resolv.conf** inside the Pod.
-

SRE Interview Tip

Many candidates jump straight to:

"CoreDNS is down."

Experienced SREs don't assume. They verify in this order:

1.  Service exists
2.  Correct namespace
3.  **nslookup** from the Pod
4.  CoreDNS Pods are healthy
5.  CoreDNS logs
6.  **/etc/resolv.conf**

This structured approach is exactly what interviewers want to see.

Commands to add to your notebook

```
kubectl get svc
```

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system <coredns-pod>
```

```
kubectl exec -it <pod> -- nslookup <service-name>
```

```
kubectl exec -it <pod> -- dig <service-name>
```

```
kubectl exec -it <pod> -- cat /etc/resolv.conf
```

```
kubectl get svc -n kube-system
```

✓ Remaining Troubleshooting Scenarios

- ✓ Service not reachable
- ✓ DNS not resolving
- → SOON Connection timeout
- → SOON 503 Service Unavailable
- → SOON Wrong selectors
- → SOON Missing endpoints

We'll continue with **Connection Timeout**, where we'll distinguish between:

- **Connection refused**
- **Connection timed out**
- **No route to host**

These three errors indicate different problems and are commonly used in Kubernetes troubleshooting interviews.

Phase 3 – Troubleshooting Scenario 3: Connection Timeout (Deep Dive)

This is a very common production issue. Many interviewers ask the difference between "Connection Refused" and "Connection Timed Out".

Production Scenario

Your frontend application calls:

`http://backend-service`

Instead of getting a response, it hangs for a few seconds and shows:

curl: (28) Connection timed out

or

dial tcp 10.96.0.20:80: i/o timeout

Connection Timeout vs Connection Refused

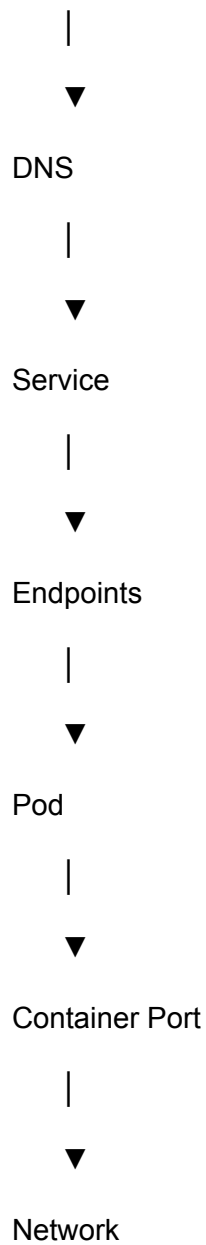
Many candidates confuse these.

Error	Meaning
Connection Refused	The destination is reachable, but nothing is listening on that port.
Connection Timed Out	The request never reached the destination or no response came back.
No Route to Host	The network path itself is unavailable.

How an SRE Thinks

Always troubleshoot layer by layer.

Application



Step 1 – Is DNS Working?

`kubectl exec -it frontend-pod -- nslookup backend-service`

If DNS fails,
don't continue.
Fix DNS first.

Step 2 – Check the Service

```
kubecttl get svc
```

Verify:

- Service exists
- ClusterIP
- Port

Example

```
backend-service
```

ClusterIP

10.96.0.20

PORT 80

Step 3 – Check Endpoints

```
kubecttl get endpoints backend-service
```

Good output:

10.244.1.10:8080

10.244.1.11:8080

Bad output:

<none>

No endpoints means the Service has nowhere to send traffic.

Step 4 – Check the Pods

kubectl get pods

Verify:

STATUS

Running

Not:

Pending

CrashLoopBackOff

ImagePullBackOff

Step 5 – Check Pod Readiness

Even if a Pod is **Running**, it may not be **Ready**.

```
kubectl get pods
```

Example:

```
READY
```

```
0/1
```

That Pod will **not** be added to the Service Endpoints.

Step 6 – Verify targetPort

Example Service:

```
ports:
```

```
- port: 80
```

```
  targetPort: 8080
```

Container:

containerPort: 8080

Correct.

Wrong example:

Service

targetPort: 80

Container

containerPort: 8080

Traffic reaches the Pod,
but nothing is listening on port 80.

Step 7 – Check the Application

Enter the Pod.

```
kubectl exec -it backend-pod -- sh
```

Run:

```
netstat -tuln
```

or

```
ss -tuln
```


Expected:

0.0.0.0:8080

If nothing is listening,

the application didn't start correctly.

Step 8 – Check Logs

`kubectl logs backend-pod`

Maybe you'll find:

Database connection failed

Application startup failed

Port binding failed

Step 9 – Check NetworkPolicy

If NetworkPolicies are enabled,

traffic might be blocked.

Example:

Frontend

↓

 Blocked

↓

Backend

Check:

`kubectl get networkpolicy`

Step 10 – Test Connectivity

From frontend Pod:

`curl http://backend-service`

or

`wget backend-service`

If timeout continues,

try the Pod IP directly.

`curl http://10.244.1.10:8080`

If Pod IP works but Service doesn't,

focus on:

- Service
 - Endpoints
 - kube-proxy
-

Troubleshooting Flow

Connection Timeout



DNS Working?



Service Exists?



Endpoints Available?



Pods Running?



Pods Ready?



Correct targetPort?



Application Listening?



NetworkPolicy?



kube-proxy?



Common Root Causes

Problem	Root Cause
Timeout	Empty Endpoints
Timeout	Pod not Ready
Timeout	NetworkPolicy blocking traffic
Timeout	kube-proxy issue
Timeout	Wrong targetPort
Timeout	Application hung
Timeout	Node networking issue

Important Commands

`kubectl get svc`

`kubectl get endpoints`

`kubectl get pods`

`kubectl describe pod <pod>`

`kubectl logs <pod>`

```
kubectl exec -it <pod> -- ss -tuln
```

```
kubectl exec -it frontend-pod -- curl http://backend-service
```

```
kubectl get networkpolicy
```

Interview Questions

Q1. What does "Connection timed out" mean?

Answer:

"It means the request didn't receive a response within the expected time. The issue could be in networking, Service routing, Endpoints, NetworkPolicies, or the application."

Q2. How is it different from "Connection refused"?

Answer:

"Connection refused means the destination is reachable, but no application is listening on the requested port. Connection timed out means the request never got a response."

Q3. What will you check first?

Answer:

"I'd first verify DNS resolution, then check the Service, Endpoints, Pod status, targetPort, application logs, and finally any NetworkPolicies or kube-proxy issues."

Real Production Example

Suppose your Service is:

port: 80

targetPort: 8080

But the developer accidentally changes the application to listen on:

9090

Result:

Frontend

|



Service (80)

|



Pod (expects 8080)

|



Nothing listening

The request fails because the Service is forwarding traffic to the wrong port.

Troubleshooting Scenario

Interviewer:

"Users report that requests to the backend are timing out. How would you troubleshoot?"

Candidate Answer:

"First, I'd confirm DNS resolution using `nslookup`. Then I'd verify the Service and its Endpoints. Next, I'd check that the backend Pods are Running and Ready. I'd confirm the Service `targetPort` matches the application's listening port, review the application logs, and test connectivity from another Pod using `curl`. If everything looks correct, I'd investigate NetworkPolicies and kube-proxy."

Key Points

- **Connection timed out** usually indicates a networking or routing problem.
 - Always check **Endpoints** before suspecting the application.
 - Verify **Pod readiness** because only Ready Pods receive traffic.
 - Ensure the Service `targetPort` matches the application's listening port.
 - Check NetworkPolicies if traffic should be allowed but isn't.
-

SRE Interview Tip

A senior interviewer may ask:

"The Service exists and the Pods are Running, but requests still time out. What's next?"

A strong answer is:

"Running doesn't always mean Ready. I'd check the Pod's readiness status and verify that the Service has healthy Endpoints using `kubectl get endpoints`. If the Endpoints are present, I'd confirm the application is listening on the expected port and then investigate NetworkPolicies or kube-proxy."

This answer demonstrates that you understand the difference between **Pod state**, **readiness**, and **Service routing**—a key skill for Kubernetes troubleshooting.

✓ Troubleshooting Progress

- ✓ Service not reachable
- ✓ DNS not resolving
- ✓ Connection timeout
- → SOON 503 Service Unavailable
- → SOON Wrong selectors
- → SOON Missing endpoints

The **503 Service Unavailable** scenario is one of the most frequently asked Kubernetes troubleshooting questions because it ties together **Ingress, Services, Endpoints, kube-proxy, and Pod health**.

Q: Difference between **Connection Refused** and **Connection Timed Out**?

Answer:

"Connection Refused means the server is reachable, but no application is listening on that port. Connection Timed Out means the request didn't get a response, usually due to a network issue, routing problem, or firewall."

Troubleshooting Scenario 4: 503 Service Unavailable (Interview Version)

Interview Question

Q: Users are getting a 503 Service Unavailable error. How will you troubleshoot?

Candidate Answer (45 seconds)

"A 503 usually means the request reached the Service or Ingress, but there are no healthy backend Pods to serve the request. First, I'll check if the Pods are Running and Ready. Then I'll verify the Service Endpoints using `kubectl get endpoints`. If the Endpoints are empty, I'll check the Service selector and Pod labels. Finally, I'll review the application logs and, if using Ingress, verify that it's routing to the correct Service."

Commands to Remember

`kubectl get pods`

kubectl get svc

kubectl get endpoints

kubectl describe svc <service-name>

kubectl logs <pod-name>

kubectl describe ingress <ingress-name>

Common Root Causes

- No healthy Pods
- Pods are **Not Ready**
- Empty Endpoints
- Wrong Service selector
- Wrong Ingress backend Service
- Application crashed

Interview Follow-up

Q: Why does Kubernetes return a 503?

Answer:

"Because the Service or Ingress has no healthy backend Pods available to handle the request."

Quick Troubleshooting Flow

503 Error

↓

Pods Running?

↓

Pods Ready?



Endpoints Available?



Service Selector Correct?



Ingress Configuration Correct?



Application Logs

★ Akamai Interview Tip

If the interviewer asks:

"What is the first command you'll run?"

A good answer is:

`kubectl get endpoints`

If it shows:

`backend-service <none>`

you've likely found the problem immediately.

✅ Remaining Topics

- ✅ Service not reachable

-  DNS not resolving
-  Connection timeout
-  503 Service Unavailable
-  Wrong selectors
-  Missing endpoints

Troubleshooting Scenario 5: Wrong Selectors (Interview Version)

Interview Question

Q: The Service exists, Pods are running, but the application is still not accessible. What could be the issue?

Candidate Answer (45 seconds)

"One common reason is a selector mismatch. A Service sends traffic only to Pods whose labels match its selector. I'll check the Service selector using `kubectl describe svc` and compare it with the Pod labels using `kubectl get pods --show-labels`."

Example

Service Selector

selector:

app: backend

Pod Label

labels:

app: my-backend

 These don't match.

Result:

Service



No matching Pods



Endpoints = <none>



Application not reachable

Commands to Remember

`kubectl describe svc <service-name>`

`kubectl get pods --show-labels`

`kubectl get endpoints`

Common Root Causes

- Wrong label on Pods
- Wrong selector in Service
- Typo in label name or value
- Labels changed after deployment

Interview Follow-up

Q: How do you confirm a selector mismatch?

Answer:

"I compare the Service selector with the Pod labels. If they don't match, Kubernetes won't create Endpoints for that Service."

★ Akamai Interview Tip

The interviewer may ask:

"Pods are Running, but `kubectl get endpoints` shows `<none>`. What do you suspect?"

Answer:

"The first thing I'd suspect is a selector mismatch between the Service and the Pods."

✓ Remaining Topic

- ✓ Service not reachable
- ✓ DNS not resolving
- ✓ Connection timeout
- ✓ 503 Service Unavailable
- ✓ Wrong selectors
- ➡️ Missing endpoints

Troubleshooting Scenario 6: Missing Endpoints (Interview Version)

Interview Question

Q: `kubectl get endpoints` shows `<none>`. How will you troubleshoot?

Candidate Answer (45 seconds)

"If the Endpoints are empty, it means the Service has no healthy Pods to send traffic to. First, I'll check whether the Pods are running. Then I'll compare the

Service selector with the Pod labels. After that, I'll verify that the Pods are in the Ready state because only Ready Pods are added to the Endpoints."

Example

Service:

selector:

app: backend

Pods:

labels:

app: backend

Output:

kubectl get endpoints

backend-service <none>

Possible reasons:

- Pods are **Not Ready**
 - Selector mismatch
 - Pods crashed
 - No Pods exist
-

Commands to Remember

kubectl get endpoints

kubectl get pods

kubectl get pods --show-labels

kubectl describe svc <service-name>

kubectl describe pod <pod-name>

Common Root Causes

- No Pods running
- Pods not Ready
- Wrong Service selector
- Pod labels don't match
- Pods in a different namespace

Perfect. From now on, **interview-only format**.

Phase 4 – Storage

Topic 1: Volumes (Interview Version)

Interview Question

Q: Why do we need Volumes in Kubernetes?

Candidate Answer (45 seconds)

"Containers have ephemeral storage, so if a container restarts, the data inside it is lost. Kubernetes Volumes provide storage that survives container restarts within the same Pod. They allow containers in a Pod to share data and are the foundation for persistent storage using PVs and PVCs."

Real Example

Without Volume:

Container

↓

Writes logs

↓

Container Restarts



Logs Lost ❌

With Volume:

Container



Volume



Container Restarts



Data Still Available ✅

Important Point

A **Volume** survives container restarts.

A **Volume** does **NOT** survive Pod deletion.

For data that survives Pod deletion, use **Persistent Volumes (PV)**.

Types You Should Know

- **emptyDir** → Temporary storage (deleted when Pod is deleted)
- **hostPath** → Uses a directory on the worker node (mostly for testing)
- **PersistentVolume** → Long-term storage (covered next)

No need to memorize every volume type for interviews.

Commands

kubectl describe pod <pod-name>

kubectl get pv

kubectl get pvc

Interview Follow-up

Q: Does a Volume survive a container restart?

Answer:

"Yes."

Q: Does a Volume survive Pod deletion?

Answer:

"No. Regular Pod volumes are deleted with the Pod. For persistent storage, we use Persistent Volumes."

★ Akamai Interview Tip

If the interviewer asks:

"Why can't we store database data inside a container?"

Answer:

"Because container storage is ephemeral. If the container or Pod is recreated, the data can be lost. Databases should use Persistent Volumes."

Commands to Remember

```
kubectl describe pod <pod-name>
```

```
kubectl get pv
```

```
kubectl get pvc
```



Next Topic

Persistent Volume (PV)

You'll learn:

- What a PV is
- How it's different from a Volume
- Why Kubernetes separates storage from Pods
- Most common interview questions on PVs

Phase 4 – Storage

Topic 2: Persistent Volume (PV) – Interview Version

Interview Question

Q: What is a Persistent Volume (PV)?

Candidate Answer (45 seconds)

"A Persistent Volume is a storage resource in Kubernetes that exists independently of Pods. Unlike normal Pod volumes, a PV is not deleted when a Pod is deleted. It is used to store persistent data like databases, application files, and logs."

Real Example

Without PV:

Pod

↓

Database

↓

Pod Deleted

↓

Data Lost ❌

With PV:

Pod

↓

Persistent Volume

↓

Pod Deleted

↓

Data Still Exists ✅

Important Point

- **Volume** → Lives only as long as the Pod.
- **Persistent Volume (PV)** → Lives independently of the Pod.

Where does a PV come from?

A PV can be backed by:

- AWS EBS

- Azure Disk
- NFS
- Local Storage
- GCP Persistent Disk

In AWS, a PV is commonly backed by an **EBS volume**.

Commands

```
kubectl get pv
```

```
kubectl describe pv <pv-name>
```

Interview Follow-up

Q: Who creates a Persistent Volume?

Answer:

"It can be created manually by an administrator or dynamically using a StorageClass."

Q: Can multiple Pods use the same PV?

Answer:

"It depends on the access mode. For example, **ReadWriteOnce** allows one node to mount it, while **ReadWriteMany** allows multiple Pods if the storage backend supports it."

(We'll cover access modes in the PVC topic.)

 **Akamai Interview Tip**

If the interviewer asks:

"What happens to the PV if the Pod is deleted?"

Answer:

"The Pod is deleted, but the Persistent Volume remains. A new Pod can reuse the same storage through a Persistent Volume Claim."

Commands to Remember

```
kubectl get pv
```

```
kubectl describe pv <pv-name>
```



Next Topic

Persistent Volume Claim (PVC)

This is one of the most frequently asked storage interview questions because it explains **how Pods actually use a Persistent Volume**.

Phase 4 – Storage

Topic 3: Persistent Volume Claim (PVC) – Interview Version

Interview Question

Q: What is a Persistent Volume Claim (PVC)?

Candidate Answer (45 seconds)

"A Persistent Volume Claim is a request for storage made by a Pod. The Pod doesn't directly use a Persistent Volume. Instead, it requests storage through a PVC, and Kubernetes automatically binds the PVC to a suitable PV."

Simple Flow

Pod



PVC (Request)



PV (Storage)

Think of it like this:

- **PV = Storage**
 - **PVC = Request for storage**
 - **Pod = Uses the request**
-

Why use PVC instead of directly using a PV?

"PVC separates the application from the actual storage. If the storage backend changes, the application doesn't need any changes."

Commands

kubectl get pvc

kubectl describe pvc <pvc-name>

Common Access Modes

- **ReadWriteOnce (RWO)** → Mounted by one node (most common, AWS EBS)
- **ReadOnlyMany (ROX)** → Multiple Pods can read
- **ReadWriteMany (RWX)** → Multiple Pods can read and write (NFS, EFS)

You don't need to memorize more than these three.

Interview Follow-up

Q: Can a Pod use a PV directly?

Answer:

"No. A Pod uses a PVC, and the PVC is bound to a PV."

Q: What if no matching PV is available?

Answer:

"The PVC stays in the Pending state until a matching PV is available or one is created dynamically."

★ Akamai Interview Tip

If the interviewer asks:

"Explain PV and PVC in one sentence."

Answer:

"A PV is the actual storage, and a PVC is the request that allows a Pod to use that storage."

Commands to Remember


```
kubectl get pvc
```

```
kubectl describe pvc
```

```
kubectl get pv
```



Next Topic

StorageClass

You'll learn:

- Why StorageClass is needed
- Dynamic provisioning
- How PVs are created automatically
- Most common interview questions on StorageClass

Phase 4 – Storage

Topic 4: StorageClass (Interview Version)

Interview Question

Q: What is a StorageClass?

Candidate Answer (45 seconds)

"A StorageClass defines how storage should be created. Instead of an administrator creating a Persistent Volume manually, Kubernetes can automatically create the required storage when a PVC is created. This is called dynamic provisioning."

Simple Flow

Pod

↓

PVC

↓

StorageClass

↓

Automatically Creates PV

↓

Pod Uses Storage

Why do we need a StorageClass?

Without StorageClass:

Admin creates PV manually

↓

PVC binds to PV

With StorageClass:

PVC created

↓

StorageClass automatically creates PV 

Real Example (AWS)

"In AWS EKS, when I create a PVC, the StorageClass automatically provisions an EBS volume and binds it to the PVC."

Commands

kubectl get storageclass

kubectl describe storageclass

kubectl get pvc

Interview Follow-up

Q: What is Dynamic Provisioning?

Answer:

"Dynamic provisioning means Kubernetes automatically creates a Persistent Volume when a PVC requests storage."

Q: What happens if no StorageClass exists?

Answer:

"The PVC remains in the Pending state unless there's already a matching Persistent Volume available."

★ Akamai Interview Tip

If the interviewer asks:

"What is the relationship between PV, PVC, and StorageClass?"

Answer:

"The Pod requests storage using a PVC. The StorageClass automatically creates a PV if needed, and then the PVC is bound to that PV."

Commands to Remember

kubectl get storageclass

kubectl describe storageclass

kubectl get pvc

kubectl get pv

Storage Interview Cheat Sheet (Memorize This)

Component	One-line Answer
Volume	Temporary storage inside a Pod.
PV	Actual persistent storage resource.
PVC	Request for storage made by a Pod.
StorageClasses	Automatically creates PVs (dynamic provisioning).

30-Second Summary

"A Pod requests storage through a PVC. If a matching PV doesn't exist, the StorageClass dynamically creates one. The PVC binds to the PV, and the Pod uses that storage."

✓ Storage Topics Completed

- ✓ Volumes
- ✓ Persistent Volumes (PV)
- ✓ Persistent Volume Claims (PVC)
- ✓ StorageClass
- ✓ Dynamic Provisioning

These are the storage concepts most commonly asked in Kubernetes and SRE interviews.

Phase 5 – Configuration

Topic 1: ConfigMap (Interview Version)

Interview Question

Q: What is a ConfigMap?

Candidate Answer (45 seconds)

"A ConfigMap is used to store non-sensitive configuration data separately from the application. Instead of hardcoding values like database host, port, or environment name inside the container image, we store them in a ConfigMap. This makes configuration changes easier without rebuilding the image."

Real Example

Instead of hardcoding:

DB_HOST=10.0.1.20

APP_ENV=production

Store them in a **ConfigMap** and mount them into the Pod.

Why use ConfigMap?

- Keep configuration separate from application code.
 - No need to rebuild the Docker image when configuration changes.
 - Same image can be used for Dev, QA, and Production.
-

Commands

```
kubectl get configmap
```

```
kubectl describe configmap <name>
```

Interview Follow-up

Q: Can we store passwords in a ConfigMap?

Answer:

"No. ConfigMaps are for non-sensitive data. Passwords and API keys should be stored in Secrets."

★ Akamai Interview Tip

If asked:

"Give an example of data stored in a ConfigMap."

Answer:

"Database host, application port, log level, environment name, or feature flags."

Commands to Remember

kubectl get configmap

kubectl describe configmap



Next Topic

Secrets

You'll learn:

- What Secrets are
- ConfigMap vs Secret
- Common interview questions about storing passwords, API keys, and certificates.

Phase 5 – Configuration

Topic 2: Secrets (Interview Version)

Interview Question

Q: What are Kubernetes Secrets?

Candidate Answer (45 seconds)

"Secrets are used to store sensitive information like passwords, API keys, tokens, and certificates. Instead of storing them inside the application or ConfigMap, Kubernetes stores them as Secrets, which can be mounted into Pods or exposed as environment variables."

Real Example

Store:

- Database password
- AWS Access Key
- API Token
- TLS Certificate

as **Secrets**, not in the application code.

ConfigMap vs Secret

ConfigMap	Secret
Non-sensitive data	Sensitive data
DB Host, Port	Passwords, Tokens
Plain configuration	Credentials

Commands

kubectl get secrets

kubectl describe secret <secret-name>

Interview Follow-up

Q: Are Kubernetes Secrets encrypted?

Answer:

"By default, Secrets are Base64 encoded, not encrypted. For better security, we should enable Encryption at Rest in Kubernetes."

Q: How does a Pod use a Secret?

Answer:

"A Secret can be mounted as a file inside the Pod or injected as an environment variable."

Akamai Interview Tip

If asked:

"Can I store a database password in a ConfigMap?"

Answer:

"No. Passwords should always be stored in Kubernetes Secrets."

Commands to Remember

kubectl get secrets

kubectl describe secret

Configuration Interview Cheat Sheet

Component

One-line Answer

ConfigMap Stores non-sensitive configuration.

Secret Stores sensitive information like passwords and API keys.

30-Second Summary

"ConfigMaps store application configuration such as database host and log level. Secrets store sensitive data like passwords, API keys, and certificates. Pods can consume both as environment variables or mounted files."



Next Phase: Security

We'll cover the Kubernetes security topics most commonly asked in interviews:

1. Service Account
2. RBAC (Role & RoleBinding)
3. NetworkPolicy
4. Security Context
5. Taints & Tolerations
6. Node Affinity vs Pod Affinity

Phase 6 – Security

Topic 1: Service Account (Interview Version)

Interview Question

Q: What is a Service Account in Kubernetes?

Candidate Answer (45 seconds)

"A Service Account provides an identity for Pods to communicate with the Kubernetes API. If an application inside a Pod needs to access Kubernetes resources, it uses a Service Account instead of a user account."

Real Example

Suppose your application needs to:

- Read Pods
- Create Jobs
- List Services

It uses a **Service Account** to authenticate with the Kubernetes API.

Simple Flow

Pod

↓

Service Account

↓

Kubernetes API Server

Why do we need it?

- Gives Pods an identity
 - Securely accesses Kubernetes resources
 - Used with RBAC to control permissions
-

Commands

```
kubectl get serviceaccounts
```

```
kubectl describe serviceaccount <name>
```

Interview Follow-up

Q: Does every Pod have a Service Account?

Answer:

"Yes. By default, every Pod uses the `default` Service Account unless another one is specified."

Q: Does a Service Account define permissions?

Answer:

"No. Permissions are granted through RBAC using Roles and RoleBindings."

Akamai Interview Tip

If the interviewer asks:

"Can a Pod access the Kubernetes API?"

Answer:

"Yes. If it has a Service Account with the required RBAC permissions."

Commands to Remember

`kubectl get serviceaccounts`

`kubectl describe serviceaccount`



Next Topic

RBAC (Role & RoleBinding)

This is one of the **most frequently asked Kubernetes security interview questions**.

Phase 6 – Security

Topic 2: RBAC (Role & RoleBinding) – Interview Version

Interview Question

Q: What is RBAC in Kubernetes?

Candidate Answer (45 seconds)

"RBAC stands for Role-Based Access Control. It is used to control who can perform what actions on Kubernetes resources. We create a Role with specific permissions and bind it to a user or Service Account using a RoleBinding."

Simple Flow

Service Account



RoleBinding



Role



Permissions (get, list, create, delete)

Components

- **Role** → Defines permissions.
 - **RoleBinding** → Assigns those permissions.
 - **Service Account/User** → Uses those permissions.
-

Real Example

Suppose an application only needs to **view Pods**.

- Create a **Role** with **get** and **list** permissions.
- Bind it to the application's **Service Account**.

The application can now view Pods but **cannot delete them**.

Commands

kubectl get roles

kubectl get rolebindings

kubectl describe role <role-name>

Interview Follow-up

Q: What's the difference between Role and RoleBinding?

Answer:

"A Role defines permissions. A RoleBinding assigns those permissions to a user or Service Account."

Q: What's the difference between Role and ClusterRole?

Answer:

"A Role works within a single namespace. A ClusterRole works across the entire Kubernetes cluster."

★ Akamai Interview Tip

If the interviewer asks:

"Can a Service Account access the Kubernetes API without RBAC?"

Answer:

"It can authenticate using the Service Account, but without the required Role and RoleBinding, it won't have permission to perform actions."

Commands to Remember

kubectl get roles

kubectl get rolebindings

kubectl describe role

🎯 30-Second Summary

"RBAC controls access in Kubernetes. A Role defines permissions like get or create Pods, and a RoleBinding assigns those permissions to a user or Service Account."

Next Topic

NetworkPolicy

You'll learn:

- How to control Pod-to-Pod traffic
- Default allow vs default deny
- Common NetworkPolicy interview questions

Phase 6 – Security

Topic 3: NetworkPolicy (Interview Version)

Interview Question

Q: What is a NetworkPolicy?

Candidate Answer (45 seconds)

"A NetworkPolicy is used to control network traffic between Pods. By default, Pods can communicate with each other. A NetworkPolicy allows us to restrict which Pods can send or receive traffic."

Real Example

Suppose you have:

- Frontend Pod
- Backend Pod
- Database Pod

Requirement:

-  Frontend → Backend
-  Backend → Database
-  Frontend → Database

A **NetworkPolicy** enforces this communication.

Simple Flow

Frontend



Commands

`kubecttl get networkpolicy`

`kubecttl describe networkpolicy <policy-name>`

Interview Follow-up

Q: What happens if there is no NetworkPolicy?

Answer:

"By default, all Pods can communicate with each other."

Q: Can a NetworkPolicy allow and deny traffic?

Answer:

"Yes. It can control both Ingress (incoming) and Egress (outgoing) traffic."

Q: Does NetworkPolicy work by itself?

Answer:

"No. It requires a network plugin like Calico or Cilium that supports NetworkPolicy."

Akamai Interview Tip

If the interviewer asks:

"The Pods are Running, DNS is working, but the application still can't connect. What could be the reason?"

Answer:

"I'd check if a NetworkPolicy is blocking the traffic between the Pods."

Commands to Remember

kubectl get networkpolicy

kubectl describe networkpolicy

30-Second Summary

"NetworkPolicy is like a firewall for Pods. It controls which Pods can communicate with each other using ingress and egress rules."

Next Topic

Security Context

You'll learn:

- Why containers shouldn't run as root
- `runAsUser`
- `runAsNonRoot`
- `readOnlyRootFilesystem`
- Common security interview questions

Phase 6 – Security

Topic 5: Taints & Tolerations (Interview Version)

Interview Question

Q: What are Taints and Tolerations?

Candidate Answer (45 seconds)

"Taints are applied to nodes to prevent Pods from being scheduled on them. Tolerations are added to Pods to allow them to run on tainted nodes. In simple terms, taints repel Pods, and tolerations let specific Pods enter."

Simple Flow

Node (Tainted)

|

|— Normal Pod ❌ Not Scheduled

|

|— Pod with Tolerations ✅ Scheduled

Real Example

Suppose you have a node dedicated to databases.

- Add a **taint** to that node.
 - Only database Pods with the matching **toleration** can run there.
 - Other application Pods are kept away.
-

Commands

```
kubectl describe node <node-name>
```

```
kubectl taint nodes <node-name> key=value:NoSchedule
```

Interview Follow-up

Q: What is the purpose of a taint?

Answer:

"To prevent unwanted Pods from being scheduled on a node."

Q: Can a Pod run on a tainted node without a toleration?

Answer:

"No. It will not be scheduled."

Q: What is **NoSchedule?**

Answer:

"It's a taint effect that prevents new Pods from being scheduled on the node unless they have a matching toleration."

★ Akamai Interview Tip

A common interview question is:

"What's the difference between Node Selector and Taints & Tolerations?"

Answer:

"Node Selector attracts Pods to specific nodes. Taints repel Pods from specific nodes unless they have a matching toleration."

Commands to Remember

`kubectl describe node`

`kubectl taint nodes <node-name> key=value:NoSchedule`

🎯 30-Second Summary

"Taints keep Pods away from a node. Tolerations allow only specific Pods to run on that node. They're commonly used to reserve nodes for workloads like databases or GPU applications."



Next Topic

Node Affinity vs Pod Affinity (Another frequently asked Kubernetes interview question)

Phase 6 – Security

Topic 6: Node Affinity vs Pod Affinity (Interview Version)

Interview Question

Q: What is Node Affinity?

Candidate Answer (30 seconds)

"Node Affinity is used to schedule Pods on specific nodes based on node labels. For example, if I want my application to run only on SSD nodes or GPU nodes, I can use Node Affinity."

Interview Question

Q: What is Pod Affinity?

Candidate Answer (30 seconds)

"Pod Affinity is used to schedule a Pod close to another Pod. For example, I may want my frontend Pod to run on the same node as the backend Pod to reduce network latency."

Interview Question

Q: What is Pod Anti-Affinity?

Candidate Answer (30 seconds)

"Pod Anti-Affinity does the opposite. It ensures Pods don't run on the same node. This improves high availability. For example, replicas of the same application should be spread across different nodes."

Simple Comparison

Feature	Purpose
Node Affinity	Schedule Pods on specific nodes

Pod Affinity Schedule Pods near other Pods

Pod Anti-Affinity Keep Pods on different nodes

Real Examples

- **Node Affinity** → Run database Pods only on SSD nodes.
 - **Pod Affinity** → Keep frontend and backend together.
 - **Pod Anti-Affinity** → Spread 3 replicas across 3 different nodes.
-

Commands

`kubectl get nodes --show-labels`

`kubectl describe pod <pod-name>`

Interview Follow-up

Q: What's the difference between Node Selector and Node Affinity?

Answer:

"Node Selector is a simple label match. Node Affinity is more flexible because it supports advanced matching rules like preferred and required scheduling."

Q: Why use Pod Anti-Affinity?

Answer:

"To improve availability. If one node fails, all replicas won't be lost."

★ Akamai Interview Tip

If the interviewer asks:

"You have 3 replicas of an application. How do you make sure they don't all run on the same node?"

Answer:

"I'd use Pod Anti-Affinity so Kubernetes spreads the replicas across different nodes."

🎯 30-Second Summary

"Node Affinity decides which node a Pod should run on. Pod Affinity keeps Pods together, while Pod Anti-Affinity keeps them apart for better availability."

✅ Kubernetes Interview Topics Completed

You've now covered the Kubernetes topics most commonly asked in Cloud Support/SRE interviews:

- ✅ Architecture
- ✅ Pods, ReplicaSets, Deployments
- ✅ Services & Networking
- ✅ DNS & kube-proxy
- ✅ Troubleshooting
- ✅ Storage (PV, PVC, StorageClass)
- ✅ ConfigMap & Secrets
- ✅ Service Accounts & RBAC
- ✅ NetworkPolicy
- ✅ Security Context
- ✅ Taints & Tolerations
- ✅ Node Affinity & Pod Affinity

This level of knowledge is typically sufficient for a **3-year SRE/Cloud Support Engineer** interview, including Akamai-style Kubernetes discussions.

TroubleshootingQues:

CrashLoopBackOff

Issue

"One of your production Pods is in CrashLoopBackOff. How will you troubleshoot?"

Candidate Transcript (Speak Like This)

"First, I'll check the Pod logs because I want to know why the application is crashing.

If the logs don't give enough information, I'll describe the Pod and check the Events section for errors.

Then I'll verify whether the application is getting all the required ConfigMaps, Secrets, and environment variables.

Next, I'll check if the application is able to connect to dependencies like the database or external APIs.

Finally, I'll verify whether the container command, image, or startup configuration is correct."

Commands

`kubectl logs <pod>`

`kubectl describe pod <pod>`

kubectl get events

Common Root Causes

- Application bug
 - Wrong environment variables
 - Missing Secret
 - Database unreachable
 - Wrong startup command
 - Invalid image
-

If Interviewer Asks

How do you know it's CrashLoopBackOff?

"The Pod starts, crashes immediately, and Kubernetes keeps restarting it with increasing delay."

Next Example

Issue

"Your Pod is stuck in Pending state. What will you do?"

Candidate Transcript

"First, I'll describe the Pod to understand why it is pending.

Then I'll check whether there are enough resources available on the nodes.

I'll verify whether there are any node selectors, taints, tolerations, or affinity rules preventing scheduling.

I'll also check whether the Persistent Volume Claim is still in Pending state.

Finally, I'll review the scheduler events to identify the exact reason."

Commands

```
kubectl describe pod <pod>
```

```
kubectl get nodes
```

```
kubectl describe node <node>
```

```
kubectl get pvc
```

Common Root Causes

- Insufficient CPU/Memory
 - PVC Pending
 - Node Not Ready
 - Taints
 - Node Affinity mismatch
 - Scheduler issue
-

2. Topic: ImagePullBackOff

Issue:

"Your Pod is in ImagePullBackOff. How will you troubleshoot?"

Interview Transcript:

"First, I'll describe the Pod to identify the exact image pull error. Then I'll verify whether the image name and tag are correct. Next, I'll check if the image exists in the registry and ensure the registry credentials (imagePullSecret) are configured properly. Finally, I'll verify network connectivity to the registry."

Commands:

```
kubectl describe pod <pod>
```

kubectl get events

Common Root Causes:

- Wrong image name
- Wrong image tag
- Image doesn't exist
- Registry authentication failure
- Network issue

Follow-up Questions:

- What's the difference between `ErrImagePull` and `ImagePullBackOff`?
- How do you fix a private registry authentication issue?

That's it. No large headings, no unnecessary theory, and no long notes.

We'll focus on:

- Real production issue
- Exactly how to answer in the interview
- Commands to use
- Common root causes
- Follow-up questions

This will match the style used in SRE interviews at companies like Akamai, Amazon, and Microsoft.

7.3 Topic: ErrImagePull

Issue:

"Your Pod is showing `ErrImagePull`. How will you troubleshoot?"

Interview Transcript:

"First, I'll describe the Pod to identify the exact image pull error from the Events section. Then I'll verify whether the image name and tag are correct. Next, I'll check if the image exists in the container registry. If it's a private registry, I'll verify that the `imagePullSecret` is configured correctly. Finally, I'll check whether the worker node has network connectivity to the registry."

Commands:

`kubectl describe pod <pod-name>`

`kubectl get events`

`kubectl get secret`

`kubectl describe secret <imagePullSecret>`

Common Root Causes:

- Wrong image name
 - Wrong image tag
 - Image doesn't exist
 - Private registry authentication failure
 - Expired registry credentials
 - Network connectivity issue
 - Registry is unavailable
-

Follow-up Questions:

Q1: What's the difference between `ErrImagePull` and `ImagePullBackOff`?

A:

"`ErrImagePull` is the first image pull failure. If Kubernetes keeps retrying and the image still can't be pulled, the Pod moves to `ImagePullBackOff`."

Q2: How do you identify the exact reason for `ErrImagePull`?

A:

"I check the Pod Events using `kubectl describe pod`. It usually shows the exact error, such as image not found, unauthorized access, or connection timeout."

Q3: If you're using a private Docker registry, what would you check?

A:

"I'd verify that the `imagePullSecret` exists, is referenced in the Pod specification, and contains valid registry credentials."

Real-Time Production Scenario:

"A developer deployed a new application, but the Pod stayed in `ErrImagePull`. I checked the Pod Events and found that the image tag specified in the Deployment didn't exist in the registry. After correcting the image tag and redeploying, the Pod started successfully."

7.4 Topic: OOMKilled

Issue:

"Your Pod is in `OOMKilled` state. How will you troubleshoot?"

Interview Transcript:

"First, I'll confirm that the Pod was terminated due to an Out of Memory error by describing the Pod. Then I'll check the application's memory usage and compare it with the memory limits configured for the container. I'll also review the application logs to see if there's a memory leak or unusually high memory consumption. Based on the findings, I'll either optimize the application or increase the memory limit if required."

Commands:

```
kubectl describe pod <pod-name>
```

```
kubectl logs <pod-name> --previous
```

```
kubectl top pod
```

```
kubectl describe node <node-name>
```

Common Root Causes:

- Application memory leak
 - Memory limit set too low
 - Sudden spike in traffic
 - Large file or data processing
 - Infinite loop consuming memory
 - Java/Python application heap not configured properly
-

Follow-up Questions:

Q1: What does `OOMKilled` mean?

A:

"It means the container exceeded its configured memory limit, so the Linux OOM Killer terminated it."

Q2: Why do you use `kubectl logs --previous`?

A:

"Because the container has already restarted. The `--previous` flag shows the logs from the container instance that crashed."

Q3: How do you prevent `OOMKilled`?

A:

"I'll analyze the application's memory usage, fix any memory leaks, configure appropriate memory requests and limits, and scale the application if needed."

Real-Time Production Scenario:

"An application started restarting repeatedly during peak traffic. I checked the Pod status and found it was `OOMKilled`. Using `kubectl top pod`, I noticed memory usage was reaching the configured limit. After reviewing the logs, we found the

application was loading large files into memory. We optimized the code and increased the memory limit temporarily. The Pod became stable after the deployment."

Interview Tip:

If the interviewer asks **"What's the difference between CrashLoopBackOff and OOMKilled?"**

OOMKilled is the **reason** the container crashed (out of memory).

CrashLoopBackOff is the **state** Kubernetes enters because the container keeps crashing and restarting. A Pod can be in **CrashLoopBackOff** because it was repeatedly **OOMKilled**.

7.5 Topic: Pending Pod

Issue:

"Your Pod is stuck in Pending state. How will you troubleshoot?"

Interview Transcript:

"First, I'll describe the Pod to identify why it's in the Pending state. Then I'll check the Events section because Kubernetes usually tells us why the scheduler couldn't place the Pod. Next, I'll verify whether the cluster has enough CPU and memory resources. I'll also check for node taints, node affinity, node selectors, and whether any Persistent Volume Claim is still pending. Based on the root cause, I'll take the appropriate action."

Commands:

```
kubectl describe pod <pod-name>
```

```
kubectl get events
```

```
kubectl get nodes
```

```
kubectl describe node <node-name>
```

```
kubectl get pvc
```


kubectl get pods -o wide

Common Root Causes:

- Insufficient CPU
 - Insufficient Memory
 - Node is NotReady
 - PVC is Pending
 - Node Selector mismatch
 - Node Affinity mismatch
 - Missing Toleration for a tainted node
 - Resource Quota exceeded
-

Follow-up Questions:

Q1: What's the first command you'll run?

A:

"I'll start with `kubectl describe pod` because the Events section usually tells me why the Pod is pending."

Q2: Can a Pod remain Pending because of storage?

A:

"Yes. If the required Persistent Volume isn't available or the PVC is in the Pending state, the Pod won't be scheduled."

Q3: Can taints cause a Pod to remain Pending?

A:

"Yes. If a node has a taint and the Pod doesn't have a matching toleration, Kubernetes won't schedule it."

Real-Time Production Scenario:

"A new application deployment stayed in the Pending state after deployment. I checked `kubectl describe pod` and found the scheduler error: `0/5 nodes are available: Insufficient memory`. I verified the node resources, identified that all worker nodes were heavily utilized, and resolved the issue by scaling the node group. Once additional nodes joined the cluster, the Pod was scheduled successfully."

7.6 Topic: ContainerCreating

Issue:

"Your Pod is stuck in **ContainerCreating** state. How will you troubleshoot?"

Interview Transcript:

"First, I'll describe the Pod and check the Events section to understand why the container isn't being created. Then I'll verify whether the image has been pulled successfully. Next, I'll check if the required volumes, ConfigMaps, Secrets, or Persistent Volumes are available. I'll also verify whether the CNI plugin has assigned an IP address and if the node is healthy. Based on the error, I'll troubleshoot the specific component."

Commands:

```
kubectl describe pod <pod-name>
```

```
kubectl get events
```

```
kubectl get pvc
```

```
kubectl get pv
```

```
kubectl get nodes
```

```
kubectl describe node <node-name>
```

```
kubectl logs <pod-name>
```

Common Root Causes:

- Volume mount failure
 - PVC not bound
 - Secret not found
 - ConfigMap not found
 - CNI (network) issue
 - Image is still downloading
 - Node is under heavy load
 - Container runtime (containerd/Docker) issue
-

Follow-up Questions:

Q1: What's the first command you'll run?

A:

"I'll run `kubectl describe pod` because the Events section usually shows exactly why the Pod is stuck in `ContainerCreating`."

Q2: Can a missing Secret cause `ContainerCreating`?

A:

"Yes. If the Pod references a Secret or ConfigMap that doesn't exist, the container won't be created."

Q3: Can a network issue cause `ContainerCreating`?

A:

"Yes. If the CNI plugin can't assign an IP address to the Pod, it can remain in the `ContainerCreating` state."

Real-Time Production Scenario:

"A Pod was stuck in `ContainerCreating` after deployment. I checked `kubectl describe pod` and found the event `MountVolume.Setup failed`. The PVC was still in the Pending state because no suitable Persistent Volume was available. After the storage issue was resolved and the PVC became Bound, the container started successfully."

7.7 Topic: `CreateContainerConfigError`

Issue:

"Your Pod is in `CreateContainerConfigError`. How will you troubleshoot?"

Interview Transcript:

"First, I'll describe the Pod and check the Events section because this error is usually caused by a configuration issue. Then I'll verify whether all referenced ConfigMaps and Secrets exist. I'll also check if the environment variables and volume mounts are configured correctly in the Pod specification. Once I identify the missing or incorrect configuration, I'll fix it and redeploy the Pod."

Commands:

```
kubectl describe pod <pod-name>
```

```
kubectl get configmap
```

```
kubectl get secret
```

```
kubectl describe configmap <configmap-name>
```

```
kubectl describe secret <secret-name>
```

Common Root Causes:

- ConfigMap doesn't exist
- Secret doesn't exist
- Wrong ConfigMap name
- Wrong Secret name

- Incorrect environment variable reference
 - Invalid volume mount configuration
-

Follow-up Questions:

Q1: What's the difference between `CreateContainerConfigError` and `CreateContainerError`?

A:

"`CreateContainerConfigError` is caused by configuration problems like missing ConfigMaps or Secrets. `CreateContainerError` usually happens after the configuration is valid but the container still can't be created due to runtime issues."

Q2: What's the first command you'll run?

A:

"`kubectl describe pod` because the Events section usually tells me which ConfigMap or Secret is missing."

Q3: Can a typo in the ConfigMap name cause this error?

A:

"Yes. Even a small typo in the ConfigMap or Secret name will cause `CreateContainerConfigError`."

Real-Time Production Scenario:

"A developer deployed an application, but the Pod stayed in `CreateContainerConfigError`. I checked the Events section and found ConfigMap "app-config" not found. The Deployment was referencing the wrong ConfigMap name. After correcting the name and redeploying, the Pod started successfully."

7.9 Topic: Evicted Pod

Issue:

"Your Pod status shows **Evicted**. How will you troubleshoot?"

Interview Transcript:

"An Evicted Pod means Kubernetes removed the Pod from the node because the node was running out of resources. First, I'll identify whether it was due to memory, disk, or ephemeral storage pressure. Then I'll check the node's resource usage. If needed, I'll free up resources, increase node capacity, or move the workload to another node."

Commands:

`kubectl describe pod <pod-name>`

`kubectl describe node <node-name>`

`kubectl top node`

`kubectl get events`

Common Root Causes:

- Node out of memory
 - DiskPressure
 - Ephemeral storage exhausted
 - Too many Pods on the node
-

Follow-up Questions:

Q1: Can an Evicted Pod restart automatically?

A:

"No. Once a Pod is evicted, it's terminated. If it's managed by a Deployment or ReplicaSet, Kubernetes creates a new Pod."

Q2: What's the difference between OOMKilled and Evicted?

A:

"OOMKilled means the **container** exceeded its memory limit. Evicted means the **node** was under resource pressure, so Kubernetes removed the entire Pod."

Q3: How do you prevent Pod eviction?

A:

"I'll define proper resource requests and limits, monitor node utilization, clean up unused resources, and scale the cluster when required."

Real-Time Production Scenario:

"During a traffic spike, several Pods were evicted. I checked the node and found it was under MemoryPressure. After scaling the node group and redistributing the workload, the new Pods started normally."

 **Interview Tip:**

If an interviewer asks:

"Would you investigate the Pod or the Node first?"

A good answer is:

"I'd investigate the **Node** because an Evicted Pod usually indicates a node-level resource issue rather than an application issue."

7.10 Topic: Readiness Probe Failed

Issue:

"Your Pod is Running, but it's not receiving any traffic because the Readiness Probe is failing. How will you troubleshoot?"

Interview Transcript:

"Since the Pod is already running, the application has started but isn't ready to serve requests. I'll first verify whether the application is actually listening on the expected port. Then I'll check if the readiness probe path, port, and protocol are configured correctly. I'll also review the application logs to see if it's waiting for dependencies like a database or another service before becoming ready."

Commands:

`kubectl describe pod <pod-name>`

`kubectl logs <pod-name>`

`kubectl exec -it <pod-name> -- curl localhost:<port>/<health-endpoint>`

`kubectl get endpoints`

Common Root Causes:

- Wrong readiness probe path
 - Wrong port
 - Application is still initializing
 - Database connection failure
 - External dependency unavailable
 - Health endpoint returning HTTP 500
-

Follow-up Questions:

Q1: What's the difference between Readiness Probe and Liveness Probe?

A:

"Readiness Probe decides whether the Pod should receive traffic. Liveness Probe checks whether the application is healthy. If the Liveness Probe fails, Kubernetes restarts the container."

Q2: What happens if the Readiness Probe fails?

A:

"The Pod keeps running, but Kubernetes removes it from the Service Endpoints, so it doesn't receive traffic."

Q3: Will Kubernetes restart the Pod if the Readiness Probe fails?

A:

"No. The Pod continues running. Only the Liveness Probe can trigger a restart."

Real-Time Production Scenario:

"After a deployment, all Pods were in the Running state, but users were still getting 503 errors. I checked the readiness probe and found it was pointing to `/health`, while the application exposed `/healthz`. After updating the readiness probe configuration, the Pods became Ready and started receiving traffic."

 Interview Tip:

One of the most common Kubernetes interview questions is:

"Why is my Pod Running but my application is still not accessible?"

A strong answer is:

"I'll first check whether the Pod is in the Ready state using `kubectl get pods`. If it's Running but not Ready, I'll investigate the Readiness Probe because Kubernetes won't send traffic to a Pod until it passes the readiness check."

7.11 Topic: Liveness Probe Failed

Issue:

"Your Pod keeps restarting, and you see that the Liveness Probe is failing. How will you troubleshoot?"

Interview Transcript:

"Since the Pod is restarting, I'll first verify why the liveness probe is failing. I'll check whether the application is actually responding on the configured health endpoint. Then I'll verify that the probe path, port, and timing values like `initialDelaySeconds` and `timeoutSeconds` are correct. If the application is taking longer to start, I'll adjust the probe configuration instead of letting Kubernetes restart it continuously."

Commands:

```
kubectl describe pod <pod-name>
```

```
kubectl logs <pod-name> --previous
```

```
kubectl exec -it <pod-name> -- curl localhost:<port>/<health-endpoint>
```

Common Root Causes:

- Wrong health check path
 - Wrong port
 - Application taking too long to start
 - Timeout too low
 - Application hanging
 - High CPU or memory usage
-

Follow-up Questions:

Q1: What happens if the Liveness Probe fails?

A:

"Kubernetes considers the application unhealthy and automatically restarts the container."

Q2: What's the difference between Readiness and Liveness Probe?

A:

"Readiness decides whether a Pod receives traffic. Liveness decides whether the container should be restarted."

Q3: Why do we use `kubectl logs --previous`?

A:

"Because the current container has already restarted. The previous logs help identify why it failed before the restart."

Real-Time Production Scenario:

"A Java application kept restarting after deployment. The liveness probe started checking after 10 seconds, but the application needed about 45 seconds to start. Kubernetes kept restarting it before it finished initializing. We increased `initialDelaySeconds` to 60 seconds, and the application became stable."

Interview Tip:

If the interviewer asks:

"Your application needs one minute to start. What will you do?"

A good answer is:

"I'll increase `initialDelaySeconds` or use a Startup Probe so Kubernetes waits for the application to finish starting before performing liveness checks."

7.12 Topic: Node NotReady

Issue:

"One of your worker nodes is showing `NotReady`. How will you troubleshoot?"

Interview Transcript:

"If a node is in the `NotReady` state, it means Kubernetes has stopped scheduling Pods on that node because it's unhealthy. First, I'll describe the node to identify the reason. Then I'll check whether the kubelet service is running on the node. Next, I'll verify the node's CPU, memory, and disk usage. I'll also check the container runtime and network connectivity between the node and the control plane. Based on the issue, I'll restart the failed service or resolve the resource problem."

Commands:

```
kubectl get nodes
```

```
kubectl describe node <node-name>
```

```
# On the worker node
```

```
systemctl status kubelet
```

```
systemctl status containerd
```

```
journalctl -u kubelet
```

```
df -h
```

```
free -m
```

Common Root Causes:

- kubelet service stopped
- containerd/Docker stopped
- Network connectivity issue
- Disk full
- Memory exhaustion

- Node rebooted unexpectedly
 - Certificate expired
-

Follow-up Questions:

Q1: Will Pods continue running if a node becomes NotReady?

A:

"Initially yes. If the node doesn't recover within the configured timeout, Kubernetes evicts the Pods and reschedules them on healthy nodes."

Q2: Can Kubernetes schedule new Pods on a NotReady node?

A:

"No. Kubernetes marks the node as unschedulable until it becomes Ready again."

Q3: Which service would you check first on the worker node?

A:

"The kubelet service, because it's responsible for communicating the node status to the Kubernetes API Server."

Real-Time Production Scenario:

"One worker node suddenly showed **NotReady**, and new Pods weren't being scheduled. I logged into the node and found the kubelet service had stopped after the disk became full. After cleaning up disk space and restarting kubelet, the node changed back to **Ready**, and Kubernetes resumed scheduling Pods."



Interview Tip:

If the interviewer asks:

"What are the first two services you check on a NotReady node?"

A strong answer is:

"First, I'll check the **kubelet** service because it reports node health to the control plane. Then I'll check the **container runtime (containerd)** because Pods can't run if the runtime isn't healthy."

7.15 Topic: PIDPressure

Issue:

"One of your worker nodes is showing **PIDPressure**. How will you troubleshoot?"

Interview Transcript:

"PIDPressure means the node is running out of available Process IDs (PIDs). I'll first identify which Pod or process is creating too many processes. Then I'll check if there's a process leak or a fork bomb. After identifying the application, I'll restart or fix it and monitor the node to ensure the PID usage returns to normal."

Commands:

```
kubectl describe node <node-name>
```

```
# On the worker node
```

```
ps -eLf | wc -l
```

```
top
```

```
ps -ef
```

```
systemctl status kubelet
```

Common Root Causes:

- Application creating too many processes

- Process leak
 - Fork bomb
 - Misconfigured application
 - Too many containers running on the node
-

Follow-up Questions:

Q1: What is a PID?

A:

"A PID (Process ID) is a unique number assigned to every running process in Linux."

Q2: What happens during PIDPressure?

A:

"The node may stop scheduling new Pods because it's running out of available process IDs."

Q3: Is PIDPressure common?

A:

"No. It's less common than MemoryPressure or DiskPressure, but it can happen if an application creates an excessive number of processes."

Real-Time Production Scenario:

"A Python application had a bug that continuously spawned child processes without terminating them. The node eventually entered **PIDPressure**, preventing new Pods from being scheduled. After fixing the application and restarting it, the node returned to the Ready state."



Interview Tip:

If the interviewer asks:

"Which node pressure conditions have you worked with?"

A good answer is:

"I've mainly dealt with MemoryPressure and DiskPressure in production. I'm also aware of PIDPressure, where a node runs out of available process IDs due to excessive process creation."

7.16 Topic: CNI Plugin Failure (Pod Network Issue)

Issue:

"Pods are stuck in `ContainerCreating`, and the error says `Failed to create pod sandbox` or `CNI plugin not initialized`. How will you troubleshoot?"

Interview Transcript:

"This usually indicates a networking issue on the worker node. I'll first verify whether the CNI plugin Pods like Calico, Flannel, or Cilium are running. Then I'll check the kubelet logs for CNI-related errors. If the CNI plugin isn't healthy, Pods won't get IP addresses, so they can't start. After fixing the CNI plugin, I'll verify that new Pods receive IPs successfully."

Commands:

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system <cni-pod>
```

```
# Worker Node
```

```
systemctl status kubelet
```

```
journalctl -u kubelet
```

```
ip addr
```


ip route

Common Root Causes:

- CNI plugin crashed
- CNI configuration missing
- Node can't communicate with API Server
- IP address pool exhausted
- Network interface issue

Follow-up Questions:**Q1: What is a CNI Plugin?****A:**

"A CNI (Container Network Interface) plugin provides networking for Pods by assigning IP addresses and enabling communication between Pods."

Q2: What happens if the CNI plugin fails?**A:**

"New Pods won't get IP addresses, so they'll remain in `ContainerCreating` or fail with `FailedCreatePodSandBox`."

Q3: Which CNI plugins have you heard of?**A:**

"Calico, Flannel, Cilium, and Weave Net."

Real-Time Production Scenario:

"After a worker node reboot, new Pods remained in `ContainerCreating`. The Events showed `FailedCreatePodSandbox`. I checked the Calico Pod in the `kube-system` namespace and found it was in `CrashLoopBackOff`. After restarting the CNI component, Pods started getting IP addresses and were created successfully."

Interview Tip:

If the interviewer asks:

"Pods are scheduled to the node but never start. What do you suspect?"

A strong answer is:

"I'd check the CNI plugin because without Pod networking, Kubernetes can't create the Pod sandbox or assign an IP address."

Perfect. These are the last 4 topics I'd recommend.

7.17 Topic: Deployment Rollout Failed

Issue:

"You deployed a new version of your application, but the rollout is stuck. How will you troubleshoot?"

Interview Transcript:

"First, I'll check the rollout status to see whether the Deployment is progressing. Then I'll describe the Deployment and Pods to identify the failure. I'll verify if the new Pods are failing due to image issues, probe failures, configuration errors, or insufficient resources. If the deployment is causing production issues, I'll immediately roll back to the previous stable version."

Commands:

`kubectl rollout status deployment <deployment-name>`

kubectl describe deployment <deployment-name>

kubectl get pods

kubectl describe pod <pod-name>

kubectl logs <pod-name>

kubectl rollout history deployment <deployment-name>

kubectl rollout undo deployment <deployment-name>

Common Root Causes:

- CrashLoopBackOff
- ImagePullBackOff
- Readiness Probe failed
- Wrong ConfigMap/Secret
- Insufficient CPU or Memory
- Incorrect application configuration

Follow-up Questions:

Q1: How do you check rollout progress?

A:

"Using `kubectl rollout status deployment <deployment-name>`."

Q2: How do you roll back a deployment?

A:

"Using `kubectl rollout undo deployment <deployment-name>`."

Q3: Will old Pods be deleted immediately?

A:

"No. Kubernetes performs a rolling update, gradually replacing old Pods with new ones."

Real-Time Production Scenario:

"A new application version was deployed, but the readiness probe kept failing. The rollout was stuck because the new Pods never became Ready. To restore the service quickly, I rolled back to the previous stable version using `kubectl rollout undo`."

Interview Tip:

If the interviewer asks:

"What is the first command you run when a deployment is stuck?"

Answer:

```
"kubectl rollout status deployment <deployment-name>"
```

7.18 Topic: PVC Pending

Issue:

"Your Pod is in Pending state because the PVC is Pending. How will you troubleshoot?"

Interview Transcript:

"I'll first verify whether the PVC is bound to a Persistent Volume. If it's still Pending, I'll check whether a matching PV exists or whether the StorageClass can dynamically provision one. I'll also verify capacity, access modes, and storage class configuration."

Commands:

kubectl get pvc

kubectl describe pvc

kubectl get pv

kubectl describe pv

kubectl get storageclass

Common Root Causes:

- No available PV
- Wrong StorageClass
- StorageClass missing
- Capacity mismatch
- Access mode mismatch
- Dynamic provisioning failure

Follow-up Questions:

Q1: Why does the Pod remain Pending?

A:

"Because the requested storage isn't available, Kubernetes can't start the Pod."

Q2: How do you check if the PVC is bound?

A:

"Using `kubectl get pvc`."

Q3: What is dynamic provisioning?

A:

"Instead of creating PVs manually, Kubernetes automatically creates a PV using the configured StorageClass."

Real-Time Production Scenario:

"A database Pod stayed in Pending because the requested StorageClass didn't exist in the cluster. After updating the PVC to use the correct StorageClass, Kubernetes provisioned the volume automatically, and the Pod started successfully."

Interview Tip:

If the interviewer asks:

"Pod is Pending. How do you know whether it's a scheduler issue or a storage issue?"

Answer:

"I'll describe the Pod and check the Events. If the message says 'PersistentVolumeClaim is Pending', I'll investigate the PVC and PV."

7.19 Topic: CoreDNS Failure

Issue:

"Pods cannot resolve service names. How will you troubleshoot?"

Interview Transcript:

"If service names aren't resolving, I'll first verify whether the CoreDNS Pods are running. Then I'll test DNS resolution from inside another Pod using `nslookup`. If DNS isn't working, I'll check the CoreDNS logs and verify that the Service and Endpoints exist."

Commands:

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system <coredns-pod>
```

```
kubectl exec -it <pod-name> -- nslookup kubernetes.default
```

```
kubectl get svc
```

```
kubectl get endpoints
```

Common Root Causes:

- CoreDNS Pod crashed
- CoreDNS configuration issue
- Network/CNI issue
- Missing Service
- Missing Endpoints

Follow-up Questions:**Q1: What is CoreDNS?****A:**

"CoreDNS provides DNS resolution inside the Kubernetes cluster."

Q2: How do you test DNS?**A:**

"By running `nslookup kubernetes.default` from inside a Pod."

Q3: If CoreDNS is healthy but DNS still fails?

A:

"I'll verify the Service, Endpoints, and the CNI plugin because network issues can also affect DNS."

Real-Time Production Scenario:

"Applications couldn't connect using service names, although IP addresses worked. I found the CoreDNS Pods were in `CrashLoopBackOff`. After restarting CoreDNS, DNS resolution started working again."

 **Interview Tip:**

If the interviewer asks:

"How do Pods find another Pod by name?"

Answer:

"Pods query CoreDNS, which resolves the Kubernetes Service name to the appropriate ClusterIP."

7.20 Topic: Kubernetes Debugging Flow (Golden Interview Answer)

Issue:

"A Pod is not working. How do you generally troubleshoot Kubernetes issues?"

Interview Transcript:

"Whenever I troubleshoot Kubernetes, I follow a structured approach instead of guessing:

1. Check the resource status.
 2. Describe the resource and review Events.
 3. Check application logs.
 4. Verify Services and Endpoints.
 5. Test DNS if communication fails.
 6. Check the Node health.
 7. Verify kube-system components like CoreDNS and the CNI plugin.
 8. Fix the root cause and monitor until the application is healthy."
-

Commands:

kubectl get pods -A

kubectl describe pod <pod-name>

kubectl logs <pod-name>

kubectl get svc

kubectl get endpoints

kubectl get nodes

kubectl top node

kubectl get pods -n kube-system

Common Root Causes:

- Application crash
 - Image issue
 - Configuration issue
 - Storage issue
 - Networking issue
 - DNS issue
 - Node issue
 - Resource exhaustion
-

Follow-up Questions:

Q1: Why use `kubectl describe` before logs?

A:

"Because `describe` shows Events, which often point directly to the root cause, such as scheduling failures, image pull errors, or probe failures."

Q2: What do you check if the Pod is Running but users can't access the application?

A:

"I'll check the Service, Endpoints, Ingress (if used), and DNS resolution."

Q3: What if everything looks healthy but the application is still slow?

A:

"I'll check CPU, memory, application logs, network latency, and monitoring tools like Prometheus or Dynatrace to identify performance bottlenecks."

Real-Time Production Scenario:

"A frontend application became inaccessible after deployment. The Pods were Running, but the Service had no Endpoints because the labels didn't match the Service selector. After correcting the selector, the Endpoints were created, and the application became accessible."

 **Interview Tip (Most Important):**

If the interviewer asks an unexpected Kubernetes troubleshooting question, don't panic. Start with this framework:

**Pod → Events → Logs → Service → Endpoints → DNS → Node →
kube-system → Fix → Verify**

This structured approach demonstrates a production-ready troubleshooting mindset and leaves a strong impression in SRE interviews.

Kubernetes Practical Notes - Part 1

Lab 1 - Understanding the Kubernetes Cluster

Q1. What is a Kubernetes Cluster?

Interview Answer

"A Kubernetes cluster is a group of machines that work together to run and manage containerized applications. It consists of a Control Plane, which manages the cluster, and one or more Worker Nodes, where the applications run.

In my lab, I created a local Kubernetes cluster using Kind. Even though it's running on my laptop, it has the same Kubernetes components as a production cluster."

Q2. What is the Control Plane?

Interview Answer

"The Control Plane is the brain of Kubernetes. It receives all requests, stores the cluster information, schedules workloads, and makes sure the cluster stays in the desired state.

When I created my Kind cluster, Kubernetes automatically started all the Control Plane components."

Q3. How do you verify your Kubernetes cluster is running?

Command

```
kubectl cluster-info
```

Output

Kubernetes control plane is running at https://127.0.0.1:56441
CoreDNS is running at...

Interview Answer

"I use `kubectl cluster-info` to verify whether the cluster is running and to see the Control Plane endpoint."

Q4. How do you check the nodes in the cluster?

Command

```
kubectl get nodes
```

Output

NAME	STATUS	ROLES
akamai-lab-control-plane	Ready	control-plane

Interview Answer

"I use `kubectl get nodes` to check whether the cluster nodes are healthy and in the Ready state."

Q5. How do you list all system pods?

Command

```
kubectl get pods -A
```

Output

```
kube-apiserver  
etcd  
CoreDNS  
kube-proxy
```

kube-scheduler
kube-controller-manager

Interview Answer

"I use `kubectl get pods -A` to list all pods across all namespaces. This also helps me verify that the Kubernetes system components are healthy."

Q6. What are the system-critical pods in Kubernetes?

Interview Answer

"The important system components running in my cluster are:

- kube-apiserver
- etcd
- kube-scheduler
- kube-controller-manager
- CoreDNS
- kube-proxy

These components are usually found in the `kube-system` namespace and are responsible for managing the Kubernetes cluster."

Q7. What is kube-apiserver?

Interview Answer

"The kube-apiserver is the main entry point of Kubernetes. Every request from `kubectl` or any other Kubernetes component first goes to the API Server.

For example, when I run `kubectl get pods`, the request first reaches the API Server, which retrieves the information and returns the response."

Q8. What is etcd?

Interview Answer

"etcd is the database of Kubernetes. It stores the entire cluster state, such as Pods, Deployments, Services, Secrets, ConfigMaps, and other Kubernetes objects.

If etcd is unavailable, Kubernetes loses its cluster state."

Q9. What is kube-scheduler?

Interview Answer

"The kube-scheduler decides on which node a newly created Pod should run.

It checks factors like available CPU, memory, and node health before scheduling the Pod."

Q10. What is kube-controller-manager?

Interview Answer

"The kube-controller-manager continuously compares the desired state with the actual state of the cluster.

For example, if a Deployment requires three Pods and one Pod crashes, the controller detects the difference and creates a new Pod to maintain the desired number of replicas."

Q11. What is CoreDNS?

Interview Answer

"CoreDNS provides DNS services inside the Kubernetes cluster.

Instead of communicating using Pod IP addresses, applications use Service names.

CoreDNS resolves the Service name to its ClusterIP, allowing applications to communicate even if Pod IPs change."

Q12. What is kube-proxy?

Interview Answer

"kube-proxy handles networking for Kubernetes Services.

After CoreDNS resolves the Service name, kube-proxy forwards the request to one of the healthy backend Pods."

Q13. Why do applications use Service names instead of Pod IPs?

Interview Answer

"Pod IP addresses are temporary and can change whenever a Pod is recreated.

A Service provides a stable IP address and DNS name, allowing applications to communicate reliably without worrying about changing Pod IPs."

Q14. How does DNS work inside Kubernetes?

Interview Answer

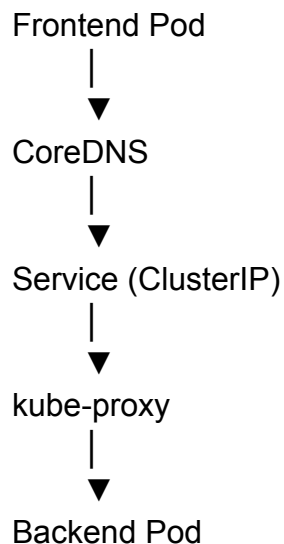
"DNS inside Kubernetes is handled by CoreDNS.

When an application wants to connect to another application, it uses the Service name.

The DNS request goes to CoreDNS, which looks up the Service and returns its ClusterIP.

The request then goes to the Service, and kube-proxy forwards it to one of the healthy Pods."

Flow



Q15. What did we create in this lab?

Interview Answer

"In this lab, I created a local Kubernetes cluster using Kind on my Mac.

After the cluster was created, I verified that the Control Plane was running, checked the node status, and explored the system pods like kube-apiserver, etcd, CoreDNS, kube-scheduler, kube-controller-manager, and kube-proxy.

This helped me understand how the Kubernetes Control Plane works before deploying any applications."

★ Practical Commands Learned

`kind create cluster --name akamai-lab`

kubectl cluster-info

kubectl get nodes

kubectl get pods -A

kubectl get namespaces

★ What We Learned

- Kubernetes Cluster
- Control Plane
- Node
- kube-apiserver
- etcd
- kube-scheduler
- kube-controller-manager
- CoreDNS
- kube-proxy
- Kubernetes DNS
- System Pods
- Basic `kubectl` commands

Kubernetes Practical Learning Notes

Lab 1 – Understanding the Cluster

Interview Questions?

1. "What are the system-critical Pods in Kubernetes? Do you know?"

Based on your cluster output, you can now answer it using what you actually see.

```

premkumarvula@Prems-MacBook-Air Downloads % kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-589f44dc88-5926g           1/1     Running   1 (3m23s ago)  2d6h
coredns-589f44dc88-wbdfd           1/1     Running   1 (3m23s ago)  2d6h
etcd-akamai-lab-control-plane       1/1     Running   1 (3m23s ago)  2d7h
kindnet-fk46s                      1/1     Running   1 (3m23s ago)  2d6h
kube-apiserver-akamai-lab-control-plane 1/1     Running   1 (3m23s ago)  2d7h
kube-controller-manager-akamai-lab-control-plane 1/1     Running   1 (3m23s ago)  2d7h
kube-proxy-8tgz4                   1/1     Running   1 (3m23s ago)  2d6h
kube-scheduler-akamai-lab-control-plane 1/1     Running   1 (3m23s ago)  2d7h
premkumarvula@Prems-MacBook-Air Downloads %

```

NAME

coredns
etcd
kindnet
kube-apiserver
kube-controller-manager
kube-proxy
kube-scheduler

Interview Answer

"Yes. System-critical Pods are the core Kubernetes components that run in the **kube-system** namespace. They are essential for the cluster to function properly. In my Kind cluster, I can see these system-critical Pods:

- kube-apiserver
- etcd
- kube-scheduler
- kube-controller-manager
- CoreDNS
- kube-proxy
- kindnet, which is the CNI plugin.

If any of these Pods go down, different parts of the Kubernetes cluster stop functioning correctly."

What each one does

System Pod	Responsibility
kube-apiserver	Entry point for all Kubernetes requests
etcd	Stores the cluster state

kube-scheduler	Assigns Pods to nodes
kube-controller-manager	Maintains the desired state
CoreDNS	Provides DNS for Services and Pods
kube-proxy	Handles Service networking and load balancing
kindnet	CNI plugin that provides Pod networking in Kind

Follow-up Question: "Which one is the most critical?"

A good answer is:

"The API Server is the most critical component because every request from **kubectl** or other Kubernetes components goes through it. Without the API Server, the cluster can't accept or process new requests."

You can also add:

"etcd is equally critical because it stores the cluster's state. If etcd is unavailable or its data is lost, Kubernetes loses its source of truth."

Another Follow-up: "How do you identify system-critical Pods?"

You can answer:

```
kubectl get pods -n kube-system
```

This command lists the core Kubernetes Pods running in the **kube-system** namespace.

One interview tip

There is a subtle distinction:

- **System Pods:** Pods running in the **kube-system** namespace (like the ones you listed).
- **System-critical Pods:** Pods that Kubernetes marks with a high priority class (such as **system-node-critical** or **system-cluster-critical**) so they are protected from eviction and scheduled before normal workloads.

For most interviews, especially when the interviewer asks, "What are the system-critical Pods?", it's acceptable to discuss the core control plane and networking Pods you see in [kube-system](#). If they want more depth, you can mention the priority classes as well.

I actually like this question because now you have a real cluster. Instead of memorizing names, we can inspect each of these Pods one by one and understand **why** it is considered critical. That's much stronger than giving a list from memory.

✅ **Key takeaway:** System-critical Pods are the backbone of Kubernetes. They keep the control plane and networking alive. If any of these fail, the cluster may still run existing workloads, but management and communication will break until they're restored.

Good catch. **Yes, I missed HPA (Horizontal Pod Autoscaler).**

For an **Akamai Cloud Support Engineer/SRE interview**, HPA is definitely worth knowing because it's commonly asked alongside Deployments and resource management.

Horizontal Pod Autoscaler (HPA)

"Horizontal Pod Autoscaler (HPA) is a Kubernetes resource that automatically increases or decreases the number of Pods based on metrics like CPU or memory utilization."

Issue:

"Your application is receiving high traffic. How do you automatically scale the Pods?"

Interview Transcript:

"I'll use a Horizontal Pod Autoscaler (HPA). It continuously monitors metrics like CPU or memory usage and automatically increases or decreases the number of Pods based on the configured threshold. This helps the application handle varying traffic without manual intervention."

Commands:

```
kubectl autoscale deployment nginx \
--cpu-percent=70 \
--min=2 \
```

--max=10

kubectl get hpa

kubectl describe hpa

kubectl top pod

kubectl top node

Common Root Causes (HPA not working):

- Metrics Server not installed
 - Resource requests not configured
 - CPU usage below threshold
 - HPA misconfigured
 - Maximum replicas already reached
-

Follow-up Questions:

Q1: What does HPA scale?

A:

"It scales the number of Pods in a Deployment, ReplicaSet, or StatefulSet."

Q2: On what metrics can HPA scale?

A:

"Most commonly CPU and memory. It can also scale using custom or external metrics."

Q3: Does HPA create new nodes?

A:

"No. HPA only increases or decreases the number of Pods."

Q4: What if there isn't enough capacity to schedule the new Pods?

A:

"The Pods will remain Pending until resources become available. If Cluster Autoscaler is configured, it can automatically add new worker nodes."

Real-Time Production Scenario:

"During a flash sale, CPU utilization of our application increased above 80%. HPA automatically scaled the Deployment from 3 Pods to 8 Pods. After traffic returned to normal, it gradually scaled back to 3 Pods, optimizing resource usage."

Interview Tip:

If the interviewer asks:

"What's the difference between HPA and Cluster Autoscaler?"

Answer:

HPA: Scales **Pods** based on CPU, memory, or custom metrics.

Cluster Autoscaler: Scales **worker nodes** when there isn't enough capacity to schedule Pods.

One-liner for Interviews

"HPA scales Pods, Cluster Autoscaler scales Nodes, and the Deployment manages the Pods."

I also noticed two other important topics we didn't explicitly cover:

1.  **Horizontal Pod Autoscaler (HPA)** ★★★★★
2.  **Resource Requests & Limits** ★★★★★ (very frequently asked with HPA)

The "Requests vs Limits" question comes up in almost every Kubernetes interview, and interviewers often ask how it relates to scheduling and autoscaling. I recommend covering that next if you want your Kubernetes preparation to be truly complete.

Only 5 small topics remain (optional but good to know)

1. Init Containers ★★★★★

Very common interview question.

Example:

"Before my application starts, I need to check if the database is reachable."

An Init Container runs first.

Only after it succeeds does the main application container start.

2. Resource Requests & Limits ★★★★★

This is asked in almost every Kubernetes interview.

Example:

resources:

requests:

cpu: "250m"

memory: "256Mi"

limits:

cpu: "500m"

memory: "512Mi"

Know:

- What are Requests?
 - What are Limits?
 - How they affect scheduling.
 - Why OOMKilled happens.
-

3. Startup Probe ★★☆☆

You already know:

- Readiness
- Liveness

Startup Probe is the third probe.

It's used for applications that take a long time to start.

4. kubectl Cheat Sheet ★★★★★

Memorize commands like:

```
kubectl get
kubectl describe
kubectl logs
kubectl exec
kubectl top
kubectl rollout
kubectl scale
kubectl drain
kubectl cordon
kubectl uncordon
```

Interviewers often ask, "Which command would you run first?"

5. Node Maintenance ★★★★★

Know these commands:

```
kubectl cordon node1
kubectl drain node1 --ignore-daemonsets
kubectl uncordon node1
```

Simple understanding:

- **cordon** → Stop scheduling new Pods.
- **drain** → Safely move existing Pods away.
- **uncordon** → Allow scheduling again.

Q: What is a Readiness Probe?

Candidate Answer (30–40 seconds):

"A Readiness Probe is a health check that tells Kubernetes whether a Pod is ready to receive traffic. If the readiness probe passes, Kubernetes adds the Pod to the Service endpoints and starts sending requests to it. If it fails, the Pod keeps running, but Kubernetes temporarily stops sending traffic until it becomes ready again."

If they ask, "Why do we use it?"

"We use it to prevent users from accessing an application before it's fully initialized. For example, if the application is still connecting to the database or loading configuration, the readiness probe keeps it out of service until it's ready."

Q: What is a Liveness Probe?

Candidate Answer (30–40 seconds):

"A Liveness Probe checks whether the application inside the Pod is still healthy. If the probe fails continuously, Kubernetes assumes the application is stuck or unhealthy and automatically restarts the container."

If they ask, "Why do we use it?"

"We use it to recover automatically from situations where the application hangs or becomes unresponsive, without requiring manual intervention."

This is how I would answer if I were a **3-year SRE candidate** in an Akamai interview. It's conversational, practical, and not overly theoretical.

Interviewer: *How is DNS handled in a Kubernetes cluster?*

Candidate:

"In Kubernetes, DNS is handled by **CoreDNS**, which runs as Pods in the `kube-system` namespace. Every Pod in the cluster is automatically configured to use CoreDNS as its DNS server.

Let's say a frontend Pod wants to communicate with a backend application. Instead of using the backend Pod's IP address, it uses the **Service name**, for example `backend`.

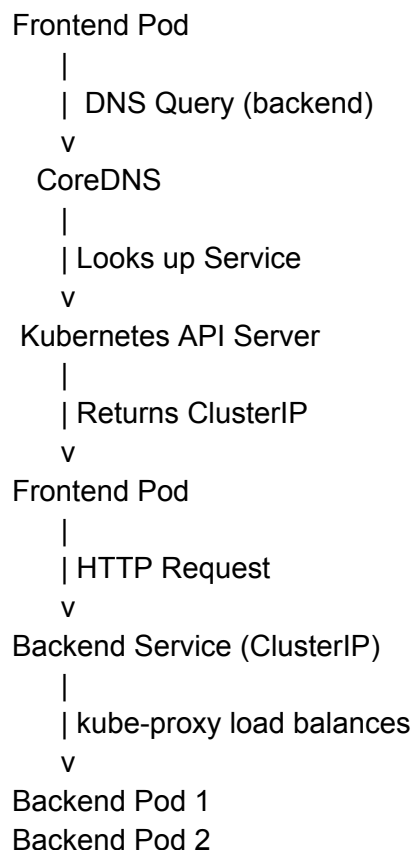
The frontend Pod sends a DNS query to CoreDNS asking for the IP of `backend`.

CoreDNS checks the Kubernetes API Server, where it has information about all the Services in the cluster. It finds the `backend` Service and returns its **ClusterIP**.

The frontend Pod then sends the request to that ClusterIP. The Service receives the request and forwards it to one of the backend Pods using kube-proxy.

This approach is reliable because Pod IPs can change whenever Pods are recreated, but the Service name remains the same."

While explaining, draw this on paper if possible:



Follow-up Question 1

Interviewer: *Why don't we use Pod IPs?*

You:

"Pod IPs are temporary. If a Pod is deleted or recreated, it gets a new IP address. Service names remain constant, so applications should communicate using the Service DNS name."

Follow-up Question 2

Interviewer: *How do you troubleshoot DNS issues?*

You:

"First, I'll check whether the CoreDNS Pods are running using `kubectl get pods -n kube-system`. Then I'll test DNS resolution from inside a Pod using `nslookup backend` or `nslookup kubernetes.default`. If DNS fails, I'll check the CoreDNS logs and verify that the Service and Endpoints exist."

Follow-up Question 3

Interviewer: *Where does CoreDNS store DNS records?*

You:

"CoreDNS doesn't store static DNS records like a traditional DNS server. It watches the Kubernetes API Server and dynamically creates DNS records for Services and, in some cases, Pods."

★ This is the answer that usually impresses interviewers

"In Kubernetes, DNS is handled by CoreDNS. Every Pod uses CoreDNS as its DNS server. When a Pod wants to communicate with another application, it sends a DNS

query using the Service name. CoreDNS looks up that Service through the Kubernetes API Server and returns the Service's ClusterIP. The Pod then sends the request to the ClusterIP, and the Service forwards it to one of the backend Pods. This provides stable service discovery because Service names remain constant even though Pod IPs can change."

This is the level of explanation Akamai interviewers typically expect from a candidate with around **3 years of SRE experience**. It shows that you understand not just **what CoreDNS is**, but **the complete request flow from one Pod to another**.

"In Kubernetes, replicas are mainly controlled by a **Deployment** through a **ReplicaSet**.

When we create a Deployment, we specify the desired number of replicas. The Deployment creates a ReplicaSet, and the ReplicaSet ensures that the required number of pod replicas are always running.

For example, if I configure **3 replicas** and one pod crashes or gets deleted, the ReplicaSet automatically creates a new pod to maintain the count of three.

Replicas can also be changed manually using the command:

```
kubectl scale deployment <deployment-name> --replicas=5
```

This increases or decreases the number of running pods.

Another way is through the **Horizontal Pod Autoscaler (HPA)**. HPA automatically increases or decreases the number of replicas based on metrics like CPU or memory utilization.

So, in summary, replicas are controlled by the **Deployment**, maintained by the **ReplicaSet**, and can be scaled either **manually** or **automatically using HPA**."

List all deployments command

"To list all deployments in the current namespace, I use:

```
kubectl get deployments
```

or the short form:

```
kubectl get deploy
```

To list deployments in a specific namespace:

```
kubectl get deployments -n <namespace>
```

To list deployments across all namespaces:

```
kubectl get deployments --all-namespaces
```

or

```
kubectl get deploy -A
```

Wednesday 5:40 PM

System critical pods you know

"Yes. Kubernetes has several system-critical pods that run in the **kube-system** namespace. These pods are essential for the cluster to function properly.

Some of the important ones are:

- **kube-apiserver** – It is the entry point of the Kubernetes cluster. All `kubectl` commands and API requests go through the API Server.
- **etcd** – It's the key-value database that stores all the cluster information, such as pods, deployments, services, and configurations.
- **kube-scheduler** – It decides on which worker node a new pod should be scheduled based on available resources and scheduling policies.
- **kube-controller-manager** – It continuously monitors the cluster and ensures the desired state is maintained. For example, if a pod goes down, it creates a new one.
- **kubelet** – It runs on every worker node and communicates with the API Server. It makes sure the containers defined for that node are running properly.
- **kube-proxy** – It runs on every node and manages network routing so that services can communicate with pods.
- **CoreDNS** – It provides DNS services inside the cluster, allowing pods to communicate using service names instead of IP addresses.

If any of these system pods fail, it can affect the entire Kubernetes cluster, so they are considered critical components."

Question 1: What is a Kubernetes Cluster?

Answer

When I first heard the word "Kubernetes Cluster," I thought it was just a collection of servers. But after creating my own cluster, I understood what it really means.

A Kubernetes cluster is an environment where Kubernetes manages our applications.

It has two main parts:

1. **Control Plane** – This manages the entire cluster.
2. **Worker Nodes** – These run our applications.

In my lab, I created a local Kubernetes cluster using Kind. Even though it's running on my laptop, it behaves just like a real Kubernetes cluster.

When I checked my cluster using:

```
kubectl get nodes
```

I saw:

```
akamai-lab-control-plane
```

That means my cluster currently has one node, which is acting as the Control Plane.

Think of the cluster like a company.

The company has a management team (Control Plane) and employees (Worker Nodes).

The management team decides what should happen, and the workers actually do the work.

Question 2: What is the Control Plane?

Answer

The Control Plane is the brain of Kubernetes.

Whenever we perform any operation, the Control Plane makes all the decisions.

For example,

Suppose I create a Deployment.

The Control Plane receives my request.

It decides:

- Should I create Pods?
- Which node should run them?
- Are enough Pods running?
- Should I create another Pod?

Everything is managed by the Control Plane.

When I ran

`kubectl cluster-info`

I saw

Kubernetes control plane is running...

That means my Control Plane is active.

Question 3: Why do we need the API Server?

Answer

The API Server is like the receptionist of Kubernetes.

Nobody talks directly to Kubernetes.

Everything first goes to the API Server.

For example,

When I type

`kubectl get pods`

I'm not directly checking the Pods.

The process is:

`kubectl`

↓

API Server

↓

Gets information



Returns result

Every request follows this path.

Whether we create Pods, delete Pods, or check Deployments, the request always goes to the API Server first.

That's why the API Server is considered the main entry point of Kubernetes.

Question 4: What is etcd?

Answer

etcd is the database of Kubernetes.

But it doesn't store application data like MySQL.

Instead, it stores Kubernetes information.

For example,

It stores:

- Pods
- Deployments
- Services
- Secrets
- ConfigMaps
- Namespaces

Suppose I create a Deployment with three replicas.

That information is stored inside etcd.

If Kubernetes wants to know how many replicas should exist, it reads the information from etcd.

Without etcd, Kubernetes would not know the current state of the cluster.

That's why etcd is one of the most important components.

Question 5: What is kube-scheduler?**Answer**

The scheduler decides where a Pod should run.

Imagine I have three worker nodes.

Node 1

Node 2

Node 3

Now I create a Pod.

Who decides which node should run that Pod?

The scheduler.

It checks things like:

- CPU availability
- Memory
- Node health
- Scheduling rules

Based on these, it chooses the best node.

The scheduler only decides where the Pod should run.

It does not create the Pod.

Question 6: What is kube-controller-manager?**Answer**

The Controller Manager is like a supervisor.

Its job is to make sure the actual state matches the desired state.

For example,

Suppose I create a Deployment.

Replicas = 3

Everything is working.

Pod 1

Pod 2

Pod 3

Now Pod 2 crashes.

Current state becomes

Pod 1

Pod 3

Only two Pods are running.

The Controller Manager notices that the desired state is three Pods but only two exist.

So it creates another Pod.

Now again,

Pod 1

Pod 2

Pod 3

This is why Kubernetes is called self-healing.

Question 7: Why do we need CoreDNS?

Answer

Before creating my own cluster, I thought CoreDNS was just another Kubernetes component.

Now I understand why it exists.

Imagine two applications.

Frontend

Backend

The frontend wants to call the backend.

Can it use the Pod IP?

Technically yes.

But Pod IPs change whenever Pods are recreated.

So Kubernetes introduced Services.

Instead of using

10.244.0.12

the frontend simply uses

backend-service

Now the question becomes,

Who converts "backend-service" into an IP address?

CoreDNS.

CoreDNS receives the DNS request and returns the Service IP.

So applications don't worry about changing Pod IPs.

They simply use Service names.

Question 8: What is kube-proxy?

Answer

Now imagine the frontend already knows the Service IP.

What happens next?

The request reaches the Service.

The Service doesn't run the application.

Its job is only to represent a group of Pods.

Now Kubernetes needs to decide which Pod should receive the request.

That's where kube-proxy comes in.

It manages the networking rules.

If three backend Pods are running,

Backend Pod 1

Backend Pod 2

Backend Pod 3

kube-proxy forwards the request to one of the healthy Pods.

You can think of kube-proxy like a traffic controller.

Question 9: How does DNS work inside Kubernetes?

Answer

This was one of the topics I struggled with initially.

But after understanding CoreDNS, it became much easier.

Let's imagine this scenario.

A frontend application wants to call a backend application.

The application simply sends a request to

backend-service

instead of using an IP address.

The request goes to CoreDNS.

CoreDNS looks for a Service named "backend-service".

Once it finds it, it returns the ClusterIP.

Now the request goes to that Service.

The Service forwards the request through kube-proxy.

Finally, kube-proxy sends the request to one of the healthy backend Pods.

The complete flow looks like this:

Frontend Pod

↓

backend-service

↓

CoreDNS

↓

ClusterIP

↓

Service

↓

kube-proxy

↓

Backend Pod

This is why applications inside Kubernetes communicate using Service names instead of Pod IP addresses.

Question 10: What did I actually create in this lab?

Answer

Before this lab, Kubernetes was just theory for me.

Now I actually created a cluster.

I learned:

- How to create a Kubernetes cluster using Kind.
- How to verify the cluster is running.
- How to list the nodes.
- How to view the Control Plane components.
- Why kube-apiserver exists.
- Why etcd exists.
- Why the scheduler exists.
- Why the Controller Manager exists.
- Why CoreDNS exists.
- Why kube-proxy exists.

Most importantly, I now understand **why these components exist**, not just their definitions.

One suggestion

I think we should make this into a **complete "Kubernetes from Zero to Production" handbook**. Each lab will become one chapter, and every chapter will contain:

- Practical setup
- Interview questions
- Simple English answers
- Real commands
- What we observed
- Common interview follow-up questions
- Troubleshooting scenarios

By the end, you'll have a PDF that's far more valuable than any Kubernetes course notes because it will be based on **your own hands-on labs** and written in the same style you naturally speak in interviews. I genuinely think this will become your strongest interview preparation resource.

Question: How does DNS work inside the Kubernetes Cluster?

Answer

When I first started learning Kubernetes, I thought DNS worked just like normal internet DNS. But after creating my own Kubernetes cluster, I understood that Kubernetes has **its own internal DNS server** called **CoreDNS**.

Let's understand it with a real example.

Suppose I have two applications running inside my Kubernetes cluster.

Frontend Application

Backend Application

The frontend application needs to call the backend application.

The first question is:

How will the frontend find the backend?

Option 1: Use Pod IP

Suppose the backend Pod has this IP.

10.244.0.5

The frontend can call

http://10.244.0.5

But this is **not a good idea**.

Why?

Because Pods are temporary.

If the Pod crashes,

Backend Pod

↓

Crash

↓

New Pod Created

The new Pod might get a different IP.

For example,

Old Pod

10.244.0.5

New Pod

10.244.0.12

Now the frontend is still trying to connect to

10.244.0.5

which no longer exists.

So Kubernetes solves this problem using **Services**.

What is a Service?

Instead of communicating with a Pod directly,

the frontend communicates with a Service.

Example:

backend-service

A Service provides:

- A stable IP (ClusterIP)
- A DNS name

Even if Pods are recreated,

the Service remains the same.

Now the question becomes:

The frontend knows only

backend-service

Who converts this name into an IP address?

The answer is **CoreDNS**.

Step-by-Step Flow

Suppose my frontend application wants to call
`http://backend-service`

Step 1

The frontend sends a DNS request.

"Where is backend-service?"

It doesn't know the IP.

Step 2

The DNS request reaches **CoreDNS**.

Remember when we ran

```
kubectl get pods -A
```

We saw

```
coredns-xxxxx
```

That is the DNS server for the Kubernetes cluster.

Step 3

CoreDNS checks the Kubernetes API Server.

It asks,

Do we have a Service named backend-service?

The API Server checks the cluster information (stored in etcd) and returns the Service details.

Step 4

CoreDNS finds that

backend-service

has a ClusterIP.

For example,

10.96.15.20

It sends this IP back to the frontend.

Step 5

Now the frontend sends the request to

10.96.15.20

This is **not** the Pod IP.

It is the **Service IP (ClusterIP)**.

Step 6

The request reaches the Service.

Now another question arises.

The Service may have three backend Pods.

Backend Pod 1

Backend Pod 2

Backend Pod 3

Which Pod should receive the request?

Step 7

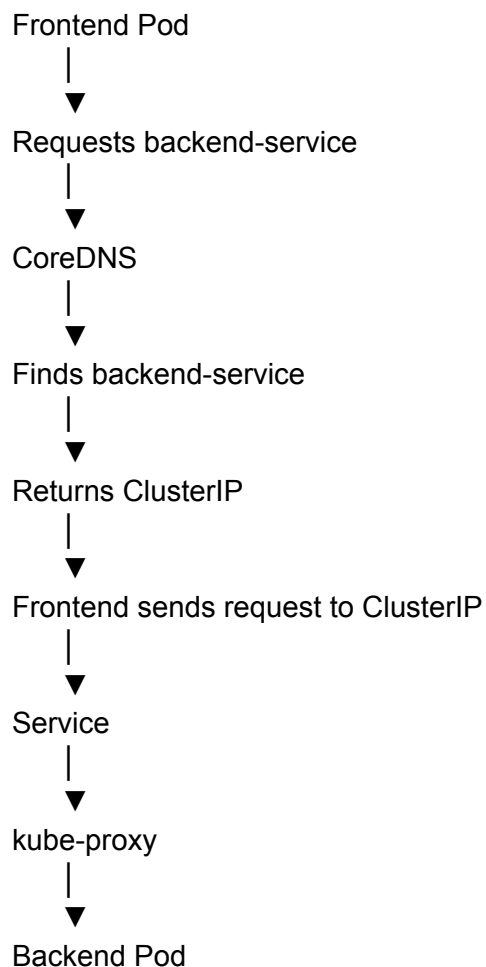
This is where **kube-proxy** comes in.

kube-proxy maintains networking rules on the node.

It forwards the request from the Service to one of the healthy backend Pods.

If three Pods are available, it load-balances the traffic among them.

Final Flow



Why do we need CoreDNS?

Without CoreDNS,

applications would need to remember Pod IP addresses.

Since Pod IPs change whenever Pods are recreated, applications would constantly break.

CoreDNS solves this by allowing applications to communicate using **Service names** instead of Pod IPs.

How can I verify this in my own cluster?

First, I can confirm that CoreDNS is running.

```
kubectl get pods -n kube-system
```

I'll see something like:

```
coredns-589f44dc88-5926g
coredns-589f44dc88-wbdfd
```

This confirms the DNS server is running.

Later, when we deploy two applications, we can verify DNS resolution from inside a Pod:

```
kubectl exec -it <frontend-pod> -- sh
```

Then run:

```
nslookup backend-service
```

or

```
dig backend-service
```

If DNS is working correctly, CoreDNS will return the Service's ClusterIP.

Interview Answer (2–3 Minutes)

"Inside Kubernetes, DNS is handled by CoreDNS. Applications don't communicate using Pod IPs because Pod IPs are temporary and change whenever a Pod is recreated. Instead, applications communicate using Service names like **backend-service**.

When a frontend Pod tries to connect to **backend-service**, it sends a DNS query to CoreDNS. CoreDNS checks the Kubernetes API Server to find the Service information and returns the Service's ClusterIP. The frontend then sends the request to that ClusterIP, and kube-proxy forwards the request to one of the healthy

backend Pods. This allows applications to communicate reliably even if Pods are recreated."

topic-wise commands :-

I recommend creating one section for each Kubernetes object. Here's the list of commands you should have.

1. Cluster Commands

```
kubectl cluster-info  
kubectl get nodes  
kubectl get nodes -o wide  
kubectl describe node <node-name>
```

```
kubectl get ns  
kubectl get all -A  
kubectl get events -A
```

```
kubectl version
kubectl config view
kubectl config current-context
kubectl get componentstatuses # (older clusters)
```

2. Pod Commands

```
kubectl run nginx --image=nginx

kubectl apply -f pod.yaml
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <pod-name>

kubectl logs <pod-name>
kubectl logs -f <pod-name>

kubectl exec -it <pod-name> -- /bin/bash
kubectl exec -it <pod-name> -- /bin/sh

kubectl delete pod <pod-name>

kubectl get pods -w

kubectl top pod
```

3. ReplicaSet Commands

```
kubectl apply -f rs.yaml

kubectl get rs

kubectl describe rs <rs-name>

kubectl get pods

kubectl scale rs <rs-name> --replicas=5

kubectl delete rs <rs-name>

kubectl get rs -o wide
```

4. Deployment Commands

kubectl apply -f deployment.yaml

kubectl get deployments

kubectl describe deployment <deployment-name>

kubectl get rs

kubectl get pods

kubectl scale deployment <deployment-name> --replicas=5

kubectl rollout status deployment <deployment-name>

kubectl rollout history deployment <deployment-name>

kubectl rollout undo deployment <deployment-name>

kubectl set image deployment/<deployment-name> nginx=nginx:1.26

kubectl delete deployment <deployment-name>

5. Namespace Commands

kubectl get ns

kubectl create namespace dev

kubectl delete namespace dev

kubectl get pods -n dev

kubectl apply -f pod.yaml -n dev

6. Service Commands

```
kubectl get svc
```

```
kubectl describe svc <service-name>
```

```
kubectl expose deployment nginx --port=80
```

```
kubectl delete svc <service-name>
```

7. ConfigMap Commands

```
kubectl create configmap app-config --from-literal=ENV=prod
```

```
kubectl get configmap
```

```
kubectl describe configmap app-config
```

```
kubectl delete configmap app-config
```

8. Secret Commands

```
kubectl create secret generic db-secret \  
--from-literal=username=admin \  
--from-literal=password=admin123
```

```
kubectl get secrets
```

```
kubectl describe secret db-secret
```

```
kubectl delete secret db-secret
```

9. Persistent Volume Commands

```
kubectl get pv
```

```
kubectl get pvc
```

```
kubectl describe pv
```

```
kubectl describe pvc
```

kubectl delete pvc <name>

10. Node Commands

kubectl get nodes

kubectl describe node <node-name>

kubectl cordon <node-name>

kubectl uncordon <node-name>

kubectl drain <node-name> --ignore-daemonsets

11. Troubleshooting Commands (Very Important)

kubectl get all

kubectl get events

kubectl describe pod <pod-name>

kubectl logs <pod-name>

kubectl logs -f <pod-name>

kubectl exec -it <pod-name> -- /bin/bash

kubectl top pod

kubectl top node

12. YAML Validation

kubectl apply -f file.yaml

kubectl delete -f file.yaml

kubectl explain deployment

kubectl explain deployment.spec

kubectl explain pod.spec.containers

13. Useful Commands

kubectl get all

kubectl get all -A

kubectl get pods --show-labels

kubectl get pods -o yaml

kubectl get pods -o json

kubectl api-resources

kubectl api-versions

Commands Every Interviewer Expects You to Know

If you remember only these, you're already in a good position:

kubectl get pods

kubectl describe pod

kubectl logs

kubectl exec

kubectl apply

kubectl delete

kubectl get deployments

kubectl rollout status

kubectl rollout undo

kubectl scale

kubectl get svc

kubectl get nodes

kubectl describe node

kubectl get events

```
kubectl get all -A
```

These are the commands you'll use most often in interviews and day-to-day SRE work.

I also have a list of **30–40 Linux commands that almost every Kubernetes/SRE interview expects** (grep, find, ps, top, netstat/ss, curl, nslookup, dig, journalctl, systemctl, etc.). We can prepare those in the same notebook after we finish Kubernetes.